

ECOM EXPERTS

Ucronia studio

Manual de usuario, técnico y de desarrollo

ucronia-studio · 08/09/2026

Contenido

De un libro (o de una idea) al vídeo publicado, sin tocar un editor de vídeo

1. Qué es esto

2. Instalación (una sola vez)

Dónde guarda las cosas

Formato del episodio al montar

Si el antivirus lo borra

Si algo falla

3. Poner las claves de API

4. Crear una serie desde cero

4.1 De dónde sale el contenido

4.2 Qué quieres que sea

La duración la eliges tú

4.3 Qué se genera

4.4 Varias series a la vez

4.6 Idiomas: el sistema completo

4.7 Mejorar cualquier cosa con IA

4.8 Generar imágenes a partir de otra imagen

4.9 Episodio completo

4.10 Si algo se atasca

4.5 Las fotos de la página, no fotos inventadas

5. Tu voz

5.1 Elegir una voz del catálogo

6. Producir un episodio

6.1 Importar el guion

6.1 bis Importar kits escritos con otra IA — un episodio cada vez

6.1 ter Quitar un episodio

6.2 Abrir el episodio

6.3 El orden correcto

6.4 Lo que sale

7. Material propio

8. Título y descripción de YouTube

9. El podcast

9 bis. Acotaciones: «(ríe)», «(susurra)», «(llora)»

10. Ajustes que merece la pena tocar

11. Problemas frecuentes

12. Consejo de producción

De un libro (o de una idea) al vídeo publicado, sin tocar un editor de vídeo

1. Qué es esto

2. Instalación (una sola vez)

Dónde guarda las cosas

Formato del episodio al montar

Si el antivirus lo borra

Si algo falla

3. Poner las claves de API

El previo: ver el episodio sin montarlo

Varias imágenes en un mismo plano

Antes de montar, te avisa de lo que falta

Que el montaje no tarde ocho horas

Remontar ya no cuesta lo mismo

Te dice cuánto va a tardar

Ritmo: lo que más cambia la sensación

Ver un solo plano

Reordenar las imágenes de un plano

El sonido sale al nivel que espera YouTube

Borrador rápido

Montar TODOS los episodios

Encaje de un plano concreto

Detalles que se notan sin saber por qué

Los rotulos, con el diseno que quieras

Subtitulos con la palabra resaltada

Zona segura

Careta y cierre

Buscar y reemplazar — Ctrl+F

Atajos de teclado

Cuanto va a costar — boton 

Plantillas

Que los personajes muevan la boca cuando toca

4. Crear una serie desde cero

4.1 De dónde sale el contenido

4.2 Qué quieres que sea

La duración la eliges tú

4.3 Qué se genera

4.4 Varias series a la vez

4.6 Idiomas: el sistema completo

4.7 Mejorar cualquier cosa con IA

4.8 Generar imágenes a partir de otra imagen

4.9 Episodio completo

4.10 Si algo se atasca

4.5 Las fotos de la página, no fotos inventadas

5. Tu voz

5.1 Elegir una voz del catálogo

6. Producir un episodio

6.1 Importar el guion

6.1 bis Importar kits escritos con otra IA — un episodio cada vez

6.1 ter Quitar un episodio

6.2 Abrir el episodio

6.3 El orden correcto

6.4 Lo que sale

7. Material propio

8. Título y descripción de YouTube

9. El podcast

9 bis. Acotaciones: «(ríe)», «(susurra)», «(llora)»

10. Ajustes que merece la pena tocar

11. Problemas frecuentes

12. Consejo de producción

1. Resumen: qué IA hace qué

2. Google AI Studio — Gemini · Imagen · Veo

Alta

Capa gratuita

Orden de magnitud

Modelos

3. ElevenLabs — la voz del narrador

Alta

Planes

Cómo elegir la voz

⚠ Permisos de la clave (el fallo nº1)

Clonar tu propia voz

Música

4. Anthropic Claude — el cerebro de texto

Alta

Coste

4b. Freepik / Magnific — el "todo en uno"

Alta

Por qué importa aquí el Upscaler

Igualar estilo

Si algo devuelve 404

5. OpenAI — alternativa

Alta

6. fal.ai y Replicate — el catálogo abierto

fal.ai

Replicate

7. Cuánto cuesta un episodio (y cómo gastar poco)

La estrategia que recomiendo

8. Privacidad y buenas prácticas

9. Enlaces rápidos

1. Resumen: qué IA hace qué

2. Google AI Studio — Gemini · Imagen · Veo

Alta

Capa gratuita

Orden de magnitud

Modelos

3. ElevenLabs — la voz del narrador

Alta

Planes

Cómo elegir la voz

⚠ Permisos de la clave (el fallo nº1)

Clonar tu propia voz

Música

4. Anthropic Claude — el cerebro de texto

Alta

Coste

4b. Freepik / Magnific — el "todo en uno"

Alta

Por qué importa aquí el Upscaler

Igualar estilo

Si algo devuelve 404

5. OpenAI — alternativa

Alta

5b. Grok Imagine (xAI) — imagen y vídeo con una clave que quizá ya tienes

6. fal.ai y Replicate — el catálogo abierto

fal.ai

Replicate

7. Cuánto cuesta un episodio (y cómo gastar poco)

7.1 El modelo importa más que el proveedor

7.2 «auto»: que Google elija el barato

7.3 Resolución y audio

7.4 La estrategia que recomiendo

8. Privacidad y buenas prácticas

9. Enlaces rápidos

Hecho con esto

Qué es

La idea que lo hace funcionar

Instalación

Opción A — Descargar la aplicación

Opción B — Desde el código

Configurar las claves

Qué no viene en el repositorio, a propósito

Cómo funciona

1 · Crear la serie

2 · Tu voz

3 · Producir

👁️ Previo: verlo antes de montarlo

Varias imágenes en un mismo plano

Antes de montar, avisa

Los labios, que es donde se nota el truco

Subtítulos con la palabra resaltada

Careta, cierre y zona segura

Buscar y reemplazar

El sonido sale al nivel de YouTube

Borrador rapido

Remontar cuesta segundos, no horas

Y avisa de los planos que se van a hacer eternos

Ritmo

Que no tarde ocho horas

IAs que conecta

Cuánto cuesta

Documentación

Cómo está hecho

Pruebas

Privacidad

Contribuir

Quién lo hace

Licencia

1. Arquitectura

2. El parser (parser.js)

2.1 Gramática reconocida

2.2 Construcción de planos

3. Modelo de datos (data/project.json)

4. Capa de proveedores

4.1 Contrato

4.2 Detalles por proveedor

4.3 Robustez de red (http.js)

5. El motor de montaje (ffmpeg.js)

5.1 Principio de sincronía

5.2 Ken Burns

5.3 Cadena completa

5.4 Rótulos y subtítulos

6. Cola de trabajos (jobs.js)

7. API HTTP

8. Seguridad y coste

9. Pruebas

10. Extensiones previstas

11. El generador de guiones (generator.js)

11.1 Las tres fases

11.2 Normalización defensiva

11.3 Derivados

12. Extracción de texto (extract.js)

13. Clonado de voz (voices.js)

14. Multi-proyecto

15. Empaquetado a .exe

16. Cambios de API absorbidos

17. Lectura de páginas web (web-read.js)

17.1 Texto

17.2 Fotos

17.3 Visión: que la IA mire las fotos

17.4 Del catálogo al plano

18. Duración real vs duración pedida

19. Que el video dure lo que se ha pedido

19.1 La interfaz mandaba siempre 600 segundos

19.2 El modelo se pasa del presupuesto de palabras

20. Por que los textos salian en telegrama

20.1 La causa: 7,9 palabras por plano

20.2 Las reglas empujaban al mismo sitio

20.3 Y algo mas grave: se inventaba datos

20.4 El recorte tambien empujaba al telegrama

21. Formato del episodio en el momento de montar

22. La orientacion era GLOBAL: un proyecto reconfiguraba a los demas

22.1 La prueba

22.2 La causa

22.3 La correccion

23. Encaje: cuando recortar y cuando rellenar

23.1 Por que los clips salieron verticales

23.2 auto era simetrico y no debia serlo

23.3 Bonus: la pantalla de Ajustes prometia algo que el codigo no cumplia

24. Sistema multiidioma (idiomas.js)

24.1 Por que el idioma de Ajustes no hacia nada

24.2 Los idiomas son del PROYECTO

24.3 Donde vive cada traduccion

24.4 La duracion cambia con el idioma

25. Lo demas que se ha anadido

25.1 Imagen de partida ("usar como base")

25.2 "Mejorar con IA" en todo

25.3 Ampliar y rehacer episodios

25.4 Marca de agua / logotipo

25.5 Reiniciar el programa

25.6 Podcast

25.7 Formatos

26. Acotaciones de dialogo (acotaciones.js)

27. Cache de voz global (cachevoz.js)

28. Traducir: el JSON que llegaba cortado

29. Varios kits en un proyecto: anadir episodios en vez de reemplazar

30. Borrar un episodio (DELETE /api/episode/:n)

31. "Se queda parado" en el montaje: no lo estaba, es que no informaba

32. Traducir: las tandas van en paralelo

1. Arquitectura

2. El parser (parser.js)

2.1 Gramática reconocida

2.2 Construcción de planos

3. Modelo de datos (data/project.json)

4. Capa de proveedores

4.1 Contrato

xai y grok: dos identificadores, un solo servicio

labios.js — sincronía labial en los prompts de vídeo

4.2 Detalles por proveedor

4.3 Robustez de red (http.js)

5. El motor de montaje (ffmpeg.js)

5.1 Principio de sincronía

5.2 Ken Burns

5.3 Cadena completa

5.4 Rótulos y subtítulos

6. Cola de trabajos (jobs.js)

7. API HTTP

8. Seguridad y coste

9. Pruebas

10. Extensiones previstas

El encaje "blur" y por qué el montaje iba lento

La dirección de cámara se leía mal

Capturas de la interfaz

Por qué un episodio tardaba ocho horas

server/codecs.js

Varias imágenes en un mismo plano

server/revision.js — mirar antes de gastar horas

El previo — public/previo.js

server/cachemontaje.js — no rehacer lo que no ha cambiado

server/estimar.js — cuánto va a tardar

Ritmo y silencios

Planos que se hacen eternos

Sonoridad: por qué el episodio sale a -14 LUFS

Ken Burns con entrada y salida suaves

El silencio inicial de las locuciones

El modo borrador, y por qué no bastaba con bajar la resolución de salida

Por qué el Ken Burns NO se baja de resolución en el montaje bueno

server/rotulos.js — diseños y subtítulos con resalte

Zona segura

Caretas

Transiciones

server/buscar.js — buscar y reemplazar

server/costes.js — cuanto va a costar

Plantillas

scripts/comprobar-ui.js

Pruebas del motor de montaje

11. El generador de guiones (generator.js)

11.1 Las tres fases

11.2 Normalización defensiva

11.3 Derivados

12. Extracción de texto (extract.js)

13. Clonado de voz (voices.js)

14. Multi-proyecto

15. Empaquetado a .exe

16. Cambios de API absorbidos

17. Lectura de páginas web (web-read.js)

17.1 Texto

17.2 Fotos

17.3 Visión: que la IA mire las fotos

17.4 Del catálogo al plano

- 18. Duración real vs duración pedida
- 19. Que el video dure lo que se ha pedido
- 19.1 La interfaz mandaba siempre 600 segundos
- 19.2 El modelo se pasa del presupuesto de palabras
- 20. Por que los textos salian en telegrama
- 20.1 La causa: 7,9 palabras por plano
- 20.2 Las reglas empujaban al mismo sitio
- 20.3 Y algo mas grave: se inventaba datos
- 20.4 El recorte tambien empujaba al telegrama
- 21. Formato del episodio en el momento de montar
- 22. La orientacion era GLOBAL: un proyecto reconfiguraba a los demas
- 22.1 La prueba
- 22.2 La causa
- 22.3 La correccion
- 23. Encaje: cuando recortar y cuando rellenar
- 23.1 Por que los clips salieron verticales
- 23.2 auto era simetrico y no debia serlo
- 23.3 Bonus: la pantalla de Ajustes prometia algo que el codigo no cumplia
- 24. Sistema multiidioma (idiomas.js)
- 24.1 Por que el idioma de Ajustes no hacia nada
- 24.2 Los idiomas son del PROYECTO
- 24.3 Donde vive cada traduccion
- 24.4 La duracion cambia con el idioma
- 25. Lo demas que se ha anadido
- 25.1 Imagen de partida ("usar como base")
- 25.2 "Mejorar con IA" en todo
- 25.3 Ampliar y rehacer episodios
- 25.4 Marca de agua / logotipo
- 25.5 Reiniciar el programa
- 25.6 Podcast
- 25.7 Formatos
- 26. Acotaciones de dialogo (acotaciones.js)

27. Cache de voz global (cachevoz.js)

28. Traducir: el JSON que llegaba cortado

29. Varios kits en un proyecto: anadir episodios en vez de reemplazar

30. Borrar un episodio (DELETE /api/episode/:n)

31. "Se queda parado" en el montaje: no lo estaba, es que no informaba

32. Traducir: las tandas van en paralelo

33. Video: el catalogo de modelos (videomodelos.js)

33.1 auto: que la API elija el barato

33.2 Los modelos caros de fabrica se cambian al arrancar

33.3 Cada familia habla distinto

33.4 El video de fal va por COLA

33.5 Parametros que Veo retira, y el filtro de contenido

34. Aviso de version nueva (actualizaciones.js)

Made with this

What it is

The idea that makes it work

Installation

Option A — Download the app

Option B — From source

Setting up API keys

How it works

1 · Create the series

2 · Your voice

3 · Produce

👁 Preview: see it before rendering it

Several images in one shot

It warns you before assembling

Lip sync, which is where the trick shows

Subtitles with the spoken word highlighted

Intro, outro and safe area

Find and replace

Audio comes out at the level YouTube expects

Quick draft

Re-assembling costs seconds, not hours

And it flags shots that will drag

Pacing

Making it not take eight hours

AI providers

What it costs

Documentation

How it's built

Tests

Privacy

Contributing

Who makes it

License

De un libro (o de una idea) al vídeo publicado, sin tocar un editor de vídeo

1. Qué es esto

Un programa que se ejecuta **en tu ordenador** y cubre toda la cadena de producción de un canal de YouTube o un podcast:

1. **Escribe los guiones** a partir de un libro, un PDF, un EPUB, un artículo o simplemente un tema.
2. Los trocea en **planos**, y para cada plano te deja: - generar varias **imágenes** con IA y quedarte con la que te guste, - **ampliarlas y retocarlas** con Magnific, - generar la **voz en off** —con tu propia voz clonada, si quieres—, - **animar** la imagen en un clip de 5–8 s (opcional), - poner **música** que baja sola cuando habla el narrador,
3. Y **monta el MP4** con los rótulos quemados, subtítulos `.srt` y capítulos de YouTube.

La clave del sistema: la duración de cada plano no la pones tú, la marca la locución. Si el narrador tarda 43 segundos en decir su párrafo, el plano dura 43 segundos más los silencios. Todo encaja solo.

Si ya tienes tus guiones escritos, puedes saltarte el paso 1: ponlos en el formato de marcadores (§5) y pulsa *Importar*.

2. Instalación (una sola vez)

Ejecuta `GeneradorContenido-1.0.0-x64.exe` (el instalador) o `GeneradorContenido-portable-1.0.0.exe` si prefieres no instalar nada.

Es una aplicación de Windows normal: se abre en **su propia ventana**, con su icono y su menú. No hay consola negra, no hay navegador, no hay que escribir ninguna dirección. **No necesitas instalar nada más** — ni Node.js, ni ffmpeg: van dentro.

Para cerrarla: la X de la ventana, como cualquier programa.

Windows SmartScreen puede avisar de que el programa no está firmado digitalmente. Es normal: firmar cuesta cientos de euros al año. Pulsa **Más información** → **Ejecutar de todas formas**.

Dónde guarda las cosas

Junto al ejecutable: `.env` (tus claves), `data/` (proyectos y material) y `output/` (los MP4). Si instalaste en *Archivos de programa*, que es de solo lectura, se usa `DocumentosGenerador de Contenido`.

Los tienes a un clic en el menú **Archivo** → **Carpeta de entregas / de material / de trabajo**.

Formato del episodio al montar

Junto al botón **5 · MONTAR EPISODIO** hay un desplegable **Formato del episodio**:

Opción	Para qué
Horizontal 16:9	YouTube
Vertical 9:16	TikTok, Reels, Shorts
Cuadrado 1:1	Instagram
Los dos	Saca dos ficheros: horizontal y vertical
Los tres formatos	Los tres a la vez

Afecta **solo a ese montaje**, así que puedes sacar el mismo episodio en horizontal para YouTube y volver a montarlo en vertical para TikTok sin tocar nada más. Si lo dejas en (*el de Ajustes*) usa el que tengas en **Ajustes** → **Montaje**, que es el que se pone solo al elegir el formato al crear la serie.

Si el antivirus lo borra

Puede pasar. ESET, por ejemplo, borra el instalador y el portable diciendo `Backdoor.PHP/Webshe11.NBV`. **Es un falso positivo**, y se comprobó así:

- El escáner de línea de comandos del propio ESET dio **0 detecciones** sobre el código fuente (39 ficheros), sobre el programa sin empaquetar (610 objetos) y sobre los tres instaladores recién generados.
- Dentro del `.exe` no hay **ni un solo fichero PHP**, ni las cadenas `<?php`, `shell_exec`, `passthru(`, `base64_decode` ni `eval($_`.

El borrado venía marcado con `@`, que en ESET significa **veredicto de reputación en la nube**, no de firma de virus. Un ejecutable autoextraíble recién compilado y **sin firmar** tiene reputación cero, y el nombre «webshell» sale porque el programa hace justo eso: levanta un servidor local y ejecuta `ffmpeg`.

Tres salidas:

- Usa el ZIP** (`GeneradorContenido-1.0.0-x64.zip`). Descomprímelo donde quieras y ejecuta el `.exe` de dentro. No es autoextraíble, así que no dispara la heurística.
- Añade una exclusión** en tu antivirus para la carpeta del programa.
- Reporta el falso positivo**: ESET lo recoge en `samples@eset.com` (asunto *False Positive*, el fichero en un ZIP con contraseña `infected`). Lo suelen corregir en 24-48 h.

Si algo falla

Menú **Ayuda** → **Diagnóstico**: te dice si el motor de vídeo funciona, qué claves tienes puestas y dónde está tu carpeta de trabajo.

3. Poner las claves de API

Ve a la pestaña **Ajustes** → **Claves de API**, pega cada clave y pulsa *Guardar*. Se escriben en el fichero `.env` de la carpeta y **no salen de tu ordenador**.

No hacen falta todas. Con **una** ya puedes empezar. La combinación mínima recomendada:

Quiero...	Necesito
Solo probar el montaje	ninguna (usa imágenes tuyas subidas a mano)
Imágenes + montaje	GEMINI_API_KEY
Lo anterior + voz profesional	+ ELEVENLABS_API_KEY
Lo anterior + vídeo animado	ya la tienes (Veo va con la de Gemini)
Lo anterior + música	ya la tienes (ElevenLabs Music)

Cómo darse de alta en cada servicio: [03_GUIA_IAS_Y_ALTAS.md](#).

4. Crear una serie desde cero

Pestaña **Crear**.

4.1 De dónde sale el contenido

Opción	Cuándo usarla
Un tema	No tienes material. Describe de qué va y la IA lo escribe con su propio conocimiento. Sé concreto: «la historia de la penicilina, del accidente de Fleming a la producción masiva en la guerra» sale mucho mejor que «la penicilina».
Un fichero	Tienes un libro, un informe, unos apuntes. Admite PDF, DOCX, EPUB, TXT, MD, HTML y RTF .
Texto pegado	Un artículo, un capítulo, unas notas.
Una dirección web	Una entrada de blog, una ficha de producto, un artículo. Pega una URL por línea si quieres varias. Se limpia el menú, el pie y los banners, y queda solo el contenido. Además se traen las fotos de la propia página (ver 4.5).
Una carpeta de módulo	Para manuales de software: lee la documentación y las opciones del programa y escribe el vídeo de ayuda, pidiéndote las capturas que hacen falta.

Un libro entero no se manda de golpe: se resume por partes, luego se diseña la escaleta, y después se escribe episodio a episodio. Los PDF se envían tal cual al modelo, porque Gemini y Claude los leen mejor que cualquier extractor.

El .doc antiguo no se puede leer. Ábrelo en Word y guárdalo como .docx o .pdf. **Un PDF escaneado** (páginas que son fotos) tampoco tiene texto: el programa te avisa. Usa el original o pásalo por un OCR.

4.2 Qué quieres que sea

Formato	Qué produce
Serie documental	Narrador único, tono cinematográfico, escenas dramatizadas y rótulos con fechas
Vídeo de YouTube	Gancho fuerte en los primeros 15 s, ritmo ágil, frases cortas, segunda persona
Videoensayo	Tesis, desarrollo y conclusión. Reflexivo y personal
Podcast narrativo	Documental sonoro. El diseño de sonido sustituye a la imagen
Curso o formación	Objetivos de aprendizaje, ejemplos, resumen por lección
Vídeo corto vertical	TikTok, Reels y Shorts. Vertical, sin presentación, un solo mensaje, subtítulos quemados
Vídeo de producto	Ficha de tienda: qué problema resuelve, para quién es y dónde comprarlo
Manual de un programa	Tutorial con capturas reales de pantalla, no imágenes inventadas
Ficción sonora	Narrador y escenas con diálogo, ambiente y foley

Cada formato se autoconfigura. Al elegirlo se ponen solos la orientación (vertical u horizontal), la resolución, la duración típica, los subtítulos y el aire entre planos. No tienes que acordarte de que TikTok es vertical ni de que un manual no lo es.

La duración la eliges tú

El desplegable **Duración de cada pieza** solo ofrece lo que tiene sentido en el formato elegido: en «vídeo corto vertical» van de 15 segundos a 5 minutos, y en «serie documental» de 5 minutos a 1 hora. Al cambiar de formato se pone sola la duración típica de ese formato — 21 s en redes, 10 min en YouTube, 15 min en documental — y a partir de ahí eliges la que quieras.

Si en un formato de redes pasas de **1 minuto**, la nota de debajo se pone en color de aviso: no está prohibido, pero a partir de ahí se desploma el porcentaje de gente que llega al final.

La duración se pide en **segundos**, no en minutos: un TikTok dura 21 segundos y eso en minutos no se puede escribir. El programa calcula cuántas palabras caben de verdad en ese tiempo —descontando el silencio de cada corte—, se las pide a la IA, y **luego las cuenta**. Si el guion se ha pasado, se lo devuelve para que lo recorte antes de grabar nada. Un vídeo de 21 s sale de 18-24 s, no de cinco minutos.

Red	Lo que mejor funciona	Límite técnico
TikTok	15-34 s (21 es el punto dulce)	10 min
Instagram Reels	7-30 s	3 min
YouTube Shorts	15-60 s	3 min
Facebook Reels	15-45 s	90 s

Más el número de episodios, la duración, el idioma, el público y el tono. En **Instrucciones extra** puedes pedir lo que quieras: «que cada episodio empiece con una historia personal», «evitar tecnicismos», «citar siempre las fuentes».

Pulsa **Generar los guiones**. Tarda unos minutos (una llamada para la escaleta y otra por episodio); el avance se ve en **Trabajos**. Al terminar, la serie queda abierta y lista en **Episodios**.

4.3 Qué se genera

Un kit de seis ficheros `.md` en la carpeta de la serie: guion de vídeo, guion de podcast, prompts de imagen, prompts de vídeo, biblia de producción e índice. **Puedes editarlos a mano** y volver a pulsar *Importar*: el programa los relee.

4.4 Varias series a la vez

Al final de la pestaña **Crear** están todas tus series. *Abrir* cambia de una a otra; cada una tiene su material y su carpeta de entregas independientes.

4.6 Idiomas: el sistema completo

Cada serie tiene **un idioma principal** —en el que la IA escribe— y los **adicionales** que quieras. No se configura en Ajustes: es de cada proyecto, así que un documental en castellano y un TikTok en inglés conviven sin pisarse.

Al crear la serie eliges el principal y marcas los adicionales. El **texto se traduce solo, de todos los episodios**, nada más crearla: traducir es barato y así el proyecto nace completo.

Lo demás se genera cuando tú lo pidas, porque cuesta dinero:

	Cuándo
Texto	Siempre, en todos los idiomas
Imágenes	Se comparten (una imagen no habla)
Locución	Solo en los idiomas que marques
Vídeo	Solo en los idiomas que marques

En **Episodios** hay una barra con los idiomas. Al pulsar uno **cambia todo lo que ves**: narración, rótulos, título y locución. Lo que edites ahí no toca el original — la estrella (★) marca el principal.

Desde esa barra puedes **traducir el episodio**, **traducir a todos los idiomas** y abrir **Idiomas del proyecto...** para añadir o quitar.

Montar en un idioma es simplemente tenerlo seleccionado y pulsar *MONTAR EPISODIO*. El fichero sale con el código en el nombre: `EP01_TITULO.en.mp4`. El principal no lo lleva.

La duración del vídeo cambia con el idioma, porque la marca la locución: una misma frase no dura lo mismo en castellano que en alemán. Es normal que el mismo episodio dure 10 s en uno y 15 en otro.

4.7 Mejorar cualquier cosa con IA

Junto a cada campo hay dos botones:

- **Mejorar con IA** — pule lo que ya hay.
- **Proponer otra/o** — ignora lo que hay y propone algo distinto. Útil cuando el texto no te convence y no quieres una variación de lo mismo.

Funciona en la narración, los rótulos, el prompt de imagen, el prompt de vídeo, la escena y el título. **Siempre te enseña la propuesta antes de aplicarla:** puedes rechazarla.

4.8 Generar imágenes a partir de otra imagen

En cada plano, debajo de *Generar imágenes*:

- **USAR como base la imagen actual** — la IA parte de la que ya tiene el plano y la modifica. Sirve para cambiar el estilo sin perder el encuadre.
- **Subir imagen de base...** — usa una imagen tuya como punto de partida.
- El campo de al lado es **qué hacerle**: «cambiar a acuarela», «añadir niebla», «quitar el texto». Si lo dejas vacío se rehace con el estilo de la serie.

4.9 Episodio completo

- **Ampliar episodio...** — escribe planos NUEVOS al final, continuando la historia. No toca nada de lo que ya hay.
- **Rehacer todo el episodio...** — vuelve a generar imágenes, voz y montaje de una pasada. Respeta los planos que hayas **fijado** con el candado, así que fija primero los que ya te valgan.
- **Marca de agua...** — sube un logotipo (mejor PNG con transparencia) y elige posición, tamaño y opacidad. Puedes ponerlo para todo el proyecto o solo para un episodio; la del episodio manda.
- **Podcast desde documento...** — sube un PDF, DOCX o EPUB, o pega texto. La IA **no lo lee tal cual**: lo reescribe para que suene bien dicho en voz alta (frases más cortas, siglas deletreadas, cifras escritas como se dicen), respetando el contenido.

4.10 Si algo se atasca

En **Ajustes** → **Mantenimiento**:

- **Recargar proyectos** — vuelve a leer la carpeta de datos sin cerrar nada. Úsalo si has copiado proyectos a mano y no aparecen.
- **Reiniciar el programa** — cierra y vuelve a abrir el motor. No se pierde nada: todo está guardado en disco.

4.5 Las fotos de la página, no fotos inventadas

Cuando la fuente es una **dirección web**, los planos no se inventan: se usan las fotos que ya están en esa página. Para una ficha de producto esto no es un detalle, es la diferencia entre un vídeo que sirve y uno que no — una foto inventada de tu producto no vale para nada.

Cómo funciona, en orden:

1. **Se buscan las fotos** de la página: la imagen social, las que declara la tienda en sus datos de producto y las que están dentro del artículo. Se tiran logos, iconos, banderas, botones de pago y píxeles de seguimiento.
2. **Se piden en grande.** Muchas webs sirven miniaturas de 250 píxeles; el programa reescribe la dirección para traer la versión de 1024, porque el zoom lento del montaje se hace *encima* de la foto y una miniatura se ve pixelada.
3. **La IA las mira, una a una.** Aquí está la diferencia. El texto alternativo de una foto suele estar vacío (en la prueba con Wikipedia lo estaba en las 22), así que sin mirarlás la IA escribiría a ciegas. Con visión, sabe que la foto 4 es «un científico con bata blanca examinando una placa de Petri» y la 6 «un diagrama del experimento en ratones».
4. **Se descarta lo que no sirve.** Lo que la vista reconoce como logo, banner o anuncio se cae aquí, aunque el nombre del fichero no lo delatara.

5. **Se escribe el guion sobre esas fotos.** La IA elige cuál va en cada plano y **narra lo que se ve en ella**. Si el plano es el primer plano del hongo, la voz habla del hongo, no de fechas.
6. **Si algo no existe en la página**, y solo entonces, ese plano se marca como imagen de IA y se genera. Por ejemplo: la página cuenta que la cepa buena salió de un melón podrido, pero no tiene ninguna foto del melón.

En el editor, esos planos salen marcados como **«foto de la web»**, con un enlace al original y con la descripción de lo que la IA ha visto. Si alguna no te convence, súbela tú o génerala con IA desde el mismo sitio: no hay nada bloqueado.

Necesita un proveedor con visión configurado en *Ajustes > Proveedores > Visión* (Gemini o Claude sirven). Sin él, las fotos se bajan igual, pero la IA escribe sin verlas y el guion encaja bastante peor.

5. Tu voz

Pestaña **Mi voz**. Grabas cinco párrafos desde el navegador y el programa crea una **voz clonada tuya** en ElevenLabs para locutar toda la serie.

1. Pulsa **Grabar** en el primer párrafo, léelo, pulsa **Detener**.
2. Repite con los cinco. Están escritos a propósito para cubrir registros distintos —narración neutra, preguntas, cifras, tensión y cierre cálido—: una muestra con variedad de entonación clona mucho mejor que leer siempre igual.
3. Cuando la barra llegue al mínimo (60 s; lo ideal son 150 s), escribe un nombre y pulsa **Crear mi voz clonada**.

Al terminar, esa voz queda seleccionada automáticamente como la del narrador.

Consejos que se notan: sitio silencioso, micrófono a un palmo de la boca, y lee **con la entonación que quieres para tus vídeos** — el clon copia tu forma de narrar, no solo tu timbre.

El clonado necesita ElevenLabs con plan de pago (desde ~5 €/mes). Tus grabaciones se quedan en `data/voz/` y solo se envían cuando pulsas el botón.

Si dice que falta un permiso: no es el plan, es la clave. Al crearla, ElevenLabs deja restringir lo que puede hacer. Ve a <https://elevenlabs.io/app/settings/api-keys>, pulsa el lápiz de tu clave y en *Restrict Key* activa **Voices** → **Write** (clonar) y **Text to Speech** → **Access** (locutar). Lo tienes desplegado en Ajustes → Voz en off. Lo más simple: crea una clave nueva sin restringir nada.

5.1 Elegir una voz del catálogo

Si prefieres no clonar la tuya: **Ajustes** → **Voz en off** → **Cargar voces de ElevenLabs**, elige una y escúchala. Tus voces clonadas salen primero en la lista.

Ajustes recomendados para narración: **Estabilidad** 0.50 · **Similitud** 0.75 · **Estilo** 0.10 · **Velocidad** 0.95.

6. Producir un episodio

6.1 Importar el guion

Pestaña **Episodios** → **Importar / recargar guion**. Aparecen los episodios con su barra de progreso.

Si retocas un `.md`, vuelve a pulsarlo: **conserva las imágenes, clips y voces ya generados** de los planos que no han cambiado.

¿Quieres que un plano no se toque nunca al reimportar? Marca la casilla **fijar** en su cabecera.

El formato de marcadores, por si escribes los guiones tú:

```
# EPISODIO 1 — TÍTULO DEL EPISODIO
**~15 min · subtítulo**

### Nombre del bloque — 0:00
[MÚSICA] tono musical
[IMAGEN IA] prompt en inglés del plano
[VOZ] la narración que se oye mientras se ve ese plano
[RÓTULO] texto que aparece en pantalla
[ESCENA] dramatización concreta
```

Cada `[IMAGEN IA]` abre un plano; todo lo que va detrás pertenece a ese plano hasta el siguiente.

6.1 bis Importar kits escritos con otra IA — un episodio cada vez

Botón **Importar kit de otra IA...** Es para guiones que vienen de fuera: escritos con ChatGPT, Gemini, a mano, o de una temporada anterior. **No genera nada ni gasta un céntimo.**

Lo importante: si ya tienes un proyecto abierto, el kit se **AÑADE** como un episodio más. No reemplaza nada. Para empezar de cero hay que pulsar **Cerrar proyecto** antes (o elegir *Empezar un proyecto nuevo* en el desplegable).

El número de episodio sale, por este orden:

1. El que escribas tú en el campo *Nº de episodio*.
2. El que se deduzca del **nombre de la carpeta**: `LA_OTRA_LINEA_EP02`, `T2E05`, `capitulo-3`, `PARTE 7`, `guion_episodio_12`.
3. Si no hay pista, va **detrás del último** que tengas.

Se deduce del nombre de la carpeta y **no de la cabecera del guion** a propósito: prácticamente todos los kits generados por IA empiezan por `# EPISODIO 1` aunque sean el séptimo de la serie.

Antes de importar, **Ver qué contiene** te dice exactamente a qué número va a ir y por qué.

Cada kit importado se guarda en su propia subcarpeta dentro de `guion/`, con un fichero `_kit.json` que recuerda su número. Así puedes recargar el guion las veces que quieras sin que los episodios se pisen entre sí.

6.1 ter Quitar un episodio

Pasa el ratón por la tarjeta del episodio: aparece una **X** arriba a la derecha.

Borra el episodio **y el guion del que salió**, porque si no volvería a aparecer la próxima vez que recargues. Pregunta aparte si quieres borrar también sus imágenes, locuciones y montajes; por defecto **se conservan**, que es lo que casi

siempre quieres cuando borras un episodio para volver a importarlo corregido.

6.2 Abrir el episodio

Pulsa la tarjeta. Verás:

- Una **línea de tiempo** arriba: cada bloque es un plano, con su anchura proporcional a su duración. Azul = tiene imagen, morado = tiene clip, apagado = no tiene voz. Púlsalo para saltar a ese plano.
- La **lista de planos**, cada uno con su prompt de imagen, su prompt de vídeo, su narración y sus rótulos.

6.3 El orden correcto

Los botones grandes del panel superior están numerados a propósito:

1 · Imágenes → 1b · Ampliar → 2 · Voz → 3 · Animar → 4 · Música → 5 · MONTAR

Paso 1 — Imágenes. Genera 3 variantes de cada plano que no tenga ninguna. Cuando terminen, repasa la lista y **haz clic en la miniatura que prefieras** de cada plano (la elegida sale con borde dorado). La **x** de la esquina borra una variante.

¿No te convence ninguna? Edita el prompt en el cuadro de texto y vuelve a pulsar *Generar imágenes*, o pulsa **Mejorar prompt con IA** y deja que Claude lo reescriba (te pregunta qué quieres cambiar).

Paso 1b — Ampliar (Magnific). Pasa las imágenes elegidas por el upscaler.

Esto importa más de lo que parece: el Ken Burns **hace zoom sobre la imagen fija**, y si la imagen no tiene resolución de sobra ese zoom se ve blando. Con 2x sobra para 1080p; usa 4x solo si vas a 4K.

En cada plano tienes además dos botones: - **Reiluminar** — cambia la luz del plano sin regenerarlo (hora del día, dramatismo). Te pregunta cómo la quieres. - **Igualar estilo** — impone el look de otro plano ya aprobado del mismo episodio. Es la forma más fiable de que toda la serie parezca de la misma mano.

Los retoques **no borran el original**: se añaden como una variante nueva y quedan seleccionados. Siempre puedes volver atrás pulsando la miniatura anterior.

Paso 2 — Voz. Locuta el texto **[VOZ]** de cada plano. Los planos sin narración (cabeceras, transiciones) se saltan solos y usan la duración sugerida por el guion.

Al terminar verás que la línea de tiempo cambia: ahora las anchuras son las reales.

Paso 3 — Animar (*opcional, es el paso caro*). Convierte la imagen elegida en un clip con movimiento real. Tarda varios minutos por plano.

Si te lo saltas no pasa nada: el montaje aplica un movimiento de cámara suave (Ken Burns) leyendo la indicación del guion — «dolly-in» hace un acercamiento, «paneo de izquierda a derecha» hace un barrido, «tilt ascendente» sube. Para un documental queda perfectamente digno y es gratis.

Estrategia razonable: anima solo los 2–3 planos más espectaculares del episodio y deja el resto en Ken Burns.

Si generas un clip y luego no te gusta, pulsa **Usar solo la imagen**.

Paso 4 — Música. Genera un lecho instrumental. El texto de descripción sale de las indicaciones **[MÚSICA]** del propio guion; puedes editarlo. También puedes subir un MP3 tuyo.

Paso 5 — Montar. Encola el montaje. Ve a la pestaña **Trabajos** para ver el progreso en directo. Un episodio de 15 minutos tarda entre 8 y 25 minutos según tu ordenador.

6.4 Lo que sale

En la carpeta `output/` (pestaña **Entregas**):

Fichero	Qué es
<code>EP01_...mp4</code>	El episodio, 1920×1080, con rótulos quemados
<code>EP01_...srt</code>	Subtítulos de la narración (súbelos a YouTube)
<code>EP01_...capitulos.txt</code>	Capítulos, pégalos al final de la descripción
<code>EP01_...rotulos.ass</code>	Los rótulos por separado, por si retocas en otro editor

7. Material propio

Cada plano tiene **Subir imagen** y **Subir clip**. Úsalo para:

- Los **mapas** ya rotulados de la saga (`Mapa_*.png`) — súbelos al plano donde el guion los pida.
- Imágenes que hayas generado en Midjourney y no en el programa.
- Metraje de archivo.

Lo subido pasa a ser la selección de ese plano automáticamente.

8. Título y descripción de YouTube

Botón **Título y descripción** del panel superior. Claude lee la narración completa del episodio y devuelve título con gancho, descripción de tres párrafos y 15 etiquetas. Se guardan en el episodio y puedes editarlos.

9. El podcast

Botón **Exportar podcast**. Usa `02_GUION_PODCAST.md` (narración algo más larga, sin rótulos porque en audio las fechas se dicen en voz alta) y saca un MP3 con la misma voz y la misma música.

Las locuciones se guardan en caché **a nivel de proyecto**: la misma frase con la misma voz se graba una sola vez. Si ya has locutado el vídeo, exportar el podcast **no vuelve a llamar a la API** ni te cobra otra vez; y al revés. La caché se invalida sola cuando cambias el texto o la voz, que es justo cuando hace falta rehacer el audio.

En la barra de Episodios verás algo como `312 locuciones guardadas (48 MB)`. Púlsalo para vaciarlas si has cambiado una voz y quieres rehacer todos los audios de verdad.

9 bis. Acotaciones: «(ríe)», «(susurra)», «(llora)»

En los guiones de diálogo aparecen constantemente indicaciones de lo que el personaje **hace**, no de lo que dice:

ELENA: (ríe) No, no, déjame que te cuente...
BRUNO: [suspira] Ya... *baja la voz* eso no me lo esperaba.

Eso **no es texto para locutar**. Antes se mandaba tal cual y el locutor lo leía en voz alta. Ahora el programa las reconoce en las tres formas en que se escriben — (entre paréntesis) , [entre corchetes] y *entre asteriscos* — y hace una de dos cosas según el modelo de voz:

Modelo de ElevenLabs	Qué hace
eleven_v3 (y cualquier *_v3)	Convierte la acotación en su etiqueta de audio ([laughs] , [sighs] , [whispers] ...) y el locutor la actúa de verdad.
eleven_multilingual_v2 y anteriores (el de por defecto)	La quita del texto y en su lugar mueve el tono de esa línea: un susurro baja estabilidad y velocidad, un grito sube el estilo.

Reconoce: risa, sonrisa, suspiro, llanto, susurro, grito, duda, nervios, emoción, tristeza, enfado, ironía, curiosidad, carraspeo y pausa. En castellano y en inglés.

Dos detalles que importan:

- Un paréntesis **normal** no se toca. Roma cayó en 476 (según los manuales) se locuta entero: solo se retiran los paréntesis que contienen una acotación reconocida y son cortos.
- Las acotaciones **no cuentan como palabras habladas**, así que ya no descuadran el cálculo de duración. Un diálogo con muchas acotaciones salía corto aunque el contador dijera lo contrario.

Si quieres que el locutor las actúe, pon eleven_v3 en Ajustes → Modelos → *Modelo de voz*.

10. Ajustes que merece la pena tocar

Ajustes → Montaje:

Ajuste	Para qué
Silencio antes / después de la voz	Da aire. 0.35 / 0.65 s va bien. Súbelos si el ritmo te parece atropellado.
Duración mínima de plano	Ningún plano baja de aquí aunque no tenga voz.
Transición	<i>Fundido a negro</i> (documental) o <i>Corte seco</i> (más nervioso).
Zoom de Ken Burns	1.16 es sutil. Por encima de 1.3 se nota demasiado.
Volumen de la música	0.14 deja la voz muy por delante.
Atenuar música al hablar	Déjalo en <i>Sí</i> : baja la música automáticamente en cada frase.
Preset x264	veryfast para pruebas, medium para la versión final.
Calidad CRF	19 es alta. 23 pesa la mitad y casi no se nota en YouTube.

Ajustes → Estilo visual: el sufijo de estilo se añade a **todos** los prompts de imagen. Es lo que mantiene la coherencia visual de la serie. Cámbialo solo si quieres cambiar el look de toda la temporada.

11. Problemas frecuentes

«Falta la clave de X» → Ajustes → Claves de API.

Una imagen sale rara o con texto raro. Los modelos meten texto inventado. Añade al prompt negativo **text, letters, signage** y regenera.

El montaje falla en un plano. Mira el mensaje en Trabajos. Casi siempre es una imagen corrupta: bórrala con la **x**, regenera y vuelve a montar.

Los rótulos no aparecen en el MP4. El programa avisa en el log y monta sin ellos en vez de perder el trabajo. Suele ser que la fuente no existe: en Ajustes → Montaje cambia *Fuente de los rótulos* a **Georgia** o **Times New Roman**. El **.ass** siempre se exporta aparte.

Tarda muchísimo. Baja el preset a **veryfast** y sube el CRF a 23 mientras pruebas. Deja **medium** /19 solo para la versión que vas a publicar.

Un trabajo se ha quedado colgado. Botón *Cancelar* en Trabajos. Los de vídeo tienen corte automático a los 15 minutos.

Quiero empezar de cero. Usa **Cerrar proyecto** en la pantalla de Episodios. A partir de ahí, lo siguiente que importes o crees será un proyecto nuevo.

He importado un kit y me ha borrado el episodio anterior. Eso ya no pasa: importar con un proyecto abierto **añade** un episodio. Si de verdad quieres empezar de cero, cierra el proyecto antes o elige *Empezar un proyecto nuevo* en el desplegable del panel de importación.

«La IA no devolvió un JSON válido al traducir». Era un episodio con bloques muy largos: se mandaban doce de golpe y la respuesta se cortaba a mitad. Ahora las tandas se miden en palabras, no en bloques, y si aun así viene cortada la tanda se parte en dos y se reintenta. Un bloque que siga sin poder traducirse se queda en el idioma original y se avisa en el log, en vez de tumbar el episodio entero.

12. Consejo de producción

Sigue la biblia (§7): **produce los 3 primeros episodios antes de estrenar**. Y monta primero un episodio entero con Ken Burns y sin animar nada — así ves el ritmo real antes de gastarte el dinero en clips de vídeo, que es con diferencia lo más caro.

GENERADOR DE CONTENIDO — Manual de usuario

De un libro (o de una idea) al vídeo publicado, sin tocar un editor de vídeo

1. Qué es esto

Un programa que se ejecuta **en tu ordenador** y cubre toda la cadena de producción de un canal de YouTube o un podcast:

1. **Escribe los guiones** a partir de un libro, un PDF, un EPUB, un artículo o simplemente un tema.
2. Los trocea en **planos**, y para cada plano te deja: - generar varias **imágenes** con IA y quedarte con la que te guste, - **ampliarlas y retocarlas** con Magnific, - generar la **voz en off** —con tu propia voz clonada, si quieres—, - **animar** la imagen en un clip de 5–8 s (opcional), - poner **música** que baja sola cuando habla el narrador,
3. Y **monta el MP4** con los rótulos quemados, subtítulos `.srt` y capítulos de YouTube.

La clave del sistema: la duración de cada plano no la pones tú, la marca la locución. Si el narrador tarda 43 segundos en decir su párrafo, el plano dura 43 segundos más los silencios. Todo encaja solo.

Si ya tienes tus guiones escritos, puedes saltarte el paso 1: ponlos en el formato de marcadores (§5) y pulsa *Importar*.

2. Instalación (una sola vez)

Ejecuta `GeneradorContenido-1.0.0-x64.exe` (el instalador) o `GeneradorContenido-portable-1.0.0.exe` si prefieres no instalar nada.

Es una aplicación de Windows normal: se abre en **su propia ventana**, con su icono y su menú. No hay consola negra, no hay navegador, no hay que escribir ninguna dirección. **No necesitas instalar nada más** — ni Node.js, ni ffmpeg: van dentro.

Para cerrarla: la X de la ventana, como cualquier programa.

Windows SmartScreen puede avisar de que el programa no está firmado digitalmente. Es normal: firmar cuesta cientos de euros al año. Pulsa **Más información** → **Ejecutar de todas formas**.

Dónde guarda las cosas

Junto al ejecutable: `.env` (tus claves), `data/` (proyectos y material) y `output/` (los MP4). Si instalaste en *Archivos de programa*, que es de solo lectura, se usa `DocumentosGenerador de Contenido`.

Los tienes a un clic en el menú **Archivo** → **Carpeta de entregas / de material / de trabajo**.

Formato del episodio al montar

Junto al botón **5 · MONTAR EPISODIO** hay un desplegable **Formato del episodio**:

Opción	Para qué
Horizontal 16:9	YouTube
Vertical 9:16	TikTok, Reels, Shorts
Cuadrado 1:1	Instagram
Los dos	Saca dos ficheros: horizontal y vertical
Los tres formatos	Los tres a la vez

Afecta **solo a ese montaje**, así que puedes sacar el mismo episodio en horizontal para YouTube y volver a montarlo en vertical para TikTok sin tocar nada más. Si lo dejas en (*el de Ajustes*) usa el que tengas en **Ajustes** → **Montaje**, que es el que se pone solo al elegir el formato al crear la serie.

Si el antivirus lo borra

Puede pasar. ESET, por ejemplo, borra el instalador y el portable diciendo `Backdoor.PHP/Webshe11.NBV`. **Es un falso positivo**, y se comprobó así:

- El escáner de línea de comandos del propio ESET dio **0 detecciones** sobre el código fuente (39 ficheros), sobre el programa sin empaquetar (610 objetos) y sobre los tres instaladores recién generados.
- Dentro del `.exe` no hay **ni un solo fichero PHP**, ni las cadenas `<?php`, `shell_exec`, `passthru(`, `base64_decode` ni `eval($_`.

El borrado venía marcado con `@`, que en ESET significa **veredicto de reputación en la nube**, no de firma de virus. Un ejecutable autoextraíble recién compilado y **sin firmar** tiene reputación cero, y el nombre «webshell» sale porque el programa hace justo eso: levanta un servidor local y ejecuta `ffmpeg`.

Tres salidas:

1. **Usa el ZIP** (`GeneradorContenido-1.0.0-x64.zip`). Descomprímelo donde quieras y ejecuta el `.exe` de dentro. No es autoextraíble, así que no dispara la heurística.
2. **Añade una exclusión** en tu antivirus para la carpeta del programa.
3. **Reporta el falso positivo**: ESET lo recoge en `samples@eset.com` (asunto *False Positive*, el fichero en un ZIP con contraseña `infected`). Lo suelen corregir en 24-48 h.

Si algo falla

Menú **Ayuda** → **Diagnóstico**: te dice si el motor de vídeo funciona, qué claves tienes puestas y dónde está tu carpeta de trabajo.

3. Poner las claves de API

Ve a la pestaña **Ajustes** → **Claves de API**, pega cada clave y pulsa *Guardar*. Se escriben en el fichero `.env` de la carpeta y **no salen de tu ordenador**.

Si usas **Grok**, con una sola clave (`xAI`) tienes texto, imágenes y vídeo: el mismo sitio cubre las tres cosas.

El previo: ver el episodio sin montarlo

Botón  **Previo**, al lado de MONTAR EPISODIO.

Enseña cada plano a pantalla completa con su locución sonando. **No codifica nada**, así que es instantáneo, y como cada plano dura lo que dura su audio —el mismo criterio que usa el montaje— el ritmo que ves ahí es el que va a salir.

Tecla	Qué hace
Espacio	Pausa y continúa
← →	Plano anterior y siguiente
Esc	Cerrar

No verás el Ken Burns, ni los rótulos quemados, ni la música. Para eso hay que montar. Esto es para lo otro: comprobar si la imagen pega con lo que se dice encima y si el episodio se sigue.

Varias imágenes en un mismo plano

Un bloque de narración largo sobre una sola foto se hace pesado. Puedes poner **varias imágenes en el mismo plano** y que se fundan entre sí.

En el Editor, sobre las miniaturas de un plano:

- **Clic normal** — deja esa imagen y quita las demás. Es lo de siempre.
- **Ctrl+clic** (Cmd en Mac) — **añade** esa imagen a la secuencia, o la quita si ya estaba.

Las elegidas salen numeradas con el orden en que van a aparecer. El plano **dura lo mismo**: se reparte entre las imágenes que tenga. Con cuatro imágenes en un plano de 12 segundos, cada una sale unos 3 segundos.

El fundido se ajusta en **Ajustes** → **Montaje** → **Fundido entre imágenes del mismo plano**. Si el plano es corto, se acorta solo para que ninguna foto se quede en nada.

Antes de montar, te avisa de lo que falta

Al darle a **MONTAR EPISODIO**, el programa revisa el episodio y **para** si encuentra algo grave:

- planos sin ninguna imagen — saldrían en **negro**
- planos con imágenes generadas pero **ninguna elegida** — el material ya está pagado y saldría en negro igual
- planos con texto escrito y **sin locutar**
- ficheros que el proyecto recuerda y ya no están

Puedes montar de todas formas confirmando, que para ver cómo va un episodio a medias es lo normal. Pero al menos ya no descubres después de ocho horas que catorce planos salieron negros.

Que el montaje no tarde ocho horas

Ajustes → **Montaje** → **Codificador de vídeo**. En **Automático** el programa comprueba qué sabe hacer tu máquina y elige el más rápido que mantenga la calidad.

Si tienes tarjeta gráfica —o gráfica integrada de Intel, que también vale— **codificar por hardware va varias veces más rápido** y además deja la CPU libre para los filtros. En un portátil Intel m3 medimos 9,4 veces más rápido que por software, con una diferencia de calidad que no se ve.

Debajo del selector te dice exactamente qué ha encontrado en tu máquina y qué no. Para medirlo con tu propio material:

```
node scripts/bench-codec.js tu-video.mp4
```

Y **Planos a la vez** controla cuántos se montan en paralelo. En 0 lo decide el programa según tus núcleos y, si codificas por GPU, según cuántas sesiones admite la tarjeta.

Remontar ya no cuesta lo mismo

Cambias la imagen de un plano y vuelves a montar: **solo se rehace ese plano**. Los demás estaban ya hechos y sin cambios, así que se reaprovechan. En el log te dice cuántos.

Y si el montaje se corta —se va la luz, cierras el programa, cancelas— al volver a darle **sigue donde estaba**. Se anota el progreso cada diez planos.

Si alguna vez algo huele raro y sospechas de lo guardado, hay una opción para descartarlo y montar de cero.

Te dice cuánto va a tardar

Antes de empezar, y en el log al arrancar el montaje. La primera vez es un cálculo a ojo y te lo dice; a partir del segundo montaje en tu máquina, el número sale de **lo que tardaron los tuyos** y acierta.

Ritmo: lo que más cambia la sensación

Ajustes → Montaje → Ritmo. Multiplica los silencios que rodean la voz en todo el episodio:

Ritmo	Para qué
Ágil	Encadenado, sin respiro. Redes y vídeos cortos
Normal	Lo de siempre
Pausado	Deja respirar. Documental narrado
Solemne	Mucho aire entre frases

Y un plano concreto puede llevar los suyos, que mandan sobre esto: un silencio antes de la frase que remata un bloque vale más que cualquier efecto.

Ver un solo plano

Botón  **Ver este plano** en cada plano del Editor. La imagen a pantalla completa con su locución, sin codificar nada. Para comprobar uno concreto sin pasar por el episodio entero.

Reordenar las imágenes de un plano

Cuando un plano lleva varias, **arrástralas** para cambiar el orden en que aparecen. El número de cada una te dice en qué posición va.

El sonido sale al nivel que espera YouTube

Sin tocar nada. YouTube normaliza **todo** lo que le subes a unos -14 LUFS: si tu episodio va más bajo, te lo sube y con él sube el ruido de fondo; si va más alto, te lo baja y pierde pegada. Ahora sale ya en su sitio, que suena mejor

que dejar que lo hagan ellos.

Se apaga en **Ajustes** → **Montaje** si prefieres el nivel crudo.

Borrador rápido

Botón al lado de MONTAR. Monta el episodio **de verdad** pero a resolución baja y a toda velocidad — en las pruebas, **más del doble de rápido**. Para comprobar si el ritmo se sostiene o si una imagen desentona no hace falta 1080p.

Dura **exactamente lo mismo** que el montaje bueno, así que el ritmo que ves es el que va a salir.

Montar TODOS los episodios

Botón **Montar TODOS**. Los hace uno detrás de otro en una sola tanda. Si alguno falla, se apunta y **se sigue con los demás**: que el cuarto esté a medias no es razón para no entregar los otros nueve.

Encaje de un plano concreto

En cada plano del Editor hay un desplegable de encaje. Normalmente el de Ajustes vale, pero de vez en cuando hay una foto que pide otra cosa, y antes había que cambiarlo para todo el episodio.

Detalles que se notan sin saber por qué

- **El Ken Burns ahora acelera y frena.** Antes arrancaba de golpe y paraba en seco.
- **Se quita el silencio que dejan las voces al principio.** Media décima aquí y allá, y el episodio entero se iba haciendo más lento.
- **Avisa si una locución existe pero no suena.** Ese plano iría mudo sin que nada fallase.

Los rotulos, con el diseno que quieras

Ajustes → **Montaje** → **Diseno de los rotulos**. Seis a elegir, y cada uno cambia fuente, color, tamaño, posición y como entra:

Diseno	Para que
Documental	Dorado con remate, abajo. El de siempre
Noticias	Banda solida inferior. Se lee sobre cualquier imagen
Minimal	Blanco fino, sin adornos
Impacto	Grande y centrado. Para videos cortos, se lee en un movil
Manuscrito	Cursiva con remate. Historia, epoca
Tecnico	Monoespaciado arriba. Tutoriales, datos

Subtitulos con la palabra resaltada

Lo que funciona en vertical: se ve marcada la palabra que se esta diciendo. Se activa en **Ajustes** → **Montaje**, y puedes cambiar los dos colores y cuantas palabras salen a la vez.

La sincronia es aproximada. Se calcula por la longitud de cada palabra, no escuchando el audio, así que con

cifras y siglas se desvia un poco. Para seguir la frase con la vista va de sobra.

El `.srt` normal **sigue saliendo aparte** para subirlo a YouTube como pista de subtítulos.

Zona segura

Las apps te tapan parte del video con sus botones: en vertical, TikTok se come como el 20 % de abajo y YouTube el 10 % de arriba. Con la zona segura activada, los rotulos y los subtítulos se apartan de ahí y se leen. Se puede desactivar.

Careta y cierre

Ajustes → Montaje. Pones la ruta a un MP4 y se pega delante o detras del episodio. **Son opcionales:** en blanco, todo va como antes.

Se adaptan solas a la resolucion y a los fotogramas de tu episodio, y si el clip no trae audio se le pone silencio. Los capitulos de YouTube se desplazan solos lo que dure la cabecera.

Buscar y reemplazar — Ctrl+F

Busca por todo el episodio en la narracion, los rotulos, los prompts, la camara y las notas. Te ensena donde esta cada coincidencia con su contexto y te lleva al plano.

Para reemplazar, **primero se prueba:** te dice cuantas cosas cambiarian y en cuantos planos, sin tocar nada. Y sobre todo te avisa de los planos que **ya tienen la voz grabada**, porque cambiar el texto no cambia el audio: esos hay que volver a locutarlos o el video dira una cosa y los subtítulos otra.

Atajos de teclado

Pulsa **?** para verlos. Los principales:

Tecla	Que hace
Ctrl + F	Buscar y reemplazar
Ctrl + P	Previo del episodio
1 ... 7	Saltar de pestana
Esc	Cerrar lo que este abierto

Cuanto va a costar — boton

Antes de generar. Te dice lo que costaria lo que falta y lo que costaria rehacerlo todo, desglosado.

Son ordenes de magnitud, no una factura: los precios cambian y dependen de tu plan, y estan redondeados **hacia arriba**. Lo que de verdad gastas lo dice la consola de cada proveedor, y el tope diario de Ajustes sigue siendo la unica red de seguridad real.

Plantillas

Ajustes → Plantillas. Guarda el look de una serie —estilo, voz, ritmo, montaje y proveedores— y aplicalo al siguiente proyecto sin volver a configurarlo todo. Son **opcionales**.

No guardan tus claves de API ni las rutas de tu disco: eso es de este ordenador, no de la serie.

Que los personajes muevan la boca cuando toca

En **Ajustes** → **Estilo** → **Movimiento de labios**. Existe porque los modelos de vídeo, en cuanto ven una cara, la ponen a hablar —aunque quien hable sea el narrador, que no está en pantalla—. El resultado era gente moviendo los labios sin decir nada, descuadrada con una voz que no era suya.

Opción	Qué hace
Automático (recomendado)	En los planos de voz en off pide bocas cerradas y quietas . Cuando quien habla es un personaje del plano, le pasa su frase literal para que los labios cuadren con lo que se va a oír
Siempre	Igual, pero avisa también en planos sin gente. Útil si el modelo se inventa caras al fondo
Nunca	No añade nada al prompt, como antes

Funciona solo si el guion marca quién habla (`[VOZ:BRUNO]`). Con narración a secas, lo que hace es asegurarse de que **nadie** mueva la boca.

No hacen falta todas. Con **una** ya puedes empezar. La combinación mínima recomendada:

Quiero...	Necesito
Solo probar el montaje	ninguna (usa imágenes tuyas subidas a mano)
Imágenes + montaje	<code>GEMINI_API_KEY</code>
Lo anterior + voz profesional	<code>+ ELEVENLABS_API_KEY</code>
Lo anterior + vídeo animado	ya la tienes (Veo va con la de Gemini) — o una de fal.ai, que sale aún más barato
Lo anterior + música	ya la tienes (ElevenLabs Music)

Cómo darse de alta en cada servicio: `03_GUIA_IAS_Y_ALTAS.md` .

4. Crear una serie desde cero

Pestaña **Crear**.

4.1 De dónde sale el contenido

Opción	Cuándo usarla
Un tema	No tienes material. Describe de qué va y la IA lo escribe con su propio conocimiento. Sé concreto: «la historia de la penicilina, del accidente de Fleming a la producción masiva en la guerra» sale mucho mejor que «la penicilina».
Un fichero	Tienes un libro, un informe, unos apuntes. Admite PDF, DOCX, EPUB, TXT, MD, HTML y RTF .
Texto pegado	Un artículo, un capítulo, unas notas.
Una dirección web	Una entrada de blog, una ficha de producto, un artículo. Pega una URL por línea si quieres varias. Se limpia el menú, el pie y los banners, y queda solo el contenido. Además se traen las fotos de la propia página (ver 4.5).
Una carpeta de módulo	Para manuales de software: lee la documentación y las opciones del programa y escribe el vídeo de ayuda, pidiéndote las capturas que hacen falta.

Un libro entero no se manda de golpe: se resume por partes, luego se diseña la escaleta, y después se escribe episodio a episodio. Los PDF se envían tal cual al modelo, porque Gemini y Claude los leen mejor que cualquier extractor.

El .doc antiguo no se puede leer. Ábrelo en Word y guárdalo como .docx o .pdf. **Un PDF escaneado** (páginas que son fotos) tampoco tiene texto: el programa te avisa. Usa el original o pásalo por un OCR.

4.2 Qué quieres que sea

Formato	Qué produce
Serie documental	Narrador único, tono cinematográfico, escenas dramatizadas y rótulos con fechas
Vídeo de YouTube	Gancho fuerte en los primeros 15 s, ritmo ágil, frases cortas, segunda persona
Videoensayo	Tesis, desarrollo y conclusión. Reflexivo y personal
Podcast narrativo	Documental sonoro. El diseño de sonido sustituye a la imagen
Curso o formación	Objetivos de aprendizaje, ejemplos, resumen por lección
Vídeo corto vertical	TikTok, Reels y Shorts. Vertical, sin presentación, un solo mensaje, subtítulos quemados
Vídeo de producto	Ficha de tienda: qué problema resuelve, para quién es y dónde comprarlo
Manual de un programa	Tutorial con capturas reales de pantalla, no imágenes inventadas
Ficción sonora	Narrador y escenas con diálogo, ambiente y foley

Cada formato se autoconfigura. Al elegirlo se ponen solos la orientación (vertical u horizontal), la resolución, la duración típica, los subtítulos y el aire entre planos. No tienes que acordarte de que TikTok es vertical ni de que un manual no lo es.

La duración la eliges tú

El desplegable **Duración de cada pieza** solo ofrece lo que tiene sentido en el formato elegido: en «vídeo corto vertical» van de 15 segundos a 5 minutos, y en «serie documental» de 5 minutos a 1 hora. Al cambiar de formato se pone sola la duración típica de ese formato — 21 s en redes, 10 min en YouTube, 15 min en documental — y a partir de ahí eliges la que quieras.

Si en un formato de redes pasas de **1 minuto**, la nota de debajo se pone en color de aviso: no está prohibido, pero a partir de ahí se desploma el porcentaje de gente que llega al final.

La duración se pide en **segundos**, no en minutos: un TikTok dura 21 segundos y eso en minutos no se puede escribir. El programa calcula cuántas palabras caben de verdad en ese tiempo —descontando el silencio de cada corte—, se las pide a la IA, y **luego las cuenta**. Si el guion se ha pasado, se lo devuelve para que lo recorte antes de grabar nada. Un vídeo de 21 s sale de 18-24 s, no de cinco minutos.

Red	Lo que mejor funciona	Límite técnico
TikTok	15-34 s (21 es el punto dulce)	10 min
Instagram Reels	7-30 s	3 min
YouTube Shorts	15-60 s	3 min
Facebook Reels	15-45 s	90 s

Más el número de episodios, la duración, el idioma, el público y el tono. En **Instrucciones extra** puedes pedir lo que quieras: «que cada episodio empiece con una historia personal», «evitar tecnicismos», «citar siempre las fuentes».

Pulsa **Generar los guiones**. Tarda unos minutos (una llamada para la escaleta y otra por episodio); el avance se ve en **Trabajos**. Al terminar, la serie queda abierta y lista en **Episodios**.

4.3 Qué se genera

Un kit de seis ficheros **.md** en la carpeta de la serie: guion de vídeo, guion de podcast, prompts de imagen, prompts de vídeo, biblia de producción e índice. **Puedes editarlos a mano** y volver a pulsar **Importar**: el programa los relee.

4.4 Varias series a la vez

Al final de la pestaña **Crear** están todas tus series. *Abrir* cambia de una a otra; cada una tiene su material y su carpeta de entregas independientes.

4.6 Idiomas: el sistema completo

Cada serie tiene **un idioma principal** —en el que la IA escribe— y los **adicionales** que quieras. No se configura en Ajustes: es de cada proyecto, así que un documental en castellano y un TikTok en inglés conviven sin pisarse.

Al crear la serie eliges el principal y marcas los adicionales. El **texto se traduce solo, de todos los episodios**, nada más crearla: traducir es barato y así el proyecto nace completo.

Lo demás se genera cuando tú lo pidas, porque cuesta dinero:

	Cuándo
Texto	Siempre, en todos los idiomas
Imágenes	Se comparten (una imagen no habla)
Locución	Solo en los idiomas que marques
Vídeo	Solo en los idiomas que marques

En **Episodios** hay una barra con los idiomas. Al pulsar uno **cambia todo lo que ves**: narración, rótulos, título y locución. Lo que edites ahí no toca el original — la estrella (★) marca el principal.

Desde esa barra puedes **traducir el episodio**, **traducir a todos los idiomas** y abrir **Idiomas del proyecto...** para añadir o quitar.

Montar en un idioma es simplemente tenerlo seleccionado y pulsar *MONTAR EPISODIO*. El fichero sale con el código en el nombre: `EP01_TITULO.en.mp4`. El principal no lo lleva.

La duración del vídeo cambia con el idioma, porque la marca la locución: una misma frase no dura lo mismo en castellano que en alemán. Es normal que el mismo episodio dure 10 s en uno y 15 en otro.

4.7 Mejorar cualquier cosa con IA

Junto a cada campo hay dos botones:

- **Mejorar con IA** — pule lo que ya hay.
- **Proponer otra/o** — ignora lo que hay y propone algo distinto. Útil cuando el texto no te convence y no quieres una variación de lo mismo.

Funciona en la narración, los rótulos, el prompt de imagen, el prompt de vídeo, la escena y el título. **Siempre te enseña la propuesta antes de aplicarla**: puedes rechazarla.

4.8 Generar imágenes a partir de otra imagen

En cada plano, debajo de *Generar imágenes*:

- **USAR como base la imagen actual** — la IA parte de la que ya tiene el plano y la modifica. Sirve para cambiar el estilo sin perder el encuadre.
- **Subir imagen de base...** — usa una imagen tuya como punto de partida.
- El campo de al lado es **qué hacerle**: «cambiar a acuarela», «añadir niebla», «quitar el texto». Si lo dejas vacío se rehace con el estilo de la serie.

4.9 Episodio completo

- **Ampliar episodio...** — escribe planos NUEVOS al final, continuando la historia. No toca nada de lo que ya hay.
- **Rehacer todo el episodio...** — vuelve a generar imágenes, voz y montaje de una pasada. Respeta los planos que hayas **fijado** con el candado, así que fija primero los que ya te valgan.
- **Marca de agua...** — sube un logotipo (mejor PNG con transparencia) y elige posición, tamaño y opacidad. Puedes ponerlo para todo el proyecto o solo para un episodio; la del episodio manda.

- **Podcast desde documento...** — sube un PDF, DOCX o EPUB, o pega texto. La IA **no lo lee tal cual**: lo reescribe para que suene bien dicho en voz alta (frases más cortas, siglas deletreadas, cifras escritas como se dicen), respetando el contenido.

4.10 Si algo se atasca

En **Ajustes** → **Mantenimiento**:

- **Recargar proyectos** — vuelve a leer la carpeta de datos sin cerrar nada. Úsalo si has copiado proyectos a mano y no aparecen.
- **Reiniciar el programa** — cierra y vuelve a abrir el motor. No se pierde nada: todo está guardado en disco.

4.5 Las fotos de la página, no fotos inventadas

Cuando la fuente es una **dirección web**, los planos no se inventan: se usan las fotos que ya están en esa página. Para una ficha de producto esto no es un detalle, es la diferencia entre un vídeo que sirve y uno que no — una foto inventada de tu producto no vale para nada.

Cómo funciona, en orden:

1. **Se buscan las fotos** de la página: la imagen social, las que declara la tienda en sus datos de producto y las que están dentro del artículo. Se tiran logos, iconos, banderas, botones de pago y píxeles de seguimiento.
2. **Se piden en grande.** Muchas webs sirven miniaturas de 250 píxeles; el programa reescribe la dirección para traer la versión de 1024, porque el zoom lento del montaje se hace *encima* de la foto y una miniatura se ve pixelada.
3. **La IA las mira, una a una.** Aquí está la diferencia. El texto alternativo de una foto suele estar vacío (en la prueba con Wikipedia lo estaba en las 22), así que sin mirarlas la IA escribiría a ciegas. Con visión, sabe que la foto 4 es «un científico con bata blanca examinando una placa de Petri» y la 6 «un diagrama del experimento en ratones».
4. **Se descarta lo que no sirve.** Lo que la vista reconoce como logo, banner o anuncio se cae aquí, aunque el nombre del fichero no lo delatara.
5. **Se escribe el guion sobre esas fotos.** La IA elige cuál va en cada plano y **narra lo que se ve en ella**. Si el plano es el primer plano del hongo, la voz habla del hongo, no de fechas.
6. **Si algo no existe en la página**, y solo entonces, ese plano se marca como imagen de IA y se genera. Por ejemplo: la página cuenta que la cepa buena salió de un melón podrido, pero no tiene ninguna foto del melón.

En el editor, esos planos salen marcados como **«foto de la web»**, con un enlace al original y con la descripción de lo que la IA ha visto. Si alguna no te convence, súbela tú o genérala con IA desde el mismo sitio: no hay nada bloqueado.

Necesita un proveedor con visión configurado en *Ajustes > Proveedores > Visión* (Gemini o Claude sirven). Sin él, las fotos se bajan igual, pero la IA escribe sin verlas y el guion encaja bastante peor.

5. Tu voz

Pestaña **Mi voz**. Grabas cinco párrafos desde el navegador y el programa crea una **voz clonada tuya** en ElevenLabs para locutar toda la serie.

1. Pulsa **Grabar** en el primer párrafo, léelo, pulsa **Detener**.

2. Repite con los cinco. Están escritos a propósito para cubrir registros distintos —narración neutra, preguntas, cifras, tensión y cierre cálido—: una muestra con variedad de entonación clona mucho mejor que leer siempre igual.
3. Cuando la barra llegue al mínimo (60 s; lo ideal son 150 s), escribe un nombre y pulsa **Crear mi voz clonada**.

Al terminar, esa voz queda seleccionada automáticamente como la del narrador.

Consejos que se notan: sitio silencioso, micrófono a un palmo de la boca, y lee **con la entonación que quieres para tus vídeos** — el clon copia tu forma de narrar, no solo tu timbre.

El clonado necesita ElevenLabs con plan de pago (desde ~5 €/mes). Tus grabaciones se quedan en `data/voz/` y solo se envían cuando pulsas el botón.

Si dice que falta un permiso: no es el plan, es la clave. Al crearla, ElevenLabs deja restringir lo que puede hacer. Ve a <https://elevenlabs.io/app/settings/api-keys>, pulsa el lápiz de tu clave y en *Restrict Key* activa **Voices** → **Write** (clonar) y **Text to Speech** → **Access** (locutar). Lo tienes desplegado en Ajustes → Voz en off. Lo más simple: crea una clave nueva sin restringir nada.

5.1 Elegir una voz del catálogo

Si prefieres no clonar la tuya: **Ajustes** → **Voz en off** → **Cargar voces de ElevenLabs**, elige una y escúchala. Tus voces clonadas salen primero en la lista.

Ajustes recomendados para narración: **Estabilidad** 0.50 · **Similitud** 0.75 · **Estilo** 0.10 · **Velocidad** 0.95.

6. Producir un episodio

6.1 Importar el guion

Pestaña **Episodios** → **Importar / recargar guion**. Aparecen los episodios con su barra de progreso.

Si retocas un `.md`, vuelve a pulsarlo: **conserva las imágenes, clips y voces ya generados** de los planos que no han cambiado.

¿Quieres que un plano no se toque nunca al reimportar? Marca la casilla **fijar** en su cabecera.

El formato de marcadores, por si escribes los guiones tú:

```
# EPISODIO 1 – TÍTULO DEL EPISODIO
**~15 min · subtítulo**

### Nombre del bloque – 0:00
[MÚSICA] tono musical
[IMAGEN IA] prompt en inglés del plano
[VOZ] la narración que se oye mientras se ve ese plano
[RÓTULO] texto que aparece en pantalla
[ESCENA] dramatización concreta
```

Cada `[IMAGEN IA]` abre un plano; todo lo que va detrás pertenece a ese plano hasta el siguiente.

6.1 bis Importar kits escritos con otra IA — un episodio cada vez

Botón **Importar kit de otra IA...** Es para guiones que vienen de fuera: escritos con ChatGPT, Gemini, a mano, o de una temporada anterior. **No genera nada ni gasta un céntimo.**

Lo importante: si ya tienes un proyecto abierto, el kit se **AÑADE** como un episodio más. No reemplaza nada. Para empezar de cero hay que pulsar **Cerrar proyecto** antes (o elegir *Empezar un proyecto nuevo* en el desplegable).

El número de episodio sale, por este orden:

1. El que escribas tú en el campo *Nº de episodio*.
2. El que se deduzca del **nombre de la carpeta**: `LA_OTRA_LINEA_EP02`, `T2E05`, `capitulo-3`, `PARTE 7`, `guion_episodio_12`.
3. Si no hay pista, va **detrás del último** que tengas.

Se deduce del nombre de la carpeta y **no de la cabecera del guion** a propósito: prácticamente todos los kits generados por IA empiezan por `# EPISODIO 1` aunque sean el séptimo de la serie.

Antes de importar, **Ver qué contiene** te dice exactamente a qué número va a ir y por qué.

Cada kit importado se guarda en su propia subcarpeta dentro de `guion/`, con un fichero `_kit.json` que recuerda su número. Así puedes recargar el guion las veces que quieras sin que los episodios se pisen entre sí.

6.1 ter Quitar un episodio

Pasa el ratón por la tarjeta del episodio: aparece una ✕ arriba a la derecha.

Borra el episodio **y el guion del que salió**, porque si no volvería a aparecer la próxima vez que recargues. Pregunta aparte si quieres borrar también sus imágenes, locuciones y montajes; por defecto **se conservan**, que es lo que casi siempre quieres cuando borras un episodio para volver a importarlo corregido.

6.2 Abrir el episodio

Pulsa la tarjeta. Verás:

- Una **línea de tiempo** arriba: cada bloque es un plano, con su anchura proporcional a su duración. Azul = tiene imagen, morado = tiene clip, apagado = no tiene voz. Púlsalo para saltar a ese plano.
- La **lista de planos**, cada uno con su prompt de imagen, su prompt de vídeo, su narración y sus rótulos.

6.3 El orden correcto

Los botones grandes del panel superior están numerados a propósito:

1 · Imágenes → 1b · Ampliar → 2 · Voz → 3 · Animar → 4 · Música → 5 · MONTAR

Paso 1 — Imágenes. Genera 3 variantes de cada plano que no tenga ninguna. Cuando terminen, repasa la lista y **haz clic en la miniatura que prefieras** de cada plano (la elegida sale con borde dorado). La ✕ de la esquina borra una variante.

¿No te convence ninguna? Edita el prompt en el cuadro de texto y vuelve a pulsar *Generar imágenes*, o pulsa **Mejorar prompt con IA** y deja que Claude lo reescriba (te pregunta qué quieres cambiar).

Paso 1b — Ampliar (Magnific). Pasa las imágenes elegidas por el upscaler.

Esto importa más de lo que parece: el Ken Burns **hace zoom sobre la imagen fija**, y si la imagen no tiene resolución de sobra ese zoom se ve blando. Con 2x sobra para 1080p; usa 4x solo si vas a 4K.

En cada plano tienes además dos botones: - **Reiluminar** — cambia la luz del plano sin regenerarlo (hora del día, dramatismo). Te pregunta cómo la quieres. - **Igualar estilo** — impone el look de otro plano ya aprobado del mismo episodio. Es la forma más fiable de que toda la serie parezca de la misma mano.

Los retoques **no borran el original**: se añaden como una variante nueva y quedan seleccionados. Siempre puedes volver atrás pulsando la miniatura anterior.

Paso 2 — Voz. Locuta el texto [VOZ] de cada plano. Los planos sin narración (cabeceras, transiciones) se saltan solos y usan la duración sugerida por el guion.

Al terminar verás que la línea de tiempo cambia: ahora las anchuras son las reales.

Paso 3 — Animar (*opcional, es el paso caro*). Convierte la imagen elegida en un clip con movimiento real.

Es el único paso donde **el modelo que elijas multiplica la factura**. Con los que vienen de fábrica —Veo 3.1 Lite o MiniMax H3 Max Turbo— un clip de 8 segundos sale por unos 30-40 céntimos y tarda menos de un minuto; con Veo 3.1 estándar, ese mismo clip cuesta 3,20 €. En Ajustes, justo debajo del bloque donde eliges qué IA hace cada cosa, tienes escrito **lo que va a costar un clip con lo que tienes puesto**, y la lista de modelos sale con su precio por segundo al lado. Antes de lanzar una tanda, el botón de coste te da el total del episodio.

Si te lo saltas no pasa nada: el montaje aplica un movimiento de cámara suave (Ken Burns) leyendo la indicación del guion — «dolly-in» hace un acercamiento, «paneo de izquierda a derecha» hace un barrido, «tilt ascendente» sube. Para un documental queda perfectamente digno y es gratis.

Estrategia razonable: anima solo los 2-3 planos más espectaculares del episodio y deja el resto en Ken Burns.

Si generas un clip y luego no te gusta, pulsa **Usar solo la imagen**.

Paso 4 — Música. Genera un lecho instrumental. El texto de descripción sale de las indicaciones [MÚSICA] del propio guion; puedes editarlo. También puedes subir un MP3 tuyo.

Paso 5 — Montar. Encola el montaje. Ve a la pestaña **Trabajos** para ver el progreso en directo. Un episodio de 15 minutos tarda entre 8 y 25 minutos según tu ordenador.

6.4 Lo que sale

En la carpeta `output/` (pestaña **Entregas**):

Fichero	Qué es
<code>EP01_...mp4</code>	El episodio, 1920×1080, con rótulos quemados
<code>EP01_...srt</code>	Subtítulos de la narración (súbelos a YouTube)
<code>EP01_...capitulos.txt</code>	Capítulos, pégalos al final de la descripción
<code>EP01_...rotulos.ass</code>	Los rótulos por separado, por si retocas en otro editor

7. Material propio

Cada plano tiene **Subir imagen** y **Subir clip**. Úsalo para:

- Los **mapas** ya rotulados de la saga (`Mapa_*.png`) — súbelos al plano donde el guion los pida.
- Imágenes que hayas generado en Midjourney y no en el programa.
- Metraje de archivo.

Lo subido pasa a ser la selección de ese plano automáticamente.

8. Título y descripción de YouTube

Botón **Título y descripción** del panel superior. Claude lee la narración completa del episodio y devuelve título con gancho, descripción de tres párrafos y 15 etiquetas. Se guardan en el episodio y puedes editarlos.

9. El podcast

Botón **Exportar podcast**. Usa `02_GUION_PODCAST.md` (narración algo más larga, sin rótulos porque en audio las fechas se dicen en voz alta) y saca un MP3 con la misma voz y la misma música.

Las locuciones se guardan en caché **a nivel de proyecto**: la misma frase con la misma voz se graba una sola vez. Si ya has locutado el vídeo, exportar el podcast **no vuelve a llamar a la API** ni te cobra otra vez; y al revés. La caché se invalida sola cuando cambias el texto o la voz, que es justo cuando hace falta rehacer el audio.

En la barra de Episodios verás algo como `312 locuciones guardadas (48 MB)` . Púlsalo para vaciarlas si has cambiado una voz y quieres rehacer todos los audios de verdad.

9 bis. Acotaciones: «(ríe)», «(susurra)», «(llora)»

En los guiones de diálogo aparecen constantemente indicaciones de lo que el personaje **hace**, no de lo que dice:

```
ELENA: (ríe) No, no, déjame que te cuente...
BRUNO: [suspira] Ya... *baja la voz* eso no me lo esperaba.
```

Eso **no es texto para locutar**. Antes se mandaba tal cual y el locutor lo leía en voz alta. Ahora el programa las reconoce en las tres formas en que se escriben — `(entre paréntesis)` , `[entre corchetes]` y `*entre asteriscos*` — y hace una de dos cosas según el modelo de voz:

Modelo de ElevenLabs	Qué hace
<code>eleven_v3</code> (y cualquier <code>*_v3</code>)	Convierte la acotación en su etiqueta de audio (<code>[laughs]</code> , <code>[sighs]</code> , <code>[whispers]</code> ...) y el locutor la actúa de verdad.
<code>eleven_multilingual_v2</code> y anteriores (el de por defecto)	La quita del texto y en su lugar mueve el tono de esa línea: un susurro baja estabilidad y velocidad, un grito sube el estilo.

Reconoce: risa, sonrisa, suspiro, llanto, susurro, grito, duda, nervios, emoción, tristeza, enfado, ironía, curiosidad, carraspeo y pausa. En castellano y en inglés.

Dos detalles que importan:

- Un paréntesis **normal** no se toca. `Roma cayó en 476 (según los manuales)` se locuta entero: solo se retiran los paréntesis que contienen una acotación reconocida y son cortos.
- Las acotaciones **no cuentan como palabras habladas**, así que ya no descuadran el cálculo de duración. Un diálogo con muchas acotaciones salía corto aunque el contador dijera lo contrario.

Si quieres que el locutor las actúe, pon `eleven_v3` en Ajustes → Modelos → *Modelo de voz*.

10. Ajustes que merece la pena tocar

Ajustes → Montaje:

Ajuste	Para qué
Silencio antes / después de la voz	Da aire. 0.35 / 0.65 s va bien. Súbelos si el ritmo te parece atropellado.
Duración mínima de plano	Ningún plano baja de aquí aunque no tenga voz.
Transición	<i>Fundido a negro</i> (documental) o <i>Corte seco</i> (más nervioso).
Zoom de Ken Burns	1.16 es sutil. Por encima de 1.3 se nota demasiado.
Volumen de la música	0.14 deja la voz muy por delante.
Atenuar música al hablar	Déjalo en <i>Sí</i> : baja la música automáticamente en cada frase.
Preset x264	<code>veryfast</code> para pruebas, <code>medium</code> para la versión final.
Calidad CRF	19 es alta. 23 pesa la mitad y casi no se nota en YouTube.

Ajustes → Estilo visual: el sufijo de estilo se añade a **todos** los prompts de imagen. Es lo que mantiene la coherencia visual de la serie. Cámbialo solo si quieres cambiar el look de toda la temporada.

11. Problemas frecuentes

«Falta la clave de X» → Ajustes → Claves de API.

Una imagen sale rara o con texto raro. Los modelos meten texto inventado. Añade al prompt negativo `text, letters, signage` y regenera.

El montaje falla en un plano. Mira el mensaje en Trabajos. Casi siempre es una imagen corrupta: bórrala con la `x`, regenera y vuelve a montar.

Los rótulos no aparecen en el MP4. El programa avisa en el log y monta sin ellos en vez de perder el trabajo. Suele ser que la fuente no existe: en Ajustes → Montaje cambia *Fuente de los rótulos* a `Georgia` o `Times New Roman`. El `.ass` siempre se exporta aparte.

Tarda muchísimo. Baja el preset a `veryfast` y sube el CRF a 23 mientras pruebas. Deja `medium` /19 solo para la versión que vas a publicar.

Un trabajo se ha quedado colgado. Botón *Cancelar* en Trabajos. Los de vídeo tienen corte automático a los 15 minutos.

Quiero empezar de cero. Usa **Cerrar proyecto** en la pantalla de Episodios. A partir de ahí, lo siguiente que importes o crees será un proyecto nuevo.

He importado un kit y me ha borrado el episodio anterior. Eso ya no pasa: importar con un proyecto abierto **añade** un episodio. Si de verdad quieres empezar de cero, cierra el proyecto antes o elige *Empezar un proyecto nuevo* en el desplegable del panel de importación.

«**La IA no devolvió un JSON válido al traducir**». Era un episodio con bloques muy largos: se mandaban doce de golpe y la respuesta se cortaba a mitad. Ahora las tandas se miden en palabras, no en bloques, y si aun así viene cortada la tanda se parte en dos y se reintenta. Un bloque que siga sin poder traducirse se queda en el idioma original y se avisa en el log, en vez de tumbar el episodio entero.

12. Consejo de producción

Sigue la biblia (§7): **produce los 3 primeros episodios antes de estrenar**. Y monta primero un episodio entero con Ken Burns y sin animar nada — así ves el ritmo real antes de gastarte el dinero en clips de vídeo, que es con diferencia lo más caro.

GUÍA DE IAs — qué contratar, para qué, y cómo darse de alta

Guía práctica para decidir dónde gastar y dónde no. Los precios son **órdenes de magnitud** para presupuestar: cámbianlos cada pocos meses, así que confirma siempre en la web del servicio antes de cargar saldo.

1. Resumen: qué IA hace qué

Tarea	Recomendado	Alternativa	Coste relativo
Imágenes	Google · Gemini 3 Pro Image	Freepik (Mystic), fal.ai (Flux), OpenAI	bajo
Vídeo	Google · Veo	fal.ai / Replicate (Kling)	muy alto
Voz en off	ElevenLabs	OpenAI TTS, Gemini TTS	medio
Música	ElevenLabs Music	Replicate (MusicGen), fal.ai	bajo
Texto y prompts	Claude	Gemini, OpenAI	muy bajo
Ampliar / retocar imagen	Magnific (Freepik)	—	bajo
Clonar tu voz	ElevenLabs	—	plan de pago

Si solo vas a dar de alta una cosa: **Google AI Studio**. Con esa única clave tienes imágenes, vídeo y texto.

 **Aviso importante sobre Google.** La capa gratuita cubre el texto, pero **los modelos de imagen y de vídeo exigen facturación activada** en tu proyecto de Google Cloud. Sin ella recibirás un error 429 de cuota. Entra en <https://aistudio.google.com/apikey> → *Set up Billing* y asocia una tarjeta.

Además, `imagen-4.0-*` **ya no se sirve a cuentas nuevas**: los modelos de imagen vigentes son `gemini-3-pro-image` (calidad) y `gemini-3.1-flash-image` (más barato). El programa migra el ajuste automáticamente.

Si vas a dar de alta dos: **Google + ElevenLabs**. La voz es lo que separa un documental de una presentación de PowerPoint, y ahí ElevenLabs está claramente por delante.

2. Google AI Studio — Gemini · Imagen · Veo

Cubre: imágenes (Gemini 3 Image), vídeo (Veo), texto (Gemini), voz (Gemini TTS).

Alta

1. Entra en <https://aistudio.google.com/apikey> con tu cuenta de Google.
2. **Create API key** → elige o crea un proyecto de Google Cloud.
3. Copia la clave (`AIza...`) y pégala en Ajustes → `GEMINI_API_KEY` .

Capa gratuita

Hay cuota gratis con límites de peticiones por minuto, suficiente para probar el sistema entero. **Veo y los modelos de imagen de más calidad exigen facturación activada.** Para eso: menú → *Get API key* → *Set up Billing*, y asocia una tarjeta al proyecto de Google Cloud.

Orden de magnitud

- Imagen: **céntimos por imagen**. Un episodio de 8 planos × 3 variantes ≈ 24 imágenes: calderilla.
- Veo: **varios euros por cada clip de 8 segundos**. Aquí es donde se va el presupuesto — ver §7.

Modelos

- **Texto:** `gemini-flash-latest` (*por defecto*) · `gemini-3-flash-preview` · `gemini-3-pro-preview` · `gemini-flash-lite-latest`
- **Imagen:** `gemini-3-pro-image` (*por defecto, mejor calidad*) · `gemini-3.1-flash-image` (*más barato*) · `gemini-2.5-flash-image`
- **Vídeo:** `veo-3.1-generate-preview` · `veo-3.1-fast-generate-preview` · `veo-3.1-lite-generate-preview`
- **Voz:** `gemini-2.5-flash-preview-tts` · `gemini-2.5-pro-preview-tts`

Todos son **campos de texto** en Ajustes → Modelos, porque Google los renombra con frecuencia. Si recibes «modelo no encontrado», consulta el nombre vigente en <https://ai.google.dev/gemini-api/docs/models> y pégalo ahí. No hace falta tocar código.

- Imagen: **céntimos por imagen**.

3. ElevenLabs — la voz del narrador

Cubre: voz en off, música instrumental, efectos de sonido.

Alta

1. <https://elevenlabs.io> → *Sign up*.
2. Perfil (arriba a la derecha) → **API Keys** → *Create API Key*.
3. Pégala en Ajustes → `ELEVENLABS_API_KEY`.

Planes

- **Gratis:** unos 10.000 caracteres al mes. Da para menos de un episodio (los guiones rondan los 5.500 caracteres... por episodio contando los diez, unos 55.000 en total). Sirve para elegir voz.
- **Starter (~5 €/mes):** ~30.000 caracteres. Un par de episodios.
- **Creator (~22 €/mes):** ~100.000 caracteres. **Este es el plan de esta serie:** cubre los 10 episodios de la temporada con margen para regenerar.

Cómo elegir la voz

En Ajustes → Voz en off → *Cargar voces de ElevenLabs*, y escucha las previsualizaciones. Busca lo que pide la biblia (§4): **grave, cálida, autoridad tranquila, español de España o neutro**. Filtra por `narration` o `documentary` en las etiquetas.

Ajustes de la biblia, ya puestos por defecto: Estabilidad 0.50 · Similitud 0.75 · Estilo 0.10 · Velocidad 0.95.

⚠ Permisos de la clave (el fallo nº1)

Al crear una clave, ElevenLabs deja **restringir** lo que puede hacer («Restrict Key»). Si la creas restringida y no activas lo que toca, todo falla con mensajes crípticos del tipo «*missing the permission create_instant_voice_clone*» — y **no es un problema de plan**, es de la clave.

Activa en <https://elevenlabs.io/app/settings/api-keys> → lápiz de tu clave → *Restrict Key*:

Para	Activa
Ver el catálogo de voces	Voices → Read
Clonar tu voz	Voices → Write
Locutar el guion	Text to Speech → Access
Música de fondo	Music Generation → Access
Efectos de sonido (opcional)	Sound Effects → Access

Lo más simple: crea una clave nueva y no restrinjas nada.

Clonar tu propia voz

El programa lo hace por ti: pestaña **Mi voz**, grabas cinco párrafos y pulsa *Crear mi voz clonada*. Es la opción más distintiva para un canal — tu voz, pero sin tener que locutar diez episodios.

- Necesita **plan de pago** (Starter, ~5 €/mes, ya lo incluye).
- Con 60 s de grabación funciona; con 2–3 minutos queda notablemente mejor.
- **Solo tu voz, o la de alguien que te haya dado permiso por escrito.** Clonar la voz de un tercero sin consentimiento es ilegal en la mayoría de países y viola los términos de ElevenLabs.

Música

`music_v1` genera lechos instrumentales por prompt. Suficiente para el fondo. Si buscas un **tema principal** memorable, Suno o Udio dan mejor resultado — no tienen API pública estable, así que se generan en su web y se suben con el botón *Subir* del programa.

4. Anthropic Claude — el cerebro de texto

Cubre: mejorar prompts de imagen y vídeo, escribir prompts de animación, generar título, descripción y etiquetas de YouTube.

Alta

1. <https://console.anthropic.com> → *Sign up*.
2. **Settings** → **API keys** → **Create Key**.
3. Carga saldo en *Billing* (mínimo unos 5 \$, dan para muchísimo).
4. Pégalas en Ajustes → `ANTHROPIC_API_KEY`.

Coste

Céntimos por toda la temporada. Es la clave más barata y la que más calidad aporta por euro: reescribir un prompt flojo cuesta menos de un céntimo y te ahorra generar imágenes que no vas a usar.

4b. Freepik / Magnific — el "todo en uno"

Magnific es de Freepik desde que lo compró: una sola clave da acceso a todo.

Cubre: Magnific **Upscaler** (subir resolución y detalle), **Relight** (reiluminar), **Style Transfer** (mismo estilo en toda la serie), **Mystic/Flux** (generación de imagen) y **Kling** (imagen a vídeo).

Alta

1. <https://www.freepik.com/api> → *Get started*.
2. Panel de desarrollador → **API Keys** → crear clave.
3. Pégala en Ajustes → `FREEPIK_API_KEY`.

Se paga por créditos, incluidos en algunos planes de Freepik y comprables aparte.

Por qué importa aquí el Upscaler

Es la mejora de calidad con mejor relación esfuerzo/resultado de todo el programa. El efecto **Ken Burns hace zoom sobre la imagen fija**: si la imagen no tiene resolución de sobra, ese zoom se ve blando y el vídeo entero parece barato. Pasar los planos elegidos por el upscaler antes de montar cambia por completo el acabado, y cuesta céntimos comparado con generar vídeo.

- **2x** basta para 1080p. **4x** si vas a 4K.
- **Creatividad 0** = fiel al original. Súbela solo si quieres que se invente detalle.

Igualar estilo

El problema clásico de una serie hecha con IA es que los planos no parecen de la misma mano. *Igualar estilo* impone el look de un plano ya aprobado sobre los demás. Es más fiable que repetir el mismo sufijo de prompt.

Si algo devuelve 404

Freepik reorganiza sus endpoints cada cierto tiempo. Las rutas son **campos de texto** en Ajustes → Magnific → *Rutas de la API*. Consulta la ruta vigente en <https://docs.freepik.com> y corrígela ahí; no hace falta tocar el programa.

5. OpenAI — alternativa

Cubre: imágenes (`gpt-image-1`), voz (`gpt-4o-mini-tts`), texto.

Alta

1. <https://platform.openai.com/api-keys> → *Create new secret key*.
2. Carga saldo en *Billing* (hace falta: no hay capa gratuita de API).

Cuándo usarlo: `gpt-image-1` respeta muy bien las instrucciones complejas y es el único que escribe texto legible dentro de la imagen — útil para la **tarjeta de título** del plano 1.2 de cada episodio. Su TTS es correcto y barato, pero por debajo de ElevenLabs en español.

6. fal.ai y Replicate — el catálogo abierto

Dos pasarelas que dan acceso a decenas de modelos con una sola clave. Útiles cuando Veo te rechaza un plano o quieres comparar estéticas.

fal.ai

- 1. <https://fal.ai> → Sign up → <https://fal.ai/dashboard/keys> → Add key.
- 2. Se paga por uso con saldo prepago. Es **rápido y barato**.
- 3. Modelos que ya vienen configurados: `fal-ai/flux-pro/v1.1-ultra` (imagen), `fal-ai/kling-video/v2/master/image-to-video` (vídeo), `fal-ai/stable-audio` (música).

Replicate

- 1. <https://replicate.com> → Sign up → <https://replicate.com/account/api-tokens>.
- 2. Se paga por segundo de GPU.
- 3. Modelos configurados: `black-forest-labs/flux-1.1-pro` , `kwaivgi/kling-v2.1` , `meta/musicgen` .

En Ajustes → Modelos puedes cambiar cualquiera por otro del catálogo (`owner/nombre`). El código no asume nada del modelo concreto.

Flux es la mejor alternativa a Imagen 4 para el look fotorrealista cinematográfico de esta serie. **Kling** es la mejor alternativa a Veo en calidad-precio para animar imágenes fijas.

7. Cuánto cuesta un episodio (y cómo gastar poco)

Un episodio son ~8 planos y ~2.000 palabras de narración.

Concepto	Cantidad	Coste orientativo
Imágenes (3 variantes × 8 planos)	24 imágenes	céntimos
Voz en off	~11.000 caracteres	incluido en el plan mensual
Música	1 pista	céntimos
Prompts y metadatos (Claude)	~15 llamadas	céntimos
Vídeo animado (8 clips)	8 × 8 s	decenas de euros

El vídeo cuesta más que todo lo demás junto, por un factor enorme.

La estrategia que recomiendo

1. **Monta el episodio entero solo con imágenes.** El Ken Burns del programa lee la indicación de cámara del guion y hace el movimiento que pedía el doc 04. Para un documental histórico es un acabado perfectamente profesional — así se han hecho toda la vida.
2. **Míralo entero.** Localiza los 2 o 3 planos que piden movimiento real: el gancho, el clímax del episodio, el plano final.
3. **Anima solo esos.** Pasas de decenas de euros por episodio a unos pocos, y el resultado percibido apenas cambia.
4. Deja el vídeo generativo para el **tráiler** de 60–90 s, que es donde de verdad importa.

8. Privacidad y buenas prácticas

- Las claves se guardan en el fichero `.env` **de tu ordenador** y no se envían a ningún sitio salvo al proveedor correspondiente. Están en `.gitignore`.
- El programa escucha solo en `127.0.0.1`: nadie de tu red puede entrar.
- Los prompts sí viajan al proveedor que elijas. Son descripciones de escenas históricas: no hay nada sensible, pero tenlo presente.
- **Pon límites de gasto** en cada consola (Google Cloud → *Budgets & alerts*; Anthropic y OpenAI → *Usage limits*). Es la única red de seguridad real contra un lote lanzado por error.
- Nada en este programa se ejecuta solo. No hay tareas programadas ni reprocesado automático: cada llamada de pago sale de un clic tuyo.

9. Enlaces rápidos

Servicio	Alta	Claves	Precios
Google AI Studio	https://aistudio.google.com	https://aistudio.google.com/apikey	https://ai.google.dev/pricing
ElevenLabs	https://elevenlabs.io	https://elevenlabs.io/app/settings/api-keys	https://elevenlabs.io/pricing
Anthropic Claude	https://console.anthropic.com	Settings → API keys	https://anthropic.com/pricing
OpenAI	https://platform.openai.com	https://platform.openai.com/api-keys	https://openai.com/api/pricing
Freepik / Magnific	https://www.freepik.com/api	Panel → API Keys	https://www.freepik.com/api/pricing
fal.ai	https://fal.ai	https://fal.ai/dashboard/keys	https://fal.ai/pricing
Replicate	https://replicate.com	https://replicate.com/account/api-tokens	por modelo, en su ficha

GUÍA DE IAs — qué contratar, para qué, y cómo darse de alta

Guía práctica para decidir dónde gastar y dónde no. Los precios son **órdenes de magnitud** para presupuestar: cámbianoslos cada pocos meses, así que confirma siempre en la web del servicio antes de cargar saldo.

1. Resumen: qué IA hace qué

Tarea	Recomendado	Alternativa	Coste relativo
Imágenes	Google · Gemini 3 Pro Image	Grok Imagine , Freepik (Mystic), fal.ai (Flux), OpenAI	bajo
Vídeo	Google · Veo	Grok Imagine , fal.ai / Replicate (Kling)	muy alto
Voz en off	ElevenLabs	OpenAI TTS, Gemini TTS	medio
Música	ElevenLabs Music	Replicate (MusicGen), fal.ai	bajo
Texto y prompts	Claude	Gemini, OpenAI	muy bajo
Ampliar / retocar imagen	Magnific (Freepik)	—	bajo
Clonar tu voz	ElevenLabs	—	plan de pago

Si solo vas a dar de alta una cosa: Google AI Studio. Con esa única clave tienes imágenes, vídeo y texto.



Aviso importante sobre Google. La capa gratuita cubre el texto, pero **los modelos de imagen y de vídeo exigen facturación activada** en tu proyecto de Google Cloud. Sin ella recibirás un error 429 de cuota. Entra en <https://aistudio.google.com/apikey> → *Set up Billing* y asocia una tarjeta.

Además, imagen-4.0-* **ya no se sirve a cuentas nuevas**: los modelos de imagen vigentes son gemini-3-pro-image (calidad) y gemini-3.1-flash-image (más barato). El programa migra el ajuste automáticamente.

Si vas a dar de alta dos: Google + ElevenLabs. La voz es lo que separa un documental de una presentación de PowerPoint, y ahí ElevenLabs está claramente por delante.

2. Google AI Studio — Gemini · Imagen · Veo

Cubre: imágenes (Gemini 3 Image), vídeo (Veo), texto (Gemini), voz (Gemini TTS).

Alta

1. Entra en <https://aistudio.google.com/apikey> con tu cuenta de Google.

2. **Create API key** → elige o crea un proyecto de Google Cloud.
3. Copia la clave (`AIza...`) y pégala en Ajustes → `GEMINI_API_KEY` .

Capa gratuita

Hay cuota gratis con límites de peticiones por minuto, suficiente para probar el sistema entero. **Veo y los modelos de imagen de más calidad exigen facturación activada.** Para eso: menú → *Get API key* → *Set up Billing*, y asocia una tarjeta al proyecto de Google Cloud.

Orden de magnitud

- Imagen: **céntimos por imagen**. Un episodio de 8 planos × 3 variantes ≈ 24 imágenes: calderilla.
- Vídeo: **de 0,32 € a 3,20 € por clip de 8 segundos**, según el modelo. Aquí se va el presupuesto — ver \$7.

Modelos

- **Texto:** `gemini-flash-latest` (por defecto) · `gemini-3-flash-preview` · `gemini-3-pro-preview` · `gemini-flash-lite-latest`
- **Imagen:** `gemini-3-pro-image` (por defecto, mejor calidad) · `gemini-3.1-flash-image` (más barato) · `gemini-2.5-flash-image`
- **Vídeo:** `auto` (por defecto: pregunta a Google y coge el Veo más barato que tenga tu clave) · `veo-3.1-lite-generate-preview` (0,05 €/s) · `veo-3.1-fast-generate-preview` (0,10 €/s) · `veo-3.1-generate-preview` (0,40 €/s)
- **Resolución de vídeo:** `720p` (por defecto, la barata) · `1080p` · `4k` (no existe en Lite)
- **Audio de los clips:** `no` (por defecto). Veo genera banda sonora, pero el montaje la descarta porque el clip se repite en bucle para cubrir la locución. Pedirlo mudo evita además el rechazo más frecuente del filtro de Google: «we encountered an issue with the audio for your prompt».
- **Voz:** `gemini-2.5-flash-preview-tts` · `gemini-2.5-pro-preview-tts`

Todos son **campos de texto** en Ajustes → Modelos, porque Google los renombra con frecuencia. Si recibes «modelo no encontrado», consulta el nombre vigente en <https://ai.google.dev/gemini-api/docs/models> y pégalo ahí. No hace falta tocar código.

- Imagen: **céntimos por imagen**.

3. ElevenLabs — la voz del narrador

Cubre: voz en off, música instrumental, efectos de sonido.

Alta

1. <https://elevenlabs.io> → *Sign up*.
2. Perfil (arriba a la derecha) → **API Keys** → *Create API Key*.
3. Pégala en Ajustes → `ELEVENLABS_API_KEY` .

Planes

- **Gratis:** unos 10.000 caracteres al mes. Da para menos de un episodio (los guiones rondan los 5.500 caracteres... por episodio contando los diez, unos 55.000 en total). Sirve para elegir voz.
- **Starter (~5 €/mes):** ~30.000 caracteres. Un par de episodios.

- **Creator (~22 €/mes):** ~100.000 caracteres. **Este es el plan de esta serie:** cubre los 10 episodios de la temporada con margen para regenerar.

Cómo elegir la voz

En Ajustes → Voz en off → *Cargar voces de ElevenLabs*, y escucha las previsualizaciones. Busca lo que pide la biblia (§4): **grave, cálida, autoridad tranquila, español de España o neutro**. Filtra por `narration` o `documentary` en las etiquetas.

Ajustes de la biblia, ya puestos por defecto: Estabilidad 0.50 · Similitud 0.75 · Estilo 0.10 · Velocidad 0.95.

⚠ Permisos de la clave (el fallo nº1)

Al crear una clave, ElevenLabs deja **restringir** lo que puede hacer («Restrict Key»). Si la creas restringida y no activas lo que toca, todo falla con mensajes crípticos del tipo «*missing the permission create_instant_voice_clone*» — y **no es un problema de plan**, es de la clave.

Activa en <https://elevenlabs.io/app/settings/api-keys> → lápiz de tu clave → *Restrict Key*:

Para	Activa
Ver el catálogo de voces	<code>Voices → Read</code>
Clonar tu voz	<code>Voices → Write</code>
Locutar el guion	<code>Text to Speech → Access</code>
Música de fondo	<code>Music Generation → Access</code>
Efectos de sonido (opcional)	<code>Sound Effects → Access</code>

Lo más simple: crea una clave nueva y no restrinjas nada.

Clonar tu propia voz

El programa lo hace por ti: pestaña **Mi voz**, grabas cinco párrafos y pulsa *Crear mi voz clonada*. Es la opción más distintiva para un canal — tu voz, pero sin tener que locutar diez episodios.

- Necesita **plan de pago** (Starter, ~5 €/mes, ya lo incluye).
- Con 60 s de grabación funciona; con 2–3 minutos queda notablemente mejor.
- **Solo tu voz, o la de alguien que te haya dado permiso por escrito.** Clonar la voz de un tercero sin consentimiento es ilegal en la mayoría de países y viola los términos de ElevenLabs.

Música

`music_v1` genera lechos instrumentales por prompt. Suficiente para el fondo. Si buscas un **tema principal** memorable, Suno o Udio dan mejor resultado — no tienen API pública estable, así que se generan en su web y se suben con el botón *Subir* del programa.

4. Anthropic Claude — el cerebro de texto

Cubre: mejorar prompts de imagen y vídeo, escribir prompts de animación, generar título, descripción y etiquetas de YouTube.

Alta

1. <https://console.anthropic.com> → *Sign up*.
2. **Settings** → **API keys** → **Create Key**.
3. Carga saldo en *Billing* (mínimo unos 5 \$, dan para muchísimo).
4. Pégala en Ajustes → `ANTHROPIC_API_KEY`.

Coste

Céntimos por toda la temporada. Es la clave más barata y la que más calidad aporta por euro: reescribir un prompt flojo cuesta menos de un céntimo y te ahorra generar imágenes que no vas a usar.

4b. Freepik / Magnific — el "todo en uno"

Magnific es de Freepik desde que lo compró: una sola clave da acceso a todo.

Cubre: Magnific **Upscaler** (subir resolución y detalle), **Relight** (reiluminar), **Style Transfer** (mismo estilo en toda la serie), **Mystic/Flux** (generación de imagen) y **Kling** (imagen a vídeo).

Alta

1. <https://www.freepik.com/api> → *Get started*.
2. Panel de desarrollador → **API Keys** → crear clave.
3. Pégala en Ajustes → `FREEPIK_API_KEY`.

Se paga por créditos, incluidos en algunos planes de Freepik y comprables aparte.

Por qué importa aquí el Upscaler

Es la mejora de calidad con mejor relación esfuerzo/resultado de todo el programa. El efecto **Ken Burns hace zoom sobre la imagen fija**: si la imagen no tiene resolución de sobra, ese zoom se ve blando y el vídeo entero parece barato. Pasar los planos elegidos por el upscaler antes de montar cambia por completo el acabado, y cuesta céntimos comparado con generar vídeo.

- **2x** basta para 1080p. **4x** si vas a 4K.
- **Creatividad 0** = fiel al original. Súbela solo si quieres que se invente detalle.

Igualar estilo

El problema clásico de una serie hecha con IA es que los planos no parecen de la misma mano. *Igualar estilo* impone el look de un plano ya aprobado sobre los demás. Es más fiable que repetir el mismo sufijo de prompt.

Si algo devuelve 404

Freepik reorganiza sus endpoints cada cierto tiempo. Las rutas son **campos de texto** en Ajustes → Magnific → *Rutas de la API*. Consulta la ruta vigente en <https://docs.freepik.com> y corrígela ahí; no hace falta tocar el programa.

5. OpenAI — alternativa

Cubre: imágenes (`gpt-image-1`), voz (`gpt-4o-mini-tts`), texto.

Alta

1. <https://platform.openai.com/api-keys> → *Create new secret key*.
2. Carga saldo en *Billing* (hace falta: no hay capa gratuita de API).

Cuándo usarlo: `gpt-image-1` respeta muy bien las instrucciones complejas y es el único que escribe texto legible dentro de la imagen — útil para la **tarjeta de título** del plano 1.2 de cada episodio. Su TTS es correcto y barato, pero por debajo de ElevenLabs en español.

5b. Grok Imagine (xAI) — imagen y vídeo con una clave que quizá ya tienes

Si ya usas Grok para escribir, **no hace falta darse de alta otra vez**: la misma `XAI_API_KEY` sirve para imagen y vídeo. En el programa aparecen como dos proveedores (`xAI (Grok)` para texto y `Grok Imagine` para lo visual) porque son endpoints distintos, pero **la clave es una sola**.

1. <https://console.x.ai> → *API Keys* → *Create API key*.
2. Pégala en **Ajustes** → **Claves de API** → **xAI (Grok)**.
3. En **Ajustes** → **Proveedores** elige `Grok Imagine` en *Imágenes* o en *Vídeo*.

	Modelo	Qué da
Imagen	<code>grok-image-image-2.0</code>	JPEG, hasta 10 variantes por petición, 1k o 2k
Vídeo	<code>grok-image-video-1.5</code>	1–15 s, 480p / 720p / 1080p, texto a vídeo o imagen a vídeo

Los dos son **campos de texto** en Ajustes → Modelos: cuando xAI saque el siguiente, se cambia ahí.

Su vídeo genera audio propio —diálogo, ambiente y música—, cosa que ningún otro proveedor del programa hace. **El montaje no lo usa**, y no es un descuido: aquí la duración de cada plano la fija la locución y el clip se repite en bucle para llegar, así que ese audio sonaría cortado. El fichero suelto en `data/` sí lo conserva, por si quieres el clip para otra cosa.

El vídeo tarda. La media son unos 30 s, pero con la cola llena se va a varios minutos. El programa sondea hasta 10 minutos antes de rendirse.

6. fal.ai y Replicate — el catálogo abierto

Dos pasarelas que dan acceso a decenas de modelos con una sola clave. Útiles cuando Veo te rechaza un plano o quieres comparar estéticas.

fal.ai

1. <https://fal.ai> → *Sign up* → <https://fal.ai/dashboard/keys> → *Add key*.
2. Se paga por uso con saldo prepago. Es **rápido y barato**.
3. Modelos que ya vienen configurados: `fal-ai/flux-pro/v1.1-ultra` (imagen), `minimax/h3-max-turbo/image-to-video` (vídeo) y `fal-ai/stable-audio` (música).
4. Vídeo, de más barato a más caro: `minimax/h3-max-turbo/image-to-video` (0,04 €/s a 768P, clips de 5 a 15 s, listos en segundos) · `fal-ai/ltx-2.3/image-to-video/fast` (0,04 €/s a 1080p, hasta 20 s) · `minimax/h3-max/image-to-video` (0,08 €/s) · `fal-ai/kling-video/v3/standard/image-to-video` (0,084 €/s, mudo) · `fal-ai/veo3.1/image-to-video` (0,40 €/s).
5. Las peticiones de vídeo van por **cola** (`queue.fal.run`), no por llamada síncrona: da igual lo que tarde el modelo, la conexión no se queda colgada y no se paga dos veces por un reintento.

Replicate

1. <https://replicate.com> → *Sign up* → <https://replicate.com/account/api-tokens>.
2. Se paga por segundo de GPU.
3. Modelos configurados: `black-forest-labs/flux-1.1-pro` , `kwaivgi/kling-v2.1` , `meta/musicgen` .

En Ajustes → Modelos puedes cambiar cualquiera por otro del catálogo (`owner/nombre`). El código no asume nada del modelo concreto.

Flux es la mejor alternativa a Imagen 4 para el look fotorrealista cinematográfico de esta serie. Para animar imágenes fijas, **MiniMax H3 Max Turbo** es hoy la mejor relación calidad-precio: cuesta la séptima parte que el Kling que venía antes de fábrica y devuelve el clip en segundos en vez de en minutos.

7. Cuánto cuesta un episodio (y cómo gastar poco)

Un episodio son ~8 planos y ~2.000 palabras de narración.

Concepto	Cantidad	Coste orientativo
Imágenes (3 variantes × 8 planos)	24 imágenes	céntimos
Voz en off	~11.000 caracteres	incluido en el plan mensual
Música	1 pista	céntimos
Prompts y metadatos (Claude)	~15 llamadas	céntimos
Vídeo animado (8 clips de 8 s)	64 s	de 2,56 € a 25,60 €, según el modelo

El vídeo cuesta más que todo lo demás junto. Y entre el modelo más caro y el más barato hay **diez veces**, por animar exactamente la misma imagen.

7.1 El modelo importa más que el proveedor

Esta es la tabla que usa el programa para estimar (`server/videomodelos.js`). Los precios son **por segundo de vídeo**, públicos en septiembre de 2026, tomados como 1 \$ = 1 € y redondeados hacia arriba.

Modelo	Proveedor	€/s	Un clip de 8 s	Audio propio
MiniMax H3 Max Turbo 768P	fal.ai	0,040	0,32 €	sí
Veo 3.1 Lite 720p	Gemini	0,050	0,40 €	sí
MiniMax H3 Max 768P	fal.ai	0,080	0,64 €	sí
Veo 3.1 Lite 1080p	Gemini	0,080	0,64 €	sí
Kling 3.0 Standard	fal.ai	0,084	0,67 €	no
LTX 2.3 Fast 1080p	fal.ai	0,040	0,32 €	sí
Veo 3.1 Fast 720p	Gemini	0,100	0,80 €	sí
Grok Imagine 1.5 720p	xAI	0,100	0,80 €	sí
Kling 3.0 Pro	fal.ai	0,112	0,90 €	no
Kling 2.1 Master	Freepik	0,250	2,00 €	no
Kling 2 Master	fal.ai	0,280	2,24 €	no
Veo 3.1 estándar 720p/1080p	Gemini	0,400	3,20 €	sí
Veo 3.1 estándar 4K	Gemini	0,600	4,80 €	sí

Lo que cambió en la versión 0.6.0. Hasta la 0.5.0 el programa venía de fábrica con `veo-3.1-generate-preview` y `fal-ai/kling-video/v2/master/image-to-video`: los dos de la parte cara de la tabla. Ahora viene con **Veo 3.1 Lite** y **MiniMax H3 Max Turbo**, y el mismo episodio de 8 clips pasa de **25,60 €** a **2,56 €**.

Si tenías una de esas dos opciones de fábrica guardadas, el programa **te las cambia solo** al arrancar y lo dice por consola. Si habías elegido tú otro modelo, no se toca nada.

7.2 «auto»: que Google elija el barato

El campo *Gemini · vídeo* admite el valor **auto**, que es el de fábrica. Con él, el programa pregunta a la API de Google **qué modelos Veo puede usar tu clave** y se queda con el más barato de los que existan, en este orden: Lite → Fast → estándar. Cuando Google saque un Veo nuevo, aparecerá solo, sin actualizar el programa.

En Ajustes verás, debajo del selector de proveedor, a qué resuelve **auto** en tu cuenta y **lo que va a costar un clip de 8 segundos** con lo que tienes puesto.

7.3 Resolución y audio

- **La resolución multiplica.** Veo 3.1 Lite cuesta 0,05 €/s a 720p y 0,08 €/s a 1080p; el estándar sube a 0,60 €/s en 4K. Para un vídeo de YouTube que se ve en un móvil, **720p en el clip animado no se distingue**: el montaje final se entrega igual en 1080p.
- **El audio del clip se tira.** El plano dura lo que dura la locución y el clip se repite en bucle para llegar, así que su banda sonora sonaría cortada: el montaje la descarta a propósito. Por eso el programa pide los clips **mudos** cuando el modelo lo permite. En Veo tiene además una segunda ventaja: es el audio lo que más veces dispara el filtro de contenido de Google.

7.4 La estrategia que recomiendo

1. **Monta el episodio entero solo con imágenes.** El Ken Burns del programa lee la indicación de cámara del guion y hace el movimiento que pedía el doc 04. Para un documental histórico es un acabado perfectamente profesional.
2. **Míralo entero.** Localiza los 2 o 3 planos que piden movimiento real: el gancho, el clímax, el plano final.
3. **Anima solo esos,** y con el modelo barato. Con H3 Max Turbo, tres clips cuestan menos de un euro.
4. Reserva Veo 3.1 estándar para el **plano estrella** o para el tráiler.

8. Privacidad y buenas prácticas

- Las claves se guardan en el fichero `.env` **de tu ordenador** y no se envían a ningún sitio salvo al proveedor correspondiente. Están en `.gitignore`.
- El programa escucha solo en `127.0.0.1`: nadie de tu red puede entrar.
- Los prompts sí viajan al proveedor que elijas. Son descripciones de escenas históricas: no hay nada sensible, pero tenlo presente.
- **Pon límites de gasto** en cada consola (Google Cloud → *Budgets & alerts*; Anthropic y OpenAI → *Usage limits*). Es la única red de seguridad real contra un lote lanzado por error.
- Nada en este programa se ejecuta solo. No hay tareas programadas ni reprocesado automático: cada llamada de pago sale de un clic tuyo.

9. Enlaces rápidos

Servicio	Alta	Claves	Precios
Google AI Studio	https://aistudio.google.com	https://aistudio.google.com/apikey	https://ai.google.dev/pricing
ElevenLabs	https://elevenlabs.io	https://elevenlabs.io/app/settings/api-keys	https://elevenlabs.io/pricing
Anthropic Claude	https://console.anthropic.com	Settings → API keys	https://anthropic.com/pricing
OpenAI	https://platform.openai.com	https://platform.openai.com/api-keys	https://openai.com/api/pricing
Freepik / Magnific	https://www.freepik.com/api	Panel → API Keys	https://www.freepik.com/api/pricing
fal.ai	https://fal.ai	https://fal.ai/dashboard/keys	https://fal.ai/pricing
Replicate	https://replicate.com	https://replicate.com/account/api-tokens	por modelo, en su ficha

📺 Generador de Contenido **Del libro (o de una idea) al vídeo de YouTube terminado, sin abrir un editor de vídeo.**
 ** [[Licencia: AFL-3.0](https://img.shields.io/badge/Licencia-AFL--3.0-blue.svg)](LICENSE) [[Node.js](https://img.shields.io/badge/Node.js-%E2%89%A518.17-339933?logo=node.js&logoColor=white)]
 ([https://nodejs.org]) [[Windows · macOS · Linux]
 ([https://img.shields.io/badge/Windows%20%C2%B7%20macOS%20%C2%B7%20Linux-0078D4?logo=windowsterminal&logoColor=white)](#instalación) [[Dependencias]
 ([https://img.shields.io/badge/dependencias-3-brightgreen.svg)](#cómo-está-hecho) [[Local]
 ([https://img.shields.io/badge/tus%20claves-nunca%20salen%20de%20tu%20PC-success.svg)](#privacidad) Escribe los guiones con IA, genera las imágenes, clona tu voz, y monta el episodio en MP4 con rótulos, subtítulos y capítulos. Todo en tu ordenador. [Instalación](#instalación) · [Cómo funciona](#cómo-funciona) · [IAs que conecta](#ias-que-conecta) · [Documentación](#documentación) **Español** · [English](README.en.md)
 ![Cuatro planos de un episodio montado con el programa](docs/img/demo.gif) **Esto no es un montaje de ejemplo: es un episodio real, entero, hecho con el programa.** Guion escrito por IA, imágenes generadas, voz clonada y montaje automático. ### ▶ [Episodios completos en YouTube]
 ([https://www.youtube.com/@podcastElOcasoRomano])

Hecho con esto

[imagen no encontrada: docs/img/muestras.jpg] />

Cuatro episodios distintos de la misma serie. El **canal entero** —guiones, voz, imágenes y montaje— sale de este programa: [youtube.com/@podcastElOcasoRomano](https://www.youtube.com/@podcastElOcasoRomano)

Qué es

Una aplicación local (servidor Node + interfaz web) que cubre **toda la cadena de producción** de un canal de YouTube o un podcast:

```

un libro · un PDF · un EPUB · un artículo · o solo un tema
      ↓
👉 la IA escribe los guiones
(vídeo + podcast + prompts de imagen y vídeo + biblia)
      ↓
🖼️ imágenes   🗣️ tu voz clonada   🎬 clips   🎵 música
      ↓
📺 MP4 montado + SRT + capítulos de YouTube
  
```

No es un generador de vídeos de un clic. Es un **estudio**: en cada paso decides tú —eliges qué imagen de las tres que te propone, editas el prompt, regeneras lo que no te convence— pero no tocas una línea de tiempo ni ajustas un fotograma.

La idea que lo hace funcionar

La duración de cada plano la fija la locución. El narrador tarda 43 segundos en decir su párrafo → ese plano dura 43 segundos más los silencios de entrada y salida. No hay que cuadrar nada a mano, y no se desincroniza: el desfase

medido en un episodio de 7 minutos y medio es de **46 milisegundos**.

Y si no generas vídeo (que es, con diferencia, lo más caro), los planos **no se quedan quietos**: se les aplica un Ken Burns que lee la indicación de cámara del propio guion — «dolly-in» hace un acercamiento, «paneo de izquierda a derecha» un barrido, «tilt ascendente» sube. Gratis, y con el acabado de un documental de toda la vida.

Instalación

Opción A — Descargar la aplicación

Todo va dentro: **Node.js y ffmpeg incluidos**, no hay que instalar nada más. Descarga de [Releases](#) el que te toque:

Sistema	Fichero	Notas
Windows	GeneradorContenido-*-x64.zip	Empieza por este. Descomprime y ejecuta <code>Generador de Contenido.exe</code> . Es el que menos problemas da con los antivirus
	GeneradorContenido-*-x64.exe	Instalador, con accesos directos
	GeneradorContenido-portable-*.exe	Un solo fichero, no instala nada
macOS	GeneradorContenido-*-mac-arm64.dmg	Apple Silicon (M1 y posteriores)
	GeneradorContenido-*-mac-x64.dmg	Intel
Linux	GeneradorContenido-*-linux-x86_64.AppImage	Vale en cualquier distribución. <code>chmod +x</code> y a correr
	GeneradorContenido-*-linux-amd64.deb	Debian, Ubuntu y derivadas
	GeneradorContenido-*-linux-x64.tar.gz	Descomprimir y ejecutar, sin instalar nada

Las sumas `SHA256SUMS.txt` van en cada release, por si quieres comprobar que el fichero es el que se publicó.

macOS: «la aplicación está dañada»

No lo está. Firmar una aplicación de Mac cuesta **99 \$ al año**, y sin firma macOS la pone en cuarentena. Se quita el marcaje con una orden:

```
xattr -cr "/Applications/Generador de Contenido.app"
```

Linux: el AppImage

```
chmod +x GeneradorContenido-*.AppImage
./GeneradorContenido-*.AppImage
```

Para que los rótulos y los subtítulos salgan con una tipografía decente, ten instaladas las DejaVu (`fonts-dejavu` en Debian/Ubuntu). El programa las usa solo en Linux, donde Georgia y Arial no vienen de serie.

Es una aplicación de escritorio normal: **su propia ventana, su icono y su menú**. Sin consola negra, sin navegador y sin escribir ninguna dirección. Node.js y ffmpeg van dentro.

Windows SmartScreen avisará de que el programa no está firmado digitalmente (una firma cuesta cientos de euros al año). Pulsa **Más información → Ejecutar de todas formas**.

Si tu antivirus lo marca como amenaza

Es un falso positivo, y aquí está la comprobación, no la promesa. ESET, por ejemplo, borra el instalador y el portable acusándolos de `Backdoor.PHP/Webshell.NBV`.

Qué se comprobó en la máquina donde pasó, con el escáner de línea de comandos del propio ESET (`ec1s.exe`):

Qué se escaneó	Resultado
Código fuente (<code>server/</code> , <code>public/</code> , <code>electron/</code>) — 39 ficheros	0 detecciones
Build sin empaquetar (<code>win-unpacked</code>) — 77 ficheros, 610 objetos	0 detecciones
Instalador, portable y ZIP recién generados	0 detecciones
Búsqueda de <code><?php</code> , <code>shell_exec</code> , <code>passthru(</code> , <code>base64_decode</code> , <code>eval(\$_</code> en el <code>.exe</code> y en el <code>app.asar</code>	0 coincidencias
Ficheros <code>.php</code> dentro de lo que se empaqueta	ninguno

Es decir: el motor de firmas local dice que está limpio. El borrado venía con prefijo `@` , que en ESET marca **veredicto de LiveGrid (reputación en la nube)**, no de firma. Un ejecutable **autoextraíble, recién compilado y sin firmar** no tiene reputación, y el sistema lo agrupa con muestras maliciosas. El nombre «webshell» encaja con lo que el programa hace de verdad —levanta un servidor HTTP local y lanza `ffmpeg` —, que es también la definición técnica de un webshell.

Qué puedes hacer, de menos a más:

1. **Usa el ZIP**. No es autoextraíble, así que no dispara la heurística. Descomprime donde quieras y ejecuta el `.exe` de dentro.
2. **Compílalo tú** (Opción B). Si no te fías de un binario, y haces bien, el código está entero aquí.
3. **Añade una exclusión** para la carpeta del programa, si decides fiarte del binario.
4. **Reporta el falso positivo** a tu antivirus. Con ESET: `samples@eset.com` con asunto *False Positive*, adjuntando el fichero en un ZIP con contraseña `infected` y el SHA-256. Suelen corregirlo en 24-48 h y beneficia a todo el mundo.

La solución de fondo es un **certificado de firma de código** (200-400 €/año). Sin él, cualquier ejecutable nuevo de Windows arranca con reputación cero.

Opción B — Desde el código

```
git clone https://github.com/ecomysteo/generador-de-contenido.git
cd generador-de-contenido
npm install
npm start          # abre la aplicación de escritorio
```

Requiere Node.js ≥ 18.17 . `ffmpeg` se instala solo como dependencia npm.

```
npm run server      # solo el motor, para usarlo desde el navegador
npm run doctor      # comprueba ffmpeg, claves y guiones
npm test            # las pruebas, sin gastar API

npm run build       # compila para Windows
npm run build-mac   # para macOS
npm run build-linux # para Linux
```

Cada sistema se compila en su propio sistema. El instalador de macOS necesita `hdiutil`, que solo existe en macOS, y `ffmpeg-static` descarga únicamente el binario de la máquina donde se instala. Para no necesitar los tres ordenadores está `.github/workflows/build.yml`: empujas una etiqueta `v*` y GitHub compila los tres y prepara la release.

La misma interfaz sirve para los dos modos: dentro de la aplicación de escritorio usa los **diálogos nativos de Windows**; en el navegador cae a los del navegador.

Configurar las claves

```
copy .env.example .env          # en Linux o macOS: cp .env.example .env
```

Abre el `.env` y rellena **solo las que vayas a usar**: ninguna es obligatoria y el programa arranca sin ninguna. También se pueden pegar desde **Ajustes** dentro de la aplicación, sin tocar ficheros.

`.env` está en el `.gitignore`. **Este repositorio no lleva ninguna clave**, y la API del programa nunca devuelve el valor de una clave: solo si existe o no.

Qué no viene en el repositorio, a propósito

Ruta	Por qué
<code>.env</code>	Claves de API. Copia <code>.env.example</code> y rellénalo tú.
<code>data/</code>	Tus proyectos, tus ajustes y tus grabaciones de voz. Se crea sola al arrancar, con los valores por defecto de <code>server/config.js</code> .
<code>output/</code>	Los MP4 montados.
<code>media/</code>	Material generado.
<code>dist/</code>	Compilados. Los genera <code>npm run build</code> .

Un clon recién hecho arranca y funciona: `npm install && npm start` , y a poner claves.

Cómo funciona

1 · Crear la serie

[imagen no encontrada: docs/img/ui/01-crear.jpg] />

Pestaña **Crear**. Eliges de dónde sale el contenido:

Fuente	Qué hace
Un tema	La IA escribe la serie desde su propio conocimiento
Un fichero	PDF, DOCX, EPUB, TXT, MD, HTML, RTF
Texto pegado	Un artículo, unas notas, un capítulo
Una dirección web	Blog, ficha de producto, artículo. Se limpia el menú y los banners... y se usan las fotos de la propia página
Una carpeta de módulo	Lee la documentación de un programa y escribe su vídeo de ayuda

Y qué quieres que sea: **documental, vídeo de YouTube, videoensayo, podcast narrativo, curso, vídeo corto vertical** (TikTok · Reels · Shorts), **vídeo de producto, manual de un programa o ficción sonora**. Más el número de episodios, la duración, el idioma, el público y el tono.

Cada formato se autoconfigura: orientación, resolución, duración típica, subtítulos y ritmo de corte. La duración se pide en **segundos**, y el programa calcula cuántas palabras caben de verdad —descontando el silencio de cada corte— para que un TikTok de 21 s dure 21 s y no 31.

Las fotos salen de la web, no de la IA

Cuando la fuente es una URL, los planos usan las **fotos reales de esa página**. Para una ficha de producto es la diferencia entre un vídeo que sirve y uno que no.

Y antes de escribir nada, **la IA mira cada foto**. El texto alternativo suele estar vacío —en la prueba con Wikipedia lo estaba en las 22 imágenes—, así que sin verlas escribiría a ciegas. Con visión sabe que la foto 4 es «*un científico con bata blanca examinando una placa de Petri*», elige cuál va en cada plano y **narra lo que se ve en ella**. Lo que la vista reconoce como logo o banner se descarta. Solo lo que no existe en la página se genera con IA.

La IA genera el kit completo:

00_README_KIT.md	índice
01_GUION_YOUTUBE.md	guion de vídeo con marcadores
02_GUION_PODCAST.md	versión sonora
03_PROMPTS_IMAGENES.md	prompts de imagen por plano
04_PROMPTS_VIDEOS.md	prompts de vídeo + movimiento de cámara
05_BIBLIA_PRODUCCION.md	tono, estilo, voz, música, pipeline

Un libro entero no se manda de golpe: se resume por partes, luego se diseña la escaleta, y después se escribe episodio a episodio. Los PDF se envían tal cual al modelo (Gemini y Claude los leen mejor que cualquier extractor).

Ya tienes tus guiones escritos? Perfecto. Ponlos en el formato de marcadores ([VOZ] , [IMAGEN IA] , [RÓTULO] , [MÚSICA] , [ESCENA]) y pulsa *Importar*. El generador es opcional.

2 · Tu voz

[imagen no encontrada: docs/img/ui/02-mi-voz.jpg] />

Pestaña **Mi voz**. Grabas cinco párrafos desde el navegador —están escritos a propósito para cubrir registros distintos: narración neutra, preguntas, cifras, tensión y cierre cálido— y el programa crea una **voz clonada tuya** en ElevenLabs que locuta toda la serie.

Con 60 segundos funciona; con 2–3 minutos queda notablemente mejor.

3 · Producir

Pestaña **Editor**. Los botones van numerados en el orden correcto:

1 · Imágenes → 1b · Ampliar (Magnific) → 2 · Voz → 3 · Animar → 4 · Música → 5 · MONTAR

- **Imágenes** — 3 variantes por plano; eliges la que quieras con un clic.
- **Ampliar** — Magnific Upscaler. Importa más de lo que parece: el Ken Burns hace zoom sobre la imagen fija, y sin resolución de sobra ese zoom se ve blando. También hay **reiluminar** e **igualar estilo** (impone el look de un plano aprobado sobre el resto, para que la serie case).
- **Voz** — locuta cada plano. Aquí es donde se fija el ritmo real del episodio.
- **Animar** — *opcional*. Convierte la imagen en un clip de 5–8 s. Los labios se sincronizan solos: ver abajo.
- **Música** — lecho instrumental que baja solo cuando habla el narrador (*ducking* por cadena lateral).
- **Montar** — MP4 1920×1080 + `.srt` + capítulos + `.ass` de rótulos.

👁 Previo: verlo antes de montarlo

Montar un episodio de 200 planos son horas. Solo para ver si una imagen pega con lo que se dice encima, no compensa.

El **Previo** enseña cada plano a pantalla completa con su locución sonando, y **no codifica nada**: es instantáneo. Como cada plano dura lo que dura su audio —el mismo criterio que usa el montaje— **el ritmo que ves ahí es el que va a salir**.

Espacio pausa, ← → se mueven, Esc cierra.

Varias imágenes en un mismo plano

Un bloque de narración largo sobre una sola foto se hace pesado. **Ctrl+clic** en las miniaturas añade imágenes a la secuencia del plano y se funden entre sí. El plano dura lo mismo: se reparte entre las que tenga.

Antes de montar, avisa

Al darle a montar, revisa el episodio y **para** si hay planos sin imagen, imágenes generadas pero sin elegir, o texto sin locutar. Puedes confirmar y montar igual —para ver cómo va un episodio a medias es lo normal— pero ya no

descubres después de ocho horas que catorce planos salieron en negro.

Los labios, que es donde se nota el truco

Los modelos de vídeo, en cuanto ven una cara, la ponen a hablar. Siempre. Y en un documental la voz es de un **narrador que no está en pantalla**, así que lo que salía era un romano moviendo la boca sin decir nada, descuadrado con una voz que no es suya. Da el pego medio segundo y luego canta muchísimo.

El guion ya sabe quién habla en cada bloque, así que basta con decírselo al modelo:

En el guion	Qué se le pide al modelo de vídeo
[VOZ] sin nombre, o [VOZ:NARRADOR]	Voz en off: todas las bocas cerradas y quietas . La cara vive por los ojos, el parpadeo y la respiración
[VOZ:BRUNO]	Labios, mandíbula y lengua sincronizados con la frase literal de Bruno , en el idioma del audio

Y cuando hablan dos, solo mueve la boca el que tiene el turno: el otro escucha con la boca cerrada.

Se apaga en **Ajustes** → **Estilo** → **Movimiento de labios** si prefieres el comportamiento de antes.

Subtítulos con la palabra resaltada

Lo que funciona en vertical: se ve marcada la palabra que se está diciendo, y eso retiene bastante mejor que un bloque de texto quieto. El `.srt` normal sigue saliendo aparte para subirlo a YouTube.

La sincronía es **aproximada**: se calcula por la longitud de cada palabra, no escuchando el audio. Para seguir la frase con la vista va de sobra; para doblaje no valdría.

Y los **rótulos tienen seis diseños** —documental, noticias, minimal, impacto, manuscrito, técnico—, cada uno con su fuente, color, tamaño y posición.

Careta, cierre y zona segura

Pones un MP4 delante y otro detrás, y se adaptan solos a la resolución y los fotogramas del episodio. **Opcionales**.

Y los rótulos se apartan de donde las apps ponen su interfaz: en vertical, TikTok se come alrededor del 20 % de abajo y YouTube el 10 % de arriba.

Buscar y reemplazar

Ctrl+F. Por toda la narración, los rótulos, los prompts y las notas. El reemplazo se prueba antes de aplicarse y **avisa de los planos que ya tienen la voz grabada**: cambiar el texto no cambia el audio, y esos hay que volver a locutarlos.

El sonido sale al nivel de YouTube

Sin tocar nada. Las plataformas **normalizan todo** lo que subes a unos -14 LUFS: si tu episodio va mas bajo, te lo suben y con el sube el ruido de fondo; si va mas alto, te lo bajan y pierde pegada. Ahora sale ya en su sitio, que suena mejor que dejar que lo hagan ellos.

Y el Ken Burns **acelera y frena** en vez de arrancar de golpe y parar en seco, que es como se mueve una cámara de verdad.

Borrador rapido

Monta el episodio **de verdad** pero a resolución baja y a toda velocidad: en las pruebas, mas del doble de rapido. Dura exactamente lo mismo que el montaje bueno, así que el ritmo que ves es el que va a salir.

Remontar cuesta segundos, no horas

Cambias la imagen de un plano y vuelves a montar: **solo se rehace ese plano**. Un segmento es determinista —sale de unas imágenes, una duración, una cámara y unos ajustes— así que se guarda una huella de esas entradas y, si nada ha cambiado, se reaprovecha.

Y si el montaje se corta a medias, al volver **sigue donde estaba**: el progreso se anota cada diez planos, no al final, que es justo el caso que duele.

Además **te dice cuánto va a tardar** antes de empezar. La primera vez es a ojo y lo avisa; después el número sale de lo que tardaron tus propios montajes.

Y avisa de los planos que se van a hacer eternos

Más de 90 segundos sobre una sola imagen fija es una foto quieta minuto y medio: pasados los ~107 s el Ken Burns es tan lento que ni se percibe. La revisión previa te los señala, y ahora tiene arreglo — repartir el plano entre varias imágenes.

Ritmo

Ajustes → Montaje → Ritmo multiplica los silencios que rodean la voz: *ágil* para redes, *pausado* para documental narrado, *solemne* para dejar mucho aire. Es lo que más cambia la sensación del episodio. Un plano concreto puede llevar los suyos y mandan sobre esto.

Que no tarde ocho horas

Un capítulo de 200 planos llegó a tardar **ocho horas**. Medido dónde se iba el tiempo, salieron tres cosas:

El codificador estaba escrito a mano — todo pasaba por `libx264 -preset medium`, que es excelente y también lo más lento. Pero ffmpeg trae compilados NVENC, Quick Sync y AMF **siempre**, estén o no la tarjeta y el driver, así que no basta con mirar la lista: hay que probarlos. Eso hace ahora al arrancar, y elige el mejor que funcione de verdad.

Medido con `scripts/bench-codec.js` sobre 6 s de documental a 1080p, en un portátil Intel m3 de dos núcleos:

Codificador	Tiempo	VMAF
h264_qsv (GPU Intel)	2,2 s	94,8
h264_mf	3,8 s	93,5
libx264 -veryfast	6,9 s	93,8
libx264 -medium ← lo de antes	20,6 s	96,3

VMAF es la métrica de Netflix: 95+ es indistinguible, 90+ muy bueno. **La GPU es 9,4 veces más rápida perdiendo punto y medio que nadie ve.**

Los planos se montaban de uno en uno — con 200 planos, la máquina se pasaba horas con un solo núcleo ocupado. Ahora van varios a la vez, con el límite puesto por los núcleos y, si se codifica por GPU, por cuántas sesiones admite la tarjeta: pasarse de ahí hace que las de más fallen a media faena.

El desenfoque del fondo se recalculaba en cada fotograma — 1.800 veces por plano, sobre una imagen fija, con resultados idénticos. Ahora una sola vez: 28,2 s → 14,0 s.

Los tres se multiplican. En **Ajustes** → **Montaje** te dice qué ha encontrado en tu máquina.

IAs que conecta

Tarea	Proveedores	Recomendado
Guiones y prompts	Claude · Gemini · OpenAI · Grok	Claude
Imágenes	Gemini · Grok Imagine · Freepik (Mystic) · fal.ai (Flux) · Replicate · OpenAI	Gemini
Ampliar / reiluminar / estilo	Magnific (Freepik)	Magnific
Vídeo	Gemini (Veo 3.1 Lite/Fast/estándar) · fal.ai (MiniMax H3 , LTX, Kling, Veo) · Grok Imagine · Replicate · Freepik	Veo 3.1 Lite o MiniMax H3 Max Turbo
Voz y clonado	ElevenLabs · OpenAI TTS · Gemini TTS	ElevenLabs
Música	ElevenLabs Music · Replicate · fal.ai	ElevenLabs

[imagen no encontrada: docs/img/ui/03-ajustes.jpg] />

Se elige proveedor **por tarea** en Ajustes. **Ninguna clave es obligatoria:** funciona con las que tengas. Las claves viven en un `.env` local y no salen de tu ordenador.

Los nombres de modelo y las rutas de API son **campos de texto editables**. Cuando un proveedor renombra un modelo o mueve un endpoint —y lo hacen a menudo— se corrige en Ajustes sin tocar el código.

Cuánto cuesta

Para una serie de 10 episodios:

Concepto	Coste orientativo
Guiones (IA de texto)	céntimos
Imágenes (3 variantes × ~8 planos × 10 eps)	unos pocos euros
Voz en off + clonado	plan mensual de ElevenLabs (~5–22 €)
Música	céntimos
Vídeo generativo	decenas de euros por episodio

El vídeo cuesta más que todo lo demás junto. **Monta primero con Ken Burns, míralo, y anima solo los 2–3 planos que de verdad lo pidan.** El tráiler es donde mejor se aprovecha.

Pon límites de gasto en la consola de cada proveedor. Es la única red de seguridad real.

Documentación

Documento	Para quién
Manual de usuario	De cero a episodio publicado
Documentación técnica	Arquitectura, API, motor de montaje
Guía de IAs y altas	Qué contratar y cómo registrarse

Cómo está hecho

```
server/
  index.js      arranque Express
  config.js     .env + ajustes + rutas (modo desarrollo y modo .exe)
  generator.js  fuente → escaleta → guiones (el kit completo)
  extract.js    PDF/DOCX/EPUB/TXT → texto (lector ZIP propio, sin dependencias)
  web-read.js   una URL → Markdown limpio, con sus imágenes reales
  labios.js     sincronía labial: quién habla en cada plano y quién calla
  codecs.js     qué sabe codificar esta máquina (los prueba de verdad)
  revision.js   si el episodio está listo, antes de gastar horas
  cachemontaje.js no rehacer los planos que no han cambiado
  estimar.js    cuánto va a tardar, medido de tus montajes anteriores
  rotulos.js    diseños de rótulo y subtítulos con resalte
  buscar.js     buscar y reemplazar por todo el episodio
  costes.js     cuánto va a costar, antes de darle al botón
  parser.js     markdown con marcadores → episodios y planos
  project.js    estado en disco, multi-proyecto, gestión de material
  studio.js     operaciones de producción y orquestación del montaje
  ffmpeg.js     Ken Burns, concatenado, mezcla con ducking, rótulos ASS, SRT
  voces.js      grabación, limpieza de audio y clonado de voz
  jobs.js       cola en segundo plano con progreso por SSE
  routes.js     API HTTP
  providers/    gemini · claude · openai · grok · elevenlabs · freepik · fal · replicate
public/        interfaz (HTML + JS sin framework, sin build)
```

Tres dependencias: `express`, `ffmpeg-static`, `ffprobe-static`. Nada más.

Añadir un proveedor = un fichero en `providers/` que exporte `{id, name, caps, ...métodos}` y una línea en el registro.

Pruebas

```
npm run doctor          # entorno, claves y parseo de guiones
node test-labios.js      # 62 comprobaciones: sincronía labial por proveedor y alta de Grok
node test-montaje.js     # 117 comprobaciones
node test-buscar.js      # 19 de buscar/reemplazar y estimación de coste
npm run comprobar-ui     # carga la interfaz en Electron y busca errores: duración exacta, encajes,
                        # cámara, varias imágenes,
                        # codificadores y revisión previa
node scripts/bench-codec.js tu-video.mp4 # qué codificador conviene en TU máquina
node test-render.js 1    # monta un episodio con material sintético, sin gastar API
```

`test-render.js` fabrica imágenes y locuciones con ffmpeg, monta el episodio completo y **falla si el desfase de sincronía supera 0,5 s**. Es la prueba de regresión del motor de montaje.

Privacidad

- El servidor escucha solo en `127.0.0.1`. No es accesible desde la red.
- Las claves están en `.env` (en `.gitignore`). La API nunca devuelve su valor, solo si existe.

- **Nada se ejecuta solo.** No hay tareas programadas ni reprocesado automático: cada llamada de pago sale de un clic tuyo.
 - Tus grabaciones de voz se quedan en `data/voz/`. Solo se envían a ElevenLabs cuando pulsas *Crear mi voz clonada*.
-

Contribuir

Los fallos y las ideas van a [Issues](#); hay plantilla para cada cosa.

Si mandas un *pull request*:

- **Sin dependencias nuevas.** Son tres a propósito, y la gracia está justo ahí. Si algo se puede hacer con el Node de serie, se hace con el Node de serie.
- **Un proveedor nuevo** = un fichero en `server/providers/` que exporte `{ id, name, caps, ...métodos }`, más una línea en el registro de `providers/index.js`. No hay que tocar nada más.
- **Antes de abrir el PR**, pasa las dos pruebas: `bash npm run doctor node test-render.js 1`. La segunda falla sola si el desfase de sincronía se pasa de 0,5 s.
- **Nunca subas un `.env`**, ni una clave, ni una ruta tuya de disco.

Quién lo hace

Lo escribe **Ecom Experts**. Si te dedicas al comercio electrónico, el otro sitio donde ando es [gmartos.es](#): módulos, tutoriales y ayuda para **PrestaShop en español**. No tiene que ver con esto, pero por si te sirve.

El canal donde se ve el resultado: youtube.com/@podcastElOcasoRomano.

Licencia

[AFL-3.0](#) · *Ecom Experts*

Las herramientas de IA que conecta tienen sus propios términos y precios. Este programa no incluye ninguna clave ni crédito.

GENERADOR DE CONTENIDO — Documentación técnica

Versión: 1.0.0 · **Autor:** Ecom Experts · **Licencia:** AFL-3.0 **Stack:** Node.js ≥18.17 · Express 4 · ffmpeg-static · front vanilla (sin framework, sin build)

1. Arquitectura

Aplicación local cliente-servidor. El servidor Node hace de orquestador entre las APIs de IA y `ffmpeg`; el navegador es solo la interfaz. No hay base de datos: **el estado vive en un único JSON** y los binarios en disco.

```

generador-de-contenido/
├── ARRANCAR.bat           Lanzador Windows (instala deps la primera vez)
├── package.json
├── .env                   Claves de API (creado desde .env.example, NO se comparte)
├── test-render.js         Prueba del motor de montaje sin gastar API
├──
├── server/
│   ├── index.js           Arranque Express, estáticos, apertura del navegador
│   ├── config.js          Parser .env propio + ajustes persistentes + claves
│   ├── generator.js        Fuente -> escaleta -> guiones (escribe el kit .md)
│   ├── extract.js         PDF/DOCX/EPUB/TXT -> texto (lector ZIP propio)
│   ├── voces.js           Grabación, limpieza de audio y clonado de voz
│   ├── parser.js          Markdown del guion -> episodios / planos
│   ├── project.js         Estado en data/project.json + gestión de assets
│   ├── studio.js          Operaciones de producción y orquestación del render
│   ├── ffmpeg.js          Motor de montaje (Ken Burns, concat, mezcla, ASS/SRT)
│   ├── jobs.js            Cola en memoria con progreso por SSE
│   ├── routes.js          API HTTP
│   ├── doctor.js          Diagnóstico del entorno (npm run doctor)
│   └── providers/
│       ├── index.js       Registro y enrutado capacidad -> proveedor
│       ├── http.js        fetch con reintentos, backoff y timeouts
│       ├── gemini.js      Imagen 4 · Veo · Gemini texto · Gemini TTS
│       ├── openai.js      gpt-image-1 · TTS · GPT
│       ├── elevenlabs.js  Voz · Music · Sound effects
│       ├── claude.js      Messages API (texto)
│       ├── replicate.js   Flux · Kling · MusicGen (predicciones con sondeo)
│       ├── fal.js         Flux · Kling · Veo · audio (síncrono)
│       └── freepik.js      MAGNIFIC: upscale, relight, style · Mystic · Kling
├── public/               index.html · app.js · styles.css
├── docs/                 Esta documentación
├── data/                 [generado] project.json, settings.json, assets/
└── output/               [generado] MP4, MP3, SRT, capítulos
  
```

2. El parser (`parser.js`)

2.1 Gramática reconocida

```
# EPISODIO <n> — <TÍTULO>
**~15 min · Libro I · <subtítulo>**
### <Sección> — <timecode>
[IMAGEN IA] <prompt en inglés>
[VOZ] <narración>
[RÓTULO] <texto en pantalla>
[MÚSICA] <indicación>
[ESCENA] <dramatización>
```

También reconoce las variantes del guion de podcast: `[SONIDO]` , `[ESCENA SONORA]` , `[DECIR FECHA]` , y las etiquetas sin tilde (`[ROTULO]` , `[MUSICA]`).

Un beat puede ocupar varias líneas: las continuaciones se concatenan hasta una línea en blanco o un nuevo marcador.

Hay dos marcadores más que **también abren plano**, y que producen planos que no se generan con IA:

Marcador	shot.kind	Origen de la imagen
<code>[IMAGEN IA] <prompt></code>	<code>ai-image</code>	La genera el proveedor de imagen
<code>[CAPTURA] Pantalla: ... \ Se debe ver: ... \ Cómo llegar: ...</code>	<code>screenshot</code>	La hace el usuario; se valida por visión
<code>[IMAGEN WEB] <n> — <por qué></code>	<code>web-image</code>	Foto real descargada de la página de origen

El número va FUERA del corchete. `TAG_RE` es `/^\[([A-ZÁÉÍÓÚÑ ]+)\]\s*(.*)$/` : dentro de los corchetes solo caben letras y espacios, así que `[IMAGEN WEB 3]` **no casaría**. Como el modelo lo escribe así la mitad de las veces, `normalizeScript()` lo reescribe a `[IMAGEN WEB] 3 ...` antes de parsear (también acepta `[FOTO WEB n]` y `[WEB n]`). El número acaba en `shot.webImageIndex` .

2.2 Construcción de planos

Regla: cada marcador de imagen abre un plano. Todo lo que sigue —voz, rótulos, escena, música— pertenece a ese plano hasta el siguiente.

Los beats que aparecen **antes** del primer `[IMAGEN IA]` del episodio (típicamente un `[MÚSICA]` de arranque) se acumulan en un buffer y se vuelcan en el primer plano real. Sin esto se creaban planos fantasma sin imagen y toda la numeración se desplazaba.

Cada plano recibe una clave `<episodio>.<ordinal>` — `1.1` , `1.2` ... — que es exactamente la nomenclatura de los documentos `03_PROMPTS_IMAGENES.md` (`**1.3**`) y `04_PROMPTS_VIDEOS.md` (`**Plano 1.3**`). Así se enriquece cada plano con:

- el prompt canónico de imagen (doc 03),
- el prompt de vídeo, la indicación de cámara y la duración sugerida (doc 04),

- y los títulos SEO de la biblia (doc 05 §8).

Del kit actual salen **10 episodios, 72 planos, 125 bloques de voz y 49 rótulos**, cifras que coinciden con el recuento directo de marcadores en los `.md`.

3. Modelo de datos (`data/project.json`)

```
{
  "name": "El OCASO ROMANO – Serie documental",
  "sourceDir": "...",
  "episodes": [{
    "n": 1, "title": "...", "subtitle": "...", "seoTitle": "...",
    "wordCount": 1013,
    "music": { "prompt": "...", "file": "assets/ep01/music/...mp3" },
    "youtube": { "title": "...", "description": "...", "tags": [] },
    "podcast": { "narration": [...], "outputFile": null },
    "outputFile": null,
    "shots": [{
      "id": "ep01_s01", "n": 1, "key": "1.1",
      "section": "Gancho – 0:00",
      "imagePrompt": "...", "videoPrompt": "...",
      "camera": "dolly-in muy lento (push-in), 6 s",
      "suggestedDuration": 6,
      "narration": [...], // líneas [VOZ]
      "titles": [...], // líneas [RÓTULO]
      "scene": "...", // [ESCENA]
      "music": "...", // [MÚSICA]

      "images": [{ "file": "assets/ep01/images/...png", "provider": "gemini", "model": "imagen-4.0-generate-001", "prompt": "...", "createdAt": "..." }],
      "selectedImage": "assets/ep01/images/...png",
      "videos": [{ "file": "...", "duration": 6.0, ... }],
      "selectedVideo": null,
      "voiceFile": "assets/ep01/voice/...mp3",
      "voiceDuration": 43.7,
      "locked": false
    }]
  }]
}
```

Rutas: en el JSON siempre son relativas a `data/`, con `/` como separador. `project.abs()` y `project.rel()` hacen la conversión. Eso permite servirlos tal cual desde `/assets/<ruta>` y mover la carpeta del proyecto sin romper nada.

Reimportación no destructiva. `importFromSource()` reparsea el markdown y reinyecta los assets del proyecto anterior emparejando por `shot.id`. Si un plano está `locked`, además conserva sus prompts editados a mano.

4. Capa de proveedores

4.1 Contrato

Cada proveedor exporta `{ id, name, caps, ...métodos }`. El registro enruta capacidad → método:

Capacidad	Método	Firma de entrada	Devuelve
image	generateImage	{prompt, negative, aspect, count}	[{buffer, ext, model, provider}]
video	generateVideo	{prompt, imageBuffer\ imageDataUri, durationSec, aspect, onProgress}	{buffer, ext, ...}
voice	tts	{text, voice, model}	{buffer, ext, ...}
music	music	{prompt, durationSec}	{buffer, ext, ...}
text	text	{system, prompt, maxTokens}	string

`providers.call(cap, args, forzado?)` resuelve el proveedor desde `settings.providers[cap]`, comprueba que hay clave y lanza un error legible si no. Añadir un proveedor nuevo = un fichero en `providers/` + una línea en el registro.

4.2 Detalles por proveedor

Gemini (`https://generativelanguage.googleapis.com/v1beta/models`, cabecera `x-goog-api-key`) - Imagen: `POST /{modelo}:predict` con `{instances:[{prompt}], parameters:{sampleCount, aspectRatio}}`. Devuelve `predictions[].bytesBase64Encoded`. - **Imagen 4 ya no acepta `negativePrompt`** (responde `400 Setting negativePrompt is no longer supported`). El prompt negativo de Ajustes solo se envía a los proveedores que lo soportan (Flux vía fal/Replicate). - `callPredictTolerant()` absorbe este tipo de cambios: ante un 400 que diga *no longer supported* / *Unknown name*, identifica el parámetro citado, lo elimina y reintenta. Si Google retira otro parámetro opcional mañana, el lote no se muere. - Si el modelo configurado empieza por `gemini-`, se usa la vía multimodal `:generateContent` con `responseModalities:['IMAGE']`. - Vídeo (Veo): `POST /{modelo}:predictLongRunning` → operación de larga duración. Se sondea `GET /v1beta/{op.name}` cada 5 s hasta 15 minutos. La URI del vídeo se descarga añadiendo `?key=`. - TTS: `:generateContent` con `responseModalities:['AUDIO']`. Devuelve **PCM crudo**; `pcmToWav()` le pone la cabecera RIFF con el sample rate que anuncia el `contentType`.

Modelos permitidos (normativa del proyecto, validados en `pick()`): `gemini-3-flash-preview`, `gemini-3-pro-preview`, `gemini-flash-latest` (*por defecto*), `gemini-flash-lite-latest`, `imagen-4.0-generate-001`, `imagen-4.0-ultra-generate-001`. Si se configura otro, se avisa por consola y se cae al de fábrica. Veo y Gemini TTS quedan fuera de esa lista: son **campo libre** en Ajustes porque su nomenclatura cambia con frecuencia. Si Google renombra el modelo, se corrige ahí sin tocar código.

ElevenLabs — `POST /v1/text-to-speech/{voiceId}?output_format=mp3_44100_128`, cabecera `xi-api-key`, cuerpo `{text, model_id, voice_settings}`. `GET /v1/voices` puebla el desplegable. `POST /v1/music` y `POST /v1/sound-generation` para música y efectos.

OpenAI — `POST /v1/images/generations` (`gpt-image-1`, devuelve `b64_json`), `POST /v1/audio/speech`, `POST /v1/chat/completions`.

Claude — `POST /v1/messages` con `x-api-key` y `anthropic-version: 2023-06-01`.

Replicate — `POST /v1/models/{owner}/{name}/predictions` (o `/v1/predictions` con `version` si el modelo lleva `:`), sondeo de `GET /v1/predictions/{id}`. La salida puede ser string, array u objeto: `run()` extrae la primera URL en cualquiera de los tres casos.

fal.ai — `POST https://fal.run/{modelId}` con `Authorization: Key <k>`, síncrono. `collectUrls()` recorre la respuesta recursivamente porque el esquema varía entre modelos.

4.3 Robustez de red (http.js)

`AbortController` con timeout por llamada (3 min texto/imagen, 10–15 min vídeo). Reintentos con backoff exponencial (2s, 4s, 8s... tope 30 s) **solo** ante `429` y `5xx`: un `400` es culpa del prompt y reintentar es tirar dinero. Los errores incluyen proveedor, código y los primeros 600 caracteres del cuerpo.

5. El motor de montaje (ffmpeg.js)

5.1 Principio de sincronía

```
duración_plano = max(minShotDuration, voiceDuration + padStart + padEnd)
```

Si el plano no tiene narración, se usa la duración sugerida por el doc 04. **Vídeo y audio se generan por separado pero con la misma duración exacta por plano**, así la concatenación no puede desincronizarse: el error acumulado medido en un episodio real de 7:29 es de **46 milisegundos**.

5.2 Ken Burns

Se genera con `zoompan` alimentado por `-loop 1 -framerate FPS -t D`, con `d=1` para que cada fotograma de entrada produzca uno de salida y la variable `on` avance de verdad (con `d=N` el zoom se atasca y da el típico tirón).

La imagen se preescala a `zoom × 1.12` (no al doble): es lo justo para que el recorte nunca se quede sin píxeles, y baja el coste del filtro a la mitad.

El movimiento se deduce del texto de cámara del guion:

Guion dice	Movimiento
dolly-in / push-in / acercamiento	<code>zoomin</code>
pull-back / retroceso	<code>zoomout</code>
paneo de izquierda a derecha	<code>panright</code>
paneo de derecha a izquierda	<code>panleft</code>
tilt ascendente / hacia el cielo	<code>tiltup</code>
órbita / parallax / cámara fija	<code>zoomin</code> lento
(sin indicación)	alterna cíclicamente entre los seis

Las expresiones de `zoompan` se construyen **sin comas** (`'1+0.16*on/449'` en vez de `min(zoom+0.001,1.16)`) para no tener que escapar el filtergraph. Nada de `min()` / `max()`.

5.3 Cadena completa

Por plano:

```

vídeo → [clip: -stream_loop -1, scale/crop, fps] ó [imagen: zoompan]
        → fade in/out → yuv420p → seg_NNN.mp4          (sin audio)
audio → [voz: adelay(padStart), apad, -t D] ó [anullsrc -t D]
        → aud_NNN.wav                                (48 kHz estéreo)

```

Concatenación:

```

concat demuxer + -c copy          (vídeo: todos los segmentos son idénticos)
concat demuxer + pcm_s16le        (audio)

```

Mezcla:

```

[música] volume, atrim, afade in/out
[voz]    asplit
sidechaincompress(música ← voz)   ducking automático
amix + alimiter

```

Mux final:

```

con rótulos: -vf ass=titles.ass[,subtitles=subs.srt] -c:v libx264
sin rótulos: -c:v copy
audio: aac 192k · +faststart

```

Doble codificación evitada. Si se van a quemar rótulos, el mux recodifica todo el vídeo de todas formas; en ese caso los segmentos se generan con `-preset veryfast -crf (CRF-5)` porque son un paso intermedio. Si no se queman, el mux copia y los segmentos se generan ya con la calidad final. Esta optimización redujo el montaje de un episodio de 7:29 de **445 s a 334 s** sin tocar la calidad de salida.

Escapado de rutas en Windows. El filtro `ass= / subtitles=` es un infierno de escapado con rutas absolutas que llevan `Z:\` y espacios. Se resuelve ejecutando `ffmpeg` con `cwd` en el directorio de trabajo del episodio y pasando **solo nombres de fichero relativos**.

Degradación controlada. Si `libass` falla (fuente inexistente, por ejemplo), `muxFinal()` lo registra y reintenta **sin** los filtros de subtítulos en vez de perder todo el render. El `.ass` y el `.srt` se exportan siempre por separado.

5.4 Rótulos y subtítulos

Los rótulos se generan como **ASS** con estilo propio (serif dorada `&H005BB4D9` = RGB `D9B45B`, alineación 1 = abajo-izquierda, `\fad(400,400)`), no con `drawtext`: permite acentos, varias líneas y desvanecidos sin escapar nada.

Los subtítulos de narración se generan repartiendo la duración real del audio de cada plano entre sus frases, proporcionalmente a la longitud en caracteres, con corte por puntuación (`(?<=[.!?…:;])\s+`), máximo 84 caracteres y plegado a dos líneas equilibradas.

Los capítulos de YouTube salen de los títulos de `###` del guion, deduplicados y con el timecode real del montaje.

6. Cola de trabajos (jobs.js)

Cola en memoria con `MAX_PARALLEL` (2 por defecto, tope 6). Las tareas rápidas —imágenes, voz— entran con `priority: true` y adelantan a los montajes.

Cada tarea recibe un `ctx` con `log()`, `progress(0-100, msg)` e `isCancelled()`. Los cambios se publican en un `EventEmitter` que el endpoint `GET /api/events` retransmite como **SSE**; el navegador se suscribe con `EventSource`

y refresca el episodio automáticamente cuando termina un trabajo suyo.

Los lotes (`/batch`) **no abortan al primer fallo**: capturan el error de cada plano, lo registran en el log del trabajo y siguen con el siguiente. Un prompt rechazado no tira por tierra los otros 7 planos.

7. API HTTP

Todas las respuestas: `{ok: true, data}` o `{ok: false, error}` con HTTP 400.

Método	Ruta	Qué hace
GET	/api/health	Node, ffmpeg, claves presentes
GET-POST	/api/settings	Leer / guardar ajustes
POST	/api/keys	Escribir una clave en .env
POST	/api/project/import	Parsear el kit (no destructivo)
GET	/api/project	Resumen con progreso por episodio
GET	/api/episode/:n	Episodio completo + línea de tiempo
POST	/api/episode/:n/shot/:id	Editar prompts, selección, locked
POST	/api/episode/:n/shot/:id/delete-asset	Borrar una variante
POST	/api/episode/:n/shot/:id/images	→ job
POST	/api/episode/:n/shot/:id/video	→ job
POST	/api/episode/:n/shot/:id/voice	→ job
POST	/api/episode/:n/batch	{what:'images'\ 'voice'\ 'video', onlyMissing} → job
POST	/api/episode/:n/music	→ job
POST	/api/episode/:n/render	{burnTitles, burnSubtitles} → job
POST	/api/episode/:n/podcast	→ job
POST	/api/ai/improve-image-prompt	Claude reescribe el prompt
POST	/api/ai/improve-video-prompt	Claude escribe el prompt de animación
POST	/api/ai/youtube-meta	Título, descripción y etiquetas (JSON estricto)
GET	/api/voices	Catálogo de voces de ElevenLabs
GET	/api/jobs · POST /api/jobs/:id/cancel	Cola
GET	/api/events	SSE de progreso
PUT	/api/upload?ep&kind&shotId&name	Subida de binario crudo
GET	/api/output	Listado de entregas

Sobre la subida: se usa `express.raw` y el fichero va como cuerpo binario. Evita la dependencia `multer` y funciona igual desde el `<input type=file>` del navegador.

8. Seguridad y coste

- El servidor escucha en `127.0.0.1`: no es accesible desde la red local.
- Las claves viven en `.env`, incluido en `.gitignore` junto con `data/` y `output/`. La API nunca devuelve el valor de una clave, solo si está presente.
- **No hay ninguna acción automática recurrente.** Nada se genera sin que el usuario pulse un botón; no hay cron, ni scans, ni reprocesado de históricos. Cada llamada a una API de pago es consecuencia directa de un clic.
- Los lotes están acotados a los planos del episodio abierto (máximo 9), nunca a los 72 de la temporada.

9. Pruebas

```
npm run doctor      # ffmpeg, claves, parseo del guion, recuento por episodio
node test-render.js 1
```

`test-render.js` fabrica imágenes y locuciones sintéticas con `ffmpeg` (proporcionales al número real de palabras de cada plano) y monta el episodio completo **sin llamar a ninguna API de pago**. Al terminar compara la duración prevista con la real y falla si el desfase supera 0,5 s. Es la prueba de regresión del motor de montaje.

Resultado de referencia (EP01, 8 planos, 1013 palabras): 449,04 s previstos · 449,00 s reales · desfase −0,046 s.

10. Extensiones previstas

Quiero...	Dónde tocar
Añadir un proveedor	<code>providers/nuevo.js</code> + registro en <code>providers/index.js</code>
Otro guion / otra serie	Ajustes → <i>Carpeta del guion</i> . El parser es genérico.
Cambiar el look de los rótulos	<code>buildTitlesAss()</code> en <code>ffmpeg.js</code>
Nuevos movimientos de cámara	<code>cameraMove()</code> y <code>kenBurnsFilter()</code>
Transiciones con <code>xfade</code>	Sustituir el concat demuxer por un <code>filter_complex</code> encadenado
Vertical / Shorts	Ajustes → Montaje: 1080×1920, y <code>aspect</code> 9:16 en Estilo

11. El generador de guiones (`generator.js`)

El principio de diseño: el generador escribe exactamente el formato de marcadores que el parser sabe leer. Por eso todo lo de aguas abajo —planos, imágenes, voz, montaje— funciona sin una línea de cambio, tanto si el guion lo escribió la IA como si lo escribió una persona.

11.1 Las tres fases

1. CONDENSAR solo si la fuente supera ~22.000 palabras
map: resumen denso por trozos de 55k caracteres
reduce: se concatenan los resúmenes
2. ESCALETA una llamada -> JSON con título, logline, biblia de estilo, títulos SEO y N episodios con sus bloques
3. GUIONES una llamada por episodio -> markdown con marcadores

Un libro no se manda entero por dos razones: no cabe en la ventana y, aunque cupiera, salen guiones peores (el modelo se pierde). Trocear y resumir da mejor resultado y cuesta menos.

11.2 Normalización defensiva

Los modelos se desvían del formato: escriben `**[Voz]:**`, `[IMAGEN]`, `[Rotulo]`, o envuelven todo en un bloque de código. `normalizeScript()` repara los desvíos conocidos con expresiones regulares antes de escribir el fichero, y garantiza que la cabecera `# EPISODIO n –` existe y lleva guion largo. Sin esto, un episodio de cada tres se perdería.

11.3 Derivados

El guion de podcast **no se genera aparte**: se deriva del de vídeo (misma narración, sin `[IMAGEN IA]`, y los `[RÓTULO]` pasan a `[DECIR FECHA]` porque en audio las fechas se dicen). Los documentos 03 y 04 se extraen del propio guion. Una sola llamada por episodio produce los cinco ficheros.

12. Extracción de texto (`extract.js`)

Sin dependencias: solo `zlib`.

Formato	Cómo
TXT · MD · RTF	Lectura directa (el RTF se limpia con regex)
HTML · XHTML	<code>stripMarkup()</code>
DOCX	Es un ZIP → <code>word/document.xml</code> → <code>stripMarkup()</code>
EPUB	Es un ZIP → <code>META-INF/container.xml</code> → OPF → spine (orden real de capítulos)
PDF	No se parsea . Se manda al modelo como documento adjunto

El lector ZIP (`readZip`) recorre el directorio central, soporta los métodos 0 (almacenado) y 8 (deflate) —los únicos que usan DOCX y EPUB— y **normaliza** `\ a /` en los nombres de entrada, porque algunas herramientas de Windows (`Compress-Archive` , entre otras) escriben separadores no estándar.

Sobre el PDF: un extractor casero de PDF es frágil (streams comprimidos, codificaciones de fuente, texto en columnas). Gemini y Claude lo leen nativamente y con mucha mejor calidad, así que el PDF viaja como `inline_data` / `document` en la propia llamada. OpenAI no lo admite por esta vía y devuelve un mensaje que lo explica.

13. Clonado de voz (`voices.js`)

1. El navegador graba con `MediaRecorder` (webm/opus), con `echoCancellation`, `noiseSuppression` y `autoGainControl` **desactivados**: esos filtros mejoran una videollamada pero estropean una muestra para clonar.
2. El servidor normaliza cada muestra con ffmpeg: `highpass=f=70` (quita retumbe), `afftdn` (reducción de ruido) y `dynaudnorm` (nivel constante), a WAV mono 44,1 kHz.
3. `POST /v1/voices/add` de ElevenLabs con **multipart construido a mano** (`buildMultipart`), para no arrastrar una dependencia por una sola llamada.
4. La voz creada se guarda en `data/voz/muestras.json` y se selecciona automáticamente como narrador.

Los cinco párrafos de lectura cubren registros distintos a propósito —enunciado largo, interrogación, enumeración de cifras, tensión dramática y cierre cálido—: la variedad prosódica de la muestra pesa más en la calidad del clon que la duración total.

14. Multi-proyecto

```
data/
  settings.json
  current.txt           slug del proyecto activo
  voz/                  muestras de voz (compartidas entre proyectos)
  fuentes/              ficheros de origen subidos
  projects/<slug>/
    guion/              los .md generados
    project.json        estado
    assets/epNN/        images · videos · voice · music · build
  output/<slug>/        entregas
```

El proyecto activo se resuelve en `projectRoot()`. El `data/project.json` de las primeras versiones se sigue leyendo como «proyecto inicial» para no romper instalaciones existentes.

15. Empaquetado a .exe

`npm run build` (`build.js`) produce `dist/GeneradorContenido/` con el ejecutable, `bin/` (ffmpeg + ffprobe), `docs/`, `.env.example` y un `LEEME.txt`.

Se usa **@yao-pkg/pkg** (el fork mantenido de `pkg`), destino `node22-win-x64`.

Tres detalles que hay que respetar:

1. **Rutas.** Con `process.pkg` activo, `__dirname` apunta al sistema de ficheros virtual del ejecutable. `config.js` distingue `ROOT` (carpeta del `.exe`, donde viven `.env`, `data/` y `output/`) de `BUNDLE_ROOT` (dentro del ejecutable, donde viven `public/` y `docs/`).
2. **Estáticos.** `express.static` no sirve de forma fiable desde el FS virtual de `pkg`. `serveBundled()` lee con `fs.readFileSync`, que sí está parcheado, y bloquea el salto de directorio. `/assets` y `/output` siguen con `express.static` porque son ficheros reales y el vídeo necesita peticiones por rango.
3. **ffmpeg.** No hay `node_modules` en el ejecutable. `locateBinary()` busca por orden: variable de entorno → `bin/` junto al `.exe` → `node_modules` → `PATH`.

`pkg` se invoca con `process.execPath` sobre `lib-es5/bin.js`, **no** con el shim `.cmd`: con `shell:true` una ruta con espacios se parte y el comando falla.

16. Cambios de API absorbidos

Los proveedores mueven el suelo con frecuencia. El programa está preparado:

Qué pasó	Cómo se absorbe
Imagen 4 retiró <code>negativePrompt</code>	<code>callPredictTolerant()</code> detecta el 400, quita el parámetro citado y reintenta
<code>imagen-4.0-*</code> dejó de servirse a cuentas nuevas	Migración automática de ajustes (<code>DEAD_MODELS</code>) a <code>gemini-3-pro-image</code>
429 por falta de facturación	<code>isHardQuota()</code> no reintenta y <code>explain()</code> da la instrucción concreta
Freepik reorganiza endpoints	Las rutas son campos de texto en Ajustes
Google renombra Veo / TTS	Los modelos son campos de texto en Ajustes

17. Lectura de páginas web (`web-read.js`)

Convierte una URL en Markdown limpio **y en un catálogo de fotos reales**. Sin dependencias: extractor propio, al estilo del modo lectura de un navegador.

17.1 Texto

`bloquePrincipal()` prueba primero las etiquetas semánticas (`<article>`, `<main>`, `role=main`). Si no las hay, puntúa cada `<div>` / `<section>` por densidad de texto en párrafos y penaliza los que se llaman `menu`, `sidebar`, `footer`, `cookie ...` `esProsa()` descarta después las líneas que no son prosa (celdas de fichas técnicas, listas de navegación).

`datosEstructurados()` lee el JSON-LD: si la tienda publica el producto ahí, es la mejor fuente —precio, marca, disponibilidad, valoraciones— sin adivinar nada.

17.2 Fotos

`extraerImagenes(html, baseUrl, contenidoHtml)` devuelve candidatas ordenadas por `peso`:

Origen	Peso
<code>og:image</code>	100
JSON-LD (<code>image</code> del producto/artículo)	95
<code>twitter:image</code>	90
<code></code> dentro del contenido	50
<code>background-image</code> en el estilo	30

Filtros: `IMG_BASURA` (sprite, icon, logo, avatar, badge, pixel, tracking, visa, gravatar, favicon...) sobre la URL y sobre el `alt`; descarte por `width / height` declarados (<300×200); resolución de carga diferida vía `data-src`, `data-srcset`, `srcset`, `data-lazy-src`, `data-original`.

`descargarImagenes()`:

- **Pide la versión grande.** `versionGrande()` reescribe `/250px-` → `/1024px-` (patrón Wikimedia) y `nombre-800x600.jpg` → `nombre.jpg` (patrón WordPress). Una miniatura de 250 px no aguanta el zoom del Ken Burns en 1080p.
- **No hace ráfagas.** 350 ms entre peticiones y reintento con `Retry-After` en 429/503. Sin esto, Wikimedia devolvía 429 a partir de la quinta imagen y se perdían 19 de 22.
- Descarta lo que no es `image/*`, los SVG (casi siempre iconos), lo que pesa menos de 15 KB y lo que pasa de 25 MB.

17.3 Visión: que la IA mire las fotos

`describirImagenes()` pasa cada foto descargada por la capacidad `vision` y pide un JSON `{vista, tipo, sirve}`. Tres en paralelo.

Por qué existe: el `alt` de una foto suele estar vacío. En la página de la penicilina de Wikipedia lo estaba en las **22 candidatas**. Sin visión, el guionista recibiría un catálogo de líneas en blanco y elegiría al azar.

Con visión, el catálogo que se le enseña al modelo dice «*placa de Petri con colonia de moho verde y halo transparente*», y las reglas del guion le piden explícitamente que **la narración de cada plano hable de lo que hay en esa foto**.

`sirve: false` descarta lo que la vista reconoce como logo, banner o anuncio: basura que el filtro por nombre de fichero no pillaba.

Tres decisiones defensivas, todas por fallos reales:

1. **maxTokens: 1500, no 400.** Los modelos que razonan gastan parte del presupuesto pensando; con 400 la respuesta llegaba cortada a mitad de frase y `JSON.parse` reventaba en las tres imágenes.
2. **leerRespuesta() rescata lo que haya:** quita el vallado ````json`, recorta entre la primera `{` y la última `}`, y si el JSON viene truncado saca el campo `"vista"` con una regex. Si el modelo contesta en prosa, esa prosa ya describe la foto y se usa tal cual.
3. **Un fallo de visión nunca tira la imagen.** Se queda con el `alt` y sigue. Sin proveedor de visión configurado (`providers.disponible('vision')`), la función devuelve las fotos intactas y no se llama a nadie.

17.4 Del catálogo al plano

1. `routes.js` (`/projects/create` , rama `url`) junta las candidatas de todas las páginas, deduplica por URL y ordena por `peso` .
 2. Dentro del **trabajo** (no en el handler HTTP: tarda) descarga hasta 14 y las describe.
 3. `brief.imagenesWeb = [{alt, vista, tipo, url}]. scriptRules()` cambia el marcador a `[IMAGEN WEB]` e imprime el catálogo numerado con las descripciones.
 4. Tras `importFromSource()` , `colocarFotosWeb()` traduce cada `webImageIndex` a un fichero real, lo guarda con `project.saveAsset(ep, 'images', ...)` y rellena `selectedImage` , `imagePrompt` (lo que se ve) y `webImageOrigen` .
 5. Un plano cuyo número esté fuera de rango coge la siguiente foto sin usar, antes que quedarse en blanco.
- Prueba real** (Wikipedia · penicilina · formato «vídeo corto vertical», 21 s): 22 candidatas → 14 descargadas → 8 planos, de los que **7 usan foto real**. El octavo es `[IMAGEN IA]` porque el guion menciona el melón podrido del que salió la cepa buena y en la página no hay ninguna foto de eso: exactamente el respaldo previsto.

18. Duración real vs duración pedida

Un vídeo de 21 s salía de **30,8 s**. El cálculo de palabras era una regla de tres con las palabras por minuto y **no descontaba el silencio de cada corte**: ocho planos con los márgenes de documental (0,35 + 0,65 s) son 8 segundos de silencio, casi la mitad del vídeo.

Dos cambios:

- `plataformas.MARGENES` ata el aire al ritmo del formato. En `corto` : `padStart 0,10` , `padEnd 0,22` , `minShotDuration 1,2` , `transitionDuration 0,18` . Los cortes van secos, que además es el lenguaje del formato. Se aplican solos al crear el proyecto, junto con la orientación.
- `palabrasPara()` descuenta `planosPara(segundos, ritmo) × (padStart + padEnd)` antes de convertir a palabras, con un suelo del 50 % del tiempo para que nunca se quede sin narración.

Pedido	Ritmo	Planos	Palabras	Duración estimada
15 s	corto	5	39	15,0 s
21 s	corto	7	55	21,1 s
30 s	corto	10	78	29,9 s
600 s	normal	24	1455	600,0 s
900 s	documental	26	2039	899,9 s

Medido en el ejecutable: el mismo TikTok de 21 s pasó de **30,8 s a 24,3 s** (1080×1920). Sigue 3 s largo porque el modelo escribió 62 palabras en vez de 55 y la voz de Gemini va algo por debajo de 175 ppm, pero entra en el rango bueno de las cuatro redes cortas.

19. Que el video dure lo que se ha pedido

Dos fallos distintos, uno encima del otro. El sintoma que se veia era el mismo: pedir un TikTok y recibir un video de cinco minutos.

19.1 La interfaz mandaba siempre 600 segundos

`aplicarFormato()` rellenaba `cEpisodes` y un campo oculto `cMinutes` con `salida.minutes`. **Ningun formato define minutes** —todos definen segundos—, así que `s.minutes || 10` valia siempre 10. Y `pintarDuraciones()` estaba definida pero **no se llamaba desde ningun sitio**: el `<select id="cDuracion">` se quedaba vacio, `Number('')` daba 0, y el envio caia al respaldo `cMinutes * 60 = 600 segundos, elijas lo que elijas`.

Se corrigio asi:

- `aplicarFormato()` llama a `pintarDuraciones(formato)`, que repinta el desplegable con la franja del formato y deja marcada su duracion propia.
- El campo oculto `cMinutes` **se ha eliminado**. La duracion sale solo del desplegable.
- `FRANJAS` limita lo que se ofrece: `corto` 15-300 s, `normal` 90-1800 s, `documental` 300-3600 s. No es cosmetica: si en "video corto vertical" se puede elegir 20 minutos, alguien lo elige.
- Por encima de `TOPE_REDES` (60 s) en un formato corto, la nota se pone en color de aviso y explica el rango que rinde.

Leccion: esto se me escapo porque probe la API (`POST /api/projects/create` con `segundos: 21`) y no el formulario. La API funcionaba perfectamente. El fallo estaba entre el formulario y la API. Las pruebas de la interfaz (`test-ui-duracion.js` , Playwright sobre Chrome del sistema, interceptando `**/api/projects/create` para leer el cuerpo sin gastar API) recorren los 9 formatos y comprueban duracion por defecto, opciones ofrecidas, el aviso y el cuerpo que se envia.

19.2 El modelo se pasa del presupuesto de palabras

Arreglado lo anterior, un TikTok de 21 s salia de **32,7 s**: el modelo habia escrito **112 palabras** cuando el presupuesto era 55. Decirle "ni una mas" en el prompt no sirve — los modelos cuentan mal.

`ajustarDuracion()` lo comprueba de forma determinista despues de `normalizeScript()`:

1. `palabrasHabladas()` cuenta **solo las lineas [VOZ]**, que es lo unico que se locuta. Los prompts de imagen en ingles y los rotulos no suenan y no deben contar.
2. Si se pasa del margen (8 % en formatos cortos, 15 % en el resto), se hace **una** llamada de recorte diciendo el numero exacto de palabras que sobran, con la orden de mantener todos los marcadores y quitar palabras, no ideas.
3. Se reintenta como mucho una vez mas.
4. **Salvaguarda:** si el recorte se deja planos por el camino o vuelve vacio, se descarta y se queda el guion original. Antes un video largo que uno roto.

Medido de punta a punta por la interfaz del ejecutable, formato `redes` a 21 s:

	Palabras habladas	Video final
Antes (fallo de la interfaz)	—	~5 min
Solo con la interfaz corregida	112	32,7 s
Con el recorte	54	18,0 s (1080×1920)

20. Por que los textos salian en telegrama

Sintoma real, subtítulos de un video de producto de 21 s:

Mi hijo se cayo de la litera. Las camas altas dan miedo.
 A ras de suelo, sin ayuda. Anade patas y crece con el.
 No parece mueble de niño. Quedan pocas unidades disponibles.

Siete frases sueltas, sin ninguna relacion entre ellas. **No era culpa del modelo: era del reparto.**

20.1 La causa: 7,9 palabras por plano

`planosPara(21, 'corto')` daba un plano cada 3 segundos = **7 planos**, y el presupuesto era de 55 palabras. Eso son **7,9 palabras por plano**. Como cada plano lleva su propio `[VOZ]`, el modelo no tiene sitio para una frase completa: solo le cabe un fragmento. Escribia exactamente lo unico que se podia escribir.

Cambiado a un plano cada 4,5 s (`Math.max(3, Math.min(9, segundos / 4.5))`):

Duracion	Planos antes	Planos ahora	Palabras/plano antes	Ahora
15 s	5	3	7,8	13,7
21 s	7	5	7,9	11,4
30 s	10	7	7,8	11,6
45 s	12	9	10,0	13,7

El corte sigue siendo rapido —cada 4,5 s en pantalla es ritmo de red social—, pero ahora cabe una frase de verdad.

20.2 Las reglas empujaban al mismo sitio

Decian literalmente "*Frases de una linea. Sujeto, verbo y al grano*", que es una receta para el telegrama. Sustituido por reglas que exigen lo contrario:

- El guion es **un texto seguido**, no una lista: cada frase encadena con la anterior y la frase de un plano puede terminar en el siguiente.
- Se incluye el **ejemplo real del fallo** como contraejemplo ("ASI NO") junto a la version buena ("ASI SI"). Los modelos corrigen mucho mejor con un contraejemplo concreto que con una instruccion abstracta.
- Prohibido el relleno ("en un mundo donde...", "todos sabemos que...").
- Concreto antes que generico: "*aguanta 200 kilos*" dice algo, "*gran calidad*" no.

20.3 Y algo mas grave: se inventaba datos

En ese guion aparecia **"Quedan pocas unidades disponibles"** y **"Reservala antes de que se agote"**. Esos datos **no estaban en el material de origen**: se los invento el modelo. No es un problema de estilo, es una afirmacion comercial falsa, y la urgencia inventada es publicidad enganosa.

Regla nueva, en todos los formatos:

NO TE INVENTES NADA. Ni precios, ni descuentos, ni unidades que quedan, ni plazos, ni garantias, ni valoraciones, ni premios, ni cifras. Si el dato no esta en el material de origen, NO se dice.

20.4 El recorte tambien empujaba al telegrama

`ajustarDuracion()` (§19.2) pedia *"quita palabras, frases mas secas"*, que sobre un texto ya corto lo acaba de romper. Ahora pide **quitar ideas enteras, no palabras sueltas** —mejor tres cosas bien contadas que siete a medias— y comprueba explicitamente que el resultado siga sonando a alguien contando algo.

21. Formato del episodio en el momento de montar

La orientacion estaba solo en *Ajustes > Montaje*, como valor global. Para sacar un episodio en vertical habia que ir a Ajustes, cambiarlo, volver, montar y deshacerlo.

Ahora hay un desplegable **Formato del episodio** junto al boton *MONTAR EPISODIO*: 16:9 horizontal, 9:16 vertical, 1:1 cuadrado, los dos y los tres. En blanco usa el de Ajustes, que es el comportamiento de siempre.

- `studio.renderEpisode()` ya aceptaba `options.orientation` ; solo faltaba la interfaz.
- La ruta **valida** el valor contra una lista blanca. `ff.formatosDe()` cae a horizontal ante cualquier cosa desconocida, y montar veinte minutos en la orientacion equivocada sin avisar es peor que fallar.
- La orientacion va en la etiqueta del trabajo (*Montaje EP1 (vertical)*): con varios montajes del mismo episodio en la cola, si no se distinguen no sirve de nada.

22. La orientacion era GLOBAL: un proyecto reconfiguraba a los demas

El sintoma: en una serie documental, las imagenes eran 16:9, los clips 16:9, y el MP4 salia vertical con la imagen encajada en el centro entre barras borrosas.

22.1 La prueba

Dimensiones reales de los MP4 entregados, ordenados por hora:

Cuando	Que se monto	Salio
30/07 08:29–10:04	penicilina, cielo azul, flamencos, camas	1080×1920 vertical
30/07 11:37–12:04	documental EP02, EP03, EP04	1080×1080 cuadrado
30/07 12:43	carburo	1920×1080 horizontal
31/07 11:41	documental EP01	1920×1080 horizontal

La orientacion seguia **la hora, no el proyecto**. Tres episodios del mismo documental salieron cuadrados porque justo antes se habia creado un proyecto de podcast (cuya `salida.orientation` es `cuadrado`).

22.2 La causa

En `POST /projects/create` habia esto:

```
config.saveSettings({ render: { orientation: salidaFormato.orientation, ... }, style: { aspect: ... }
});
```

`settings.json` es **uno solo para todo el programa**. Cada proyecto que se creaba reescribia la orientacion, el aspecto de las imagenes y los margenes de ritmo para TODOS los demas. Crear un TikTok dejaba en vertical el documental que tenias a medias, y quien mandaba siempre era el ultimo proyecto creado.

Peor todavia: como las imagenes ya se habian generado antes en 16:9, al montar sobre un lienzo 9:16 entraba `decideFit()` y las metia con fondo borroso. Todo "funcionaba"; el resultado era inservible.

22.3 La correccion

La configuracion de salida es **del proyecto**, no del programa:

- `project.json` guarda `salida: {orientation, aspect, ritmo, burnSubtitles, formato}`, escrita al crear el proyecto y conservada al reimportar.
- `studio.getSettings()` (interno) devuelve los ajustes globales **con la salida del proyecto superpuesta**, solo en los campos que son suyos. Todo `studio.js` ya lo usaba por ese nombre, asi que el cambio es de una sola funcion.
- `POST /projects/create` **ya no escribe ajustes globales**.
- Prioridad al montar: `options.orientation` (selector del episodio) → `project.salida.orientation` → `settings.render.orientation`.
- Los proyectos anteriores no tienen `salida`: caen a la global y se comportan como siempre.
- `project.summary()` expone `salida`, y el selector del episodio muestra el formato real de la serie — (el de esta serie: `horizontal 16:9`) —. Que no se viera en ninguna parte es lo que permitio que tres episodios salieran cuadrados sin que nadie lo notara hasta abrir el MP4.

Comprobado con `test-salida-proyecto.js`, que trabaja sobre una carpeta de datos aparte y **no gasta ninguna llamada de IA**: crea un documental, crea un TikTok despues, vuelve al documental y verifica que conserva orientacion, aspecto y ritmo; que los ajustes globales quedan intactos; y que un proyecto sin `salida` sigue usando la global.

23. Encaje: cuando recortar y cuando rellenar

Despues de arreglar §22 seguia saliendo mal, y por otro motivo: **los clips de video estaban generados en vertical** aunque el lienzo ya era 16:9. En el montaje entraban con fondo borroso a los lados, ocupando un tercio de la pantalla.

23.1 Por que los clips salieron verticales

`generateShotVideo()` pide el clip con `settings.style.aspect`. Y **la mayoría de los formatos no declara `aspect`**: solo lo hacen los de redes (`redes`, `producto`). Un documental solo declara `orientation: 'horizontal'`. Así que la proporcion caia a la global, que estaba contaminada en 9:16 por §22.

Corregido en `studio.getSettings()`: si la salida del proyecto no trae `aspect`, **se deriva de la orientacion**.

salida.orientation	Se piden imagenes y clips en
<code>horizontal</code>	16:9
<code>vertical</code>	9:16
<code>cuadrado</code>	1:1
<code>both / todos</code>	16:9 (el principal; los otros se sacan encajando)

Comprobado con la proporcion global puesta a 9:16 **a proposito**: los cuatro tipos de proyecto piden la suya y ninguno hace caso de la global.

23.2 `auto` era simetrico y no debia serlo

`decideFit()` devolvia `blur` para cualquier diferencia mayor del 12%. Eso trata igual dos casos que no se parecen en nada:

- **Vertical dentro de horizontal** → rellenando, el contenido queda en una franja central que ocupa **un tercio** de la pantalla y los dos lados son fondo borroso. Se ve pobre y se desperdicia el video. **Ahora recorta.**
- **Horizontal dentro de vertical** → recortando, de un 16:9 solo sobrevive el **32% del ancho**, lo que parte por la mitad casi cualquier encuadre. Ademas en redes el fondo desenfocado es el lenguaje normal del formato. **Se sigue rellenando.**

La perdida es la misma en ambos sentidos (32% de lo que habia); lo que cambia es que en un caso el recorte da un resultado utilizable y en el otro no. Por eso la regla es asimetrica a proposito.

Tambien se recorta cuando fuente y destino tienen **la misma orientacion** y la diferencia es menor del 35% (4:3 en 16:9, por ejemplo): recortar quita poco y llena la pantalla, mientras que el fondo borroso ahi canta mucho.

Fuente	Lienzo	Antes	Ahora
9:16	16:9	blur	cover
16:9	9:16	blur	blur
4:3	16:9	blur	cover
16:9	16:9	cover	cover
9:16	1:1	blur	blur

23.3 Bonus: la pantalla de Ajustes prometia algo que el codigo no cumplia

El texto decia "*Las capturas de pantalla nunca se recortan*", pero la comprobacion de `kind === 'screenshot'` estaba **despues** de la preferencia manual:

```
if (preferido && preferido !== 'auto') { return preferido; } // <- salia por aqui
if (kind === 'screenshot') { ... } // <- no llegaba nunca
```

Poner "Llenar recortando" en Ajustes recortaba tambien las capturas, que es exactamente lo que no puede pasar en un video de ayuda: se pierde parte de la interfaz que se esta explicando. La proteccion va ahora la primera.

24. Sistema multiidioma (`idiomas.js`)

24.1 Por que el idioma de Ajustes no hacia nada

Habia **dos cosas distintas que no se hablaban**:

- En *Crear* un campo de TEXTO LIBRE (`español de Espana`) que iba a `brief.language` y era lo unico que miraba el generador.
- En *Ajustes* unos desplegables con CODIGOS (`es` , `en`) en `settings.languages` , que solo usaban el doblaje y los subtítulos.

Cambiabas el idioma en Ajustes, el generador seguia leyendo el campo de texto y todo salia en castellano. No era un fallo de guardado: es que apuntaban a sitios distintos.

Ahora manda **un codigo**: `brief.idiomaPrincipal` . De ahí se saca el texto que ve el modelo (`translate.idioma(code).prompt`). El campo de texto libre ha desaparecido.

24.2 Los idiomas son del PROYECTO

Misma leccion que con la orientacion (\$22): en `settings.json` habrian sido globales y el ultimo proyecto creado le habria cambiado el idioma a los demas. Viven en `project.json` :

```

"idiomas": {
  "principal": "es",           // en el que ESCRIBE la IA
  "adicionales": ["en", "fr"], // a los que se traduce
  "conVoz": ["es"],           // en cuales se locuta (cuesta)
  "conVideo": ["es"],          // en cuales se genera video (lo mas caro)
  "conSubtitulos": ["es","en","fr"],
  "viendo": "es"              // lo que se esta editando ahora
}

```

Prioridad, que es lo que pidio el usuario y coincide con el coste real:

	Se hace en	Motivo
TEXTO	todos los idiomas, al crear	traducir es barato
IMAGEN	se comparte	la imagen no habla
VOZ	solo los marcados	cuesta por caracter
VIDEO	solo los marcados	lo mas caro con diferencia

24.3 Donde vive cada traduccion

El idioma **principal NO se duplica**: sigue en los campos de siempre (`shot.narration` , `shot.titles` , `shot.voiceFile`). Los demas van en `shot.i18n[codigo]` . Asi un proyecto de un solo idioma queda byte a byte como antes y no hay migracion que hacer.

`versionPlano(shot, lang, principal)` resuelve cual toca. Es la unica funcion que sabe de esa asimetria; el resto del programa pide "la version en X" y no se entera.

24.4 La duracion cambia con el idioma

`buildTimeline(episode, lang)` calcula la linea de tiempo **por idioma**, porque la duracion de cada plano la marca su locucion y una frase no dura lo mismo en espanol que en aleman. El mismo episodio puede durar 10 s en un idioma y 15 en otro; comprobado en la prueba en frio.

El MP4 lleva el codigo en el nombre salvo el principal: `EP01_TITULO.mp4` , `EP01_TITULO.en.mp4` .

25. Lo demas que se ha anadido

25.1 Imagen de partida ("usar como base")

`generateShotImages({ base, instruccion })` . `base` puede ser `'seleccionada'` (la imagen que ya tiene el plano) o la ruta de cualquier material del proyecto, incluida una que subas en ese momento.

Con imagen de partida el prompt **cambia de naturaleza**: ya no describe una imagen nueva, es la instruccion de que hacerle a la que hay. Sin instruccion se asume "el mismo plano con el estilo de la serie".

En el proveedor de Gemini la imagen va **primero** en las partes del mensaje y el texto detras: asi el modelo la trata como material y el texto como orden. Los modelos de la familia `imagen-*` son de texto a imagen y no la mirarian, asi que con `base` se fuerza un modelo `gemini-*` .

25.2 "Mejorar con IA" en todo

`studio.mejorarConIA({ campo, usarActual })` cubre narracion, rotulos, prompt de imagen, prompt de video, escena y titulo. Un solo camino con una receta por campo (`MEJORAS`), en vez de una funcion por campo que se irian separando con el tiempo.

`usarActual: false` es "propon otra cosa": se le da el contexto del plano pero **no** el texto actual, para que no se limite a parafrasear. La propuesta se ensena antes de aplicarla — la IA acierta a menudo, pero perder un texto bueno por un boton es muy molesto.

25.3 Ampliar y rehacer episodios

- `ampliarEpisodio()` escribe planos NUEVOS al final. Se le pasa lo que ya hay para que no repita y enlace. Se reutiliza el parser del kit, asi que no hay dos gramaticas.
- `regenerarEpisodio()` rehace imagenes, voz y montaje de una pasada. **Respetar los planos fijados** (`locked`) por defecto: es donde el usuario ya decidio algo a mano.

25.4 Marca de agua / logotipo

Se compone en el mux final con `filter_complex` (no `-vf`: hay que mezclar dos flujos de video). El logo se escala a un **porcentaje del ancho**, asi que el mismo PNG vale para 16:9 y para 9:16.

La marca del EPISODIO manda sobre la del PROYECTO, para poder poner un patrocinador distinto en un episodio suelto. Si el fichero ya no existe se avisa y se monta sin el; si el filtro falla, el rescate parte de cero (el `-map [v]` del filtro roto no sirve).

25.5 Reiniciar el programa

En la version de escritorio **el servidor corre dentro del proceso de Electron**, asi que `process.exit()` cerraria el programa y no lo volveria a abrir. El boton usa `app.relaunch()` por IPC. En modo navegador si sale del proceso y sondea `/api/health` hasta que vuelve a responder, en vez de esperar un tiempo fijo.

Hay tambien un **Recargar proyectos** que solo vacia el cache en memoria: para cuando se copian proyectos a mano en la carpeta de datos.

25.6 Podcast

- Con "**generar siempre el podcast**" marcado, la version sonora se **reescribe con IA** en vez de derivarse quitando las lineas de imagen. Un guion de video sin sus imagenes deja frases que no se entienden ("aquí vemos...") y pierde los datos que iban en rotulos.
- `POST /episode/:n/podcast/desde-texto` convierte un documento subido —o texto pegado— en guion de podcast: frases mas cortas, siglas deletreadas, cifras escritas como se dicen, respetando el contenido.

25.7 Formatos

De 9 a 29. Los nuevos: entrevista, noticia, resumen de libro, caso de estudio, receta, viaje, cuento infantil, corporativo, anuncio, testimonio, preguntas frecuentes, comparativa, historia personal, lista/top N, explicativo, meditacion, audiolibro, mini documental, trailer y resumen periodico. Cada uno con su `salida` para que la interfaz se autoconfigure.

26. Acotaciones de dialogo (`acotaciones.js`)

El sintoma. En los guiones de dialogo aparece `(Rie)`, `[suspira]`, `*baja la voz*`. Eso llegaba tal cual a ElevenLabs y el locutor lo LEIA: se oia "parentesis rie".

Por que. El modelo por defecto (`eleven_multilingual_v2`) no entiende etiquetas de audio; para el son texto normal. Las etiquetas (`[laughs]`, `[sighs]`) solo las actuan los modelos `*_v3`.

La solucion. Un modulo que separa lo que se DICE de lo que se HACE, y dos caminos segun el modelo:

Modelo	Camino
<code>*_v3</code>	La acotacion se traduce a su etiqueta de audio y se antepone a la linea. El modelo la actua.
resto	Se QUITA del texto y se convierte en un delta sobre <code>voice_settings</code> de esa linea.

Detalles que importan:

- **Tres formas** aceptadas: `(...)`, `[...]`, `*...*`. Cada IA que escribe guiones usa la suya.
- **Limite de 60 caracteres.** Un parentesis largo casi nunca es una acotacion: es texto del guion que hay que locutar (`Roma cayo en 476 (segun los manuales)`). Cortar por longitud + catalogo cerrado de senales evita comerse frases enteras.
- Los deltas de tono se **acumulan y se recortan** al rango valido. `speed` se limita a 0.7–1.2 porque fuera de ahi ElevenLabs devuelve error de validacion.
- Una linea que es SOLO una acotacion se queda sin texto. Mandar cadena vacia a la API es un 400: se devuelve un MP3 de silencio del largo del gesto, generado con ffmpeg en memoria.
- `acotaciones.palabras()` sustituye al conteo crudo en `palabrasHabladas()`: una acotacion no se locuta, luego no ocupa tiempo, luego no puede contar para el presupuesto de palabras.

27. Cache de voz global (`cachevoz.js`)

El sintoma. Locutar el episodio para el VIDEO y despues exportar el PODCAST cobraba dos veces exactamente el mismo texto con las mismas voces.

Por que. Cada camino guardaba su audio en su carpeta y con su nombre. Ninguno veia el del otro. Habia ocho sitios distintos que locutaban y cada uno cacheaba a su manera (o no cacheaba).

La solucion. Una cache en el PROYECTO, enganchada en `providers.call('voice', ...)`, que es el unico punto por el que pasan todas las locuciones. Poner el control en cada ruta era justo lo que fallaba: siempre quedaba una sin enganchar.

La firma es `sha1(texto + voz + proveedor + modelo + idioma + ajustes redondeados)`.

- Los **ajustes se redondean a 2 decimales** antes de firmar. Sin eso, `0.5` y `0.5000000001` (que salen de sumar los deltas de las acotaciones) darian firmas distintas y no acertaria nunca.
- `previo` / `siguiente` **NO entran en la firma** a proposito: cambian la entonacion un pelo, pero incluirlos significaria que corregir una coma en el bloque 3 invalida tambien el 2 y el 4.
- Limpieza por **fecha de ultimo uso**, no de creacion: la sintonia que suena en veinte episodios tiene que sobrevivir aunque se grabara el primer dia.
- Si algo falla (sin proyecto abierto, sin permisos), se locuta y ya esta. La cache es un extra, nunca un requisito.

Se ve y se vacia desde la barra de Episodios (`GET /api/voz/cache` , `POST /api/voz/cache/vaciar`).

28. Traducir: el JSON que llegaba cortado

El sintoma. La IA no devolvio un JSON valido al traducir. Empieza por: { .

Por que. `translateMany` mandaba **12 bloques FIJOS** por llamada, contando bloques y no contenido, con `maxTokens: 8000` fijo. En un documental un bloque son 30 palabras y no pasa nada; en un podcast conversacional un bloque son 500, asi que doce bloques eran ~6000 palabras de entrada y otras tantas de salida. El modelo cortaba la respuesta a mitad, el JSON llegaba **empezado pero sin cerrar** —de ahí el "Empieza por: {"— y `JSON.parse` lo rechazaba entero, perdiendo tambien las traducciones que SI estaban completas.

Tres arreglos, en este orden:

1. `repartirEnTandas()` mide en **palabras** (tope 900) ademas de en bloques (tope 12). Manda el limite que se alcance antes. Un bloque que por si solo pase del tope va en su propia tanda: partirlo descuadraria la correspondencia uno a uno.
2. `maxTokens` se calcula sobre la entrada (`palabras * 4.5 + 800` , techo 32000) en vez de ser fijo. Traducir ocupa mas que el original en casi todos los idiomas.
3. `leerJson()` tolerante: quita vallas ``json,` extrae el primer `,` y si sigue sin parsear ****rescata por regex los pares** "clave": "valor" que si estan cerrados**. Quien llama ve que faltan claves.

Y una red debajo: si faltan bloques, `traducirTanda()` **parte la tanda en dos y reintenta cada mitad** (hasta 4 niveles). Repitiendo, cualquier bloque acaba yendo solo en su llamada, que es el caso en el que es imposible que no quepa. Un bloque que aun asi falle se queda en el idioma original y se avisa en el log, en vez de tumbar el episodio entero.

29. Varios kits en un proyecto: anadir episodios en vez de reemplazar

El sintoma. "Importar kit de otra IA" **reemplazaba** el proyecto en vez de anadir el episodio 2.

Dos causas distintas:

1. La ruta `/projects/importar-kit` **siempre creaba un proyecto nuevo** (`slug + Date.now()`).
2. Aunque no lo hiciera, `importProject` solo sabia leer UN kit: `localizarKit()` busca el mejor `.md` de todo el arbol y se queda con uno. Copiar un segundo kit en `guion/` habria hecho que uno de los dos guiones desapareciera.

La solucion.

- Cada kit se copia en **su propia subcarpeta** de `guion/` con una marca `_kit.json` que guarda su numero de episodio, el nombre y la carpeta de origen.
- `importProject()` mira si hay marcas. **Si no las hay, se comporta exactamente como antes** (`leerUnKit`): un kit normal viene repartido en subcarpetas (`01_YOUTUBE/` , `02_PODCAST/`) y trocearlo por carpetas romperia todos los que ya funcionan. Solo se parte con marcas explicitas.
- `unirKits()` junta los episodios resolviendo choques de numeracion: si el numero esta cogido, se pasa al primero libre. Perder un episodio por un choque es mucho peor que verlo reenumerado.
- `renumerar()` cambia tambien el prefijo de los ids de plano (`ep02_s07`), que es lo que ata cada imagen y cada locucion a su plano.

- Un proyecto antiguo con los `.md` sueltos en `guion/` se **recoge** a su propia carpeta la primera vez que se le anade un segundo episodio (`recogerKitSuelto`), respetando su numeracion original.

`numeroDesdeNombre()` deduce el numero del nombre de carpeta: `EP02`, `T2E05`, `capitulo-3`, `PARTE 7`, `episodio_12`. El separador se escribe a mano (`(?:^[^A-Za-z0-9])`) y **no con** `\b`, porque el guion bajo cuenta como letra: en `LA_OTRA_LINEA_EP02` no hay ningun limite de palabra delante de `EP` y `\bEP` no encuentra nada — que es justo el nombre de carpeta de los kits reales. `EP2026` se descarta por el tope de 300: es un ano, no un episodio.

Se deduce del **nombre de la carpeta y no de la cabecera del guion** a proposito: practicamente todos los kits generados por IA empiezan por `# EPISODIO 1` aunque sean el septimo.

30. Borrar un episodio (`DELETE /api/episode/:n`)

Se lleva por delante tres cosas, y las tres hacen falta:

1. El episodio de `project.json`.
2. **La carpeta del kit que lo genero.** Sin esto la siguiente reimportacion lo resucita y parece que el borrado no ha funcionado.
3. Sus assets — **solo si se pide** (`?assets=1`). Por defecto se conservan: casi siempre se borra un episodio para volver a importarlo corregido, y rehacer treinta imagenes cuesta dinero.

Si el episodio venia de un kit suelto (proyecto de un solo episodio, sin marca), no se toca el guion —borrarlo dejaria el proyecto sin nada— y se devuelve un `aviso` para decirlo.

31. "Se queda parado" en el montaje: no lo estaba, es que no informaba

El sintoma. El montaje se planta en `Plano 7/9 – 1.7 (image)` y no se mueve.

La causa. `run()` buscaba el avance con un regex sobre `time=` del stderr. Esa linea SOLO la escribe ffmpeg con el nivel de log por defecto, y aqui se usa `-loglevel warning`: **el progreso no llegaba nunca**. Dentro de un plano no habia ni una senal. En un documental daba igual —cada plano son 8 segundos— pero un podcast conversacional montado en video tiene planos de tres y cuatro minutos, y ahi la pantalla se queda quieta un cuarto de hora.

Los arreglos:

1. `-progress pipe:1 -nostats` en todas las llamadas. Escribe `clave=valor` en la salida estandar independientemente del nivel de log; se lee `out_time_us` y se traduce a porcentaje y a tiempo restante estimado. El buffer puede partir una linea en dos, de ahi el acumulador `resto`.
2. La etiqueta del plano lleva ahora su DURACION (`Plano 7/9 – 1.7 (image, 3:24)`), para que se vea de antemano que ese plano va a tardar.
3. **Vigilante de bloqueo.** El corte NO va por tiempo total —un plano puede tardar minutos de verdad— sino por tiempo SIN AVANZAR: si ffmpeg no emite nada en 4 minutos, se le mata y se dice que casi siempre es una imagen corrupta en ese plano. Antes no habia ningun timeout: se podia quedar colgado para siempre.

El coste del Ken Burns: cuidado con medir a la resolucion equivocada.

Primera medicion, a **1280x720**: 60 s tardaban 19,8 s con Ken Burns y 19,6 s sin el. Conclusion —equivocada— "no cuesta nada, manda el codificador". A esa resolucion el filtro cabe de sobra en el tiempo que tarda x264 y la diferencia se diluye.

Repetida a **1920x1080**, que es como se monta de verdad:

Plano (1080p, medium, crf 19)	Con Ken Burns	Sin Ken Burns
60 s	30,9 s	20,6 s

Un **50 % mas**. La leccion: medir a la resolucion real, no a una comoda.

El criterio para quitarlo no es la duracion, es la VELOCIDAD del movimiento. Lo que se percibe no es el zoom total sino cuanto cambia el encuadre por segundo. Un 16 % en 8 s son 2 %/s y se ve; el mismo 16 % en 3 min y medio son 0,076 %/s y no lo ve nadie. Por debajo de `MIN_ZOOM_POR_SEGUNDO = 0.15` el zoompan no aporta nada y cuesta ese 50 %, asi que se quita y se dice en el log con el numero exacto.

Con el zoom por defecto el umbral cae en planos de mas de ~107 s. **En un documental no salta nunca.** Salta en un podcast conversacional montado en video, donde cada bloque son tres o cuatro minutos sobre una sola imagen — que es justo el caso donde el montaje se hacia eterno. Medido: un plano de 210 s pasa de ~108 s a 72,5 s.

Otra idea probada y descartada. Codificar los segmentos intermedios (los que se recodifican despues al quemar subtítulos) con `ultrafast` en vez de `veryfast`. Medido a 1080p: 60 s pasan de 31,5 s a 29,7 s (un 6 %), pero el temporal pasa de 3,9 MB a 21,6 MB. En un episodio de media hora son 650 MB de temporales para ahorrar un minuto. No compensa.

32. Traducir: las tandas van en paralelo

Al pasar el reparto de lotes de "12 bloques" a "900 palabras" (ver el apartado 28) salen mas tandas que antes, y se lanzaban una detras de otra. Cada tanda es una llamada independiente —no comparten contexto— asi que encadenarlas solo servia para esperar.

Ahora van **de tres en tres**. Tres y no mas: por encima de eso los proveedores empiezan a devolver 429 y se tarda mas por los reintentos. Una tanda que falle del todo no tumba a las demas: esos bloques se quedan en el idioma original y se avisa en el log.

GENERADOR DE CONTENIDO — Documentación técnica

Versión: 1.0.0 · **Autor:** Ecom Experts · **Licencia:** AFL-3.0 **Stack:** Node.js ≥18.17 · Express 4 · ffmpeg-static · front vanilla (sin framework, sin build)

1. Arquitectura

Aplicación local cliente-servidor. El servidor Node hace de orquestador entre las APIs de IA y `ffmpeg`; el navegador es solo la interfaz. No hay base de datos: **el estado vive en un único JSON** y los binarios en disco.

```

generador-de-contenido/
├── ARRANCAR.bat           Lanzador Windows (instala deps la primera vez)
├── package.json
├── .env                   Claves de API (creado desde .env.example, NO se comparte)
├── test-render.js         Prueba del motor de montaje sin gastar API
├──
├── server/
│   ├── index.js           Arranque Express, estáticos, apertura del navegador
│   ├── config.js          Parser .env propio + ajustes persistentes + claves
│   ├── generator.js        Fuente -> escaleta -> guiones (escribe el kit .md)
│   ├── extract.js         PDF/DOCX/EPUB/TXT -> texto (lector ZIP propio)
│   ├── voces.js           Grabación, limpieza de audio y clonado de voz
│   ├── parser.js          Markdown del guion -> episodios / planos
│   ├── project.js         Estado en data/project.json + gestión de assets
│   ├── studio.js          Operaciones de producción y orquestación del render
│   ├── ffmpeg.js          Motor de montaje (Ken Burns, concat, mezcla, ASS/SRT)
│   ├── jobs.js            Cola en memoria con progreso por SSE
│   ├── routes.js          API HTTP
│   ├── doctor.js          Diagnóstico del entorno (npm run doctor)
│   └── providers/
│       ├── index.js       Registro y enrutado capacidad -> proveedor
│       ├── http.js        fetch con reintentos, backoff y timeouts
│       ├── gemini.js      Imagen · Veo 3.1 · Gemini texto · Gemini TTS
│       ├── openai.js      gpt-image-1 · TTS · GPT
│       ├── elevenlabs.js  Voz · Music · Sound effects
│       ├── claude.js      Messages API (texto)
│       ├── replicate.js   Flux · Kling · MusicGen (predicciones con sondeo)
│       ├── fal.js         Flux (síncrono) · MiniMax H3 · LTX · Kling · Veo (por cola)
│       ├── freepik.js     MAGNIFIC: upscale, relight, style · Mystic · Kling
│       └── grok.js         Grok Imagine de xAI: imagen y vídeo (vídeo asíncrono)
├── public/               index.html · app.js · styles.css
├── docs/                 Esta documentación
├── data/                 [generado] project.json, settings.json, assets/
└── output/               [generado] MP4, MP3, SRT, capítulos
  
```

2. El parser (`parser.js`)

2.1 Gramática reconocida

```
# EPISODIO <n> — <TÍTULO>
**~15 min · Libro I · <subtítulo>**
### <Sección> — <timecode>
[IMAGEN IA] <prompt en inglés>
[VOZ] <narración>
[RÓTULO] <texto en pantalla>
[MÚSICA] <indicación>
[ESCENA] <dramatización>
```

También reconoce las variantes del guion de podcast: `[SONIDO]` , `[ESCENA SONORA]` , `[DECIR FECHA]` , y las etiquetas sin tilde (`[ROTULO]` , `[MUSICA]`).

Un beat puede ocupar varias líneas: las continuaciones se concatenan hasta una línea en blanco o un nuevo marcador.

Hay dos marcadores más que **también abren plano**, y que producen planos que no se generan con IA:

Marcador	shot.kind	Origen de la imagen
<code>[IMAGEN IA] <prompt></code>	<code>ai-image</code>	La genera el proveedor de imagen
<code>[CAPTURA] Pantalla: ... \ Se debe ver: ... \ Cómo llegar: ...</code>	<code>screenshot</code>	La hace el usuario; se valida por visión
<code>[IMAGEN WEB] <n> — <por qué></code>	<code>web-image</code>	Foto real descargada de la página de origen

El número va FUERA del corchete. `TAG_RE` es `/^\[([A-ZÁÉÍÓÚÑ ]+)\]\s*(.*)$/` : dentro de los corchetes solo caben letras y espacios, así que `[IMAGEN WEB 3]` **no casaría**. Como el modelo lo escribe así la mitad de las veces, `normalizeScript()` lo reescribe a `[IMAGEN WEB] 3 ...` antes de parsear (también acepta `[FOTO WEB n]` y `[WEB n]`). El número acaba en `shot.webImageIndex` .

2.2 Construcción de planos

Regla: cada marcador de imagen abre un plano. Todo lo que sigue —voz, rótulos, escena, música— pertenece a ese plano hasta el siguiente.

Los beats que aparecen **antes** del primer `[IMAGEN IA]` del episodio (típicamente un `[MÚSICA]` de arranque) se acumulan en un buffer y se vuelcan en el primer plano real. Sin esto se creaban planos fantasma sin imagen y toda la numeración se desplazaba.

Cada plano recibe una clave `<episodio>.<ordinal>` — `1.1` , `1.2` ... — que es exactamente la nomenclatura de los documentos `03_PROMPTS_IMAGENES.md` (`**1.3**`) y `04_PROMPTS_VIDEOS.md` (`**Plano 1.3**`). Así se enriquece cada plano con:

- el prompt canónico de imagen (doc 03),
- el prompt de vídeo, la indicación de cámara y la duración sugerida (doc 04),

- y los títulos SEO de la biblia (doc 05 §8).

Del kit actual salen **10 episodios, 72 planos, 125 bloques de voz y 49 rótulos**, cifras que coinciden con el recuento directo de marcadores en los `.md`.

3. Modelo de datos (`data/project.json`)

```
{
  "name": "Mi serie – Serie documental",
  "sourceDir": "...",
  "episodes": [{
    "n": 1, "title": "...", "subtitle": "...", "seoTitle": "...",
    "wordCount": 1013,
    "music": { "prompt": "...", "file": "assets/ep01/music/...mp3" },
    "youtube": { "title": "...", "description": "...", "tags": [] },
    "podcast": { "narration": [...], "outputFile": null },
    "outputFile": null,
    "shots": [{
      "id": "ep01_s01", "n": 1, "key": "1.1",
      "section": "Gancho – 0:00",
      "imagePrompt": "...", "videoPrompt": "...",
      "camera": "dolly-in muy lento (push-in), 6 s",
      "suggestedDuration": 6,
      "narration": [...], // líneas [VOZ]
      "titles": [...], // líneas [RÓTULO]
      "scene": "...", // [ESCENA]
      "music": "...", // [MÚSICA]

      "images": [{ "file": "assets/ep01/images/...png", "provider": "gemini", "model": "imagen-4.0-generate-001", "prompt": "...", "createdAt": "..." }],
      "selectedImage": "assets/ep01/images/...png",
      "videos": [{ "file": "...", "duration": 6.0, ... }],
      "selectedVideo": null,
      "voiceFile": "assets/ep01/voice/...mp3",
      "voiceDuration": 43.7,
      "locked": false
    }]
  }]
}
```

Rutas: en el JSON siempre son relativas a `data/`, con `/` como separador. `project.abs()` y `project.rel()` hacen la conversión. Eso permite servirlos tal cual desde `/assets/<ruta>` y mover la carpeta del proyecto sin romper nada.

Reimportación no destructiva. `importFromSource()` reparsea el markdown y reinyecta los assets del proyecto anterior emparejando por `shot.id`. Si un plano está `locked`, además conserva sus prompts editados a mano.

4. Capa de proveedores

4.1 Contrato

Cada proveedor exporta `{ id, name, caps, ...métodos }`. El registro enruta capacidad → método:

Capacidad	Método	Firma de entrada	Devuelve
image	generateImage	{prompt, negative, aspect, count}	[{buffer, ext, model, provider}]
video	generateVideo	{prompt, imageBuffer\ imageDataUri, durationSec, aspect, onProgress}	{buffer, ext, ...}
voice	tts	{text, voice, model}	{buffer, ext, ...}
music	music	{prompt, durationSec}	{buffer, ext, ...}
text	text	{system, prompt, maxTokens}	string

`providers.call(cap, args, forzado?)` resuelve el proveedor desde `settings.providers[cap]`, comprueba que hay clave y lanza un error legible si no. Añadir un proveedor nuevo = un fichero en `providers/` + una línea en el registro.

xai y grok : dos identificadores, un solo servicio

Son la misma empresa y la **misma clave** (`XAI_API_KEY`), pero endpoints que no se parecen en nada:

id	Endpoint	Fichero	Capacidades
xai	<code>/v1/chat/completions</code>	<code>providers/openai-compat.js</code>	texto, visión
grok	<code>/v1/images/generations</code> y <code>/v1/videos/generations</code>	<code>providers/grok.js</code>	imagen, vídeo

La parte visual **no habla el dialecto de** `/chat/completions`, así que no cabía en el motor compatible. El vídeo además es **asíncrono**: la petición devuelve un `request_id` y hay que sondear `GET /v1/videos/{id}` hasta que `status` pasa a `done`.

`grok.js` copia de `gemini.js` el patrón de **reintento tolerante**: xAI retoca los parámetros de Image cada pocas semanas y responde 400 nombrando el que ya no vale; en vez de dejar el lote muerto se quita ese campo opcional y se repite. `model` y `prompt` no se tocan nunca.

Su vídeo trae audio propio (diálogo, ambiente y música) y el montaje lo **descarta a propósito**: la duración del plano la fija la locución y el clip se estira con `-stream_loop`, así que su audio sonaría entrecortado. El clip suelto de `data/` sí lo conserva. Se anota en `entry.tieneAudio` y se avisa por pantalla.

labios.js — sincronía labial en los prompts de vídeo

Los modelos de vídeo, cuando ven una cara, la hacen hablar. Siempre. En un documental la voz es de un narrador que **no está en pantalla**, así que lo que salía era un personaje moviendo la boca sin decir nada y desincronizado con una voz que no es suya.

Cada plano ya sabe quién habla en cada bloque (`shot.speakers` va en paralelo a `shot.narration`), así que se puede distinguir el caso y decírselo al modelo:

speakers[i]	Qué se le pide al modelo
<code>null</code> o <code>NARRADOR</code>	Voz en off: todas las bocas cerradas y quietas . La cara vive por los ojos, el parpadeo y la respiración
un personaje	Labios, mandíbula y lengua sincronizados con su frase literal , en el idioma del audio

El bloque va **al final del prompt**, detrás del estilo: es la instrucción que más se incumple y lo último que se lee pesa más. El texto hablado va literal y en su idioma, que es justo lo que determina la forma de la boca; las instrucciones van en inglés, que es lo que entienden los modelos.

Se controla en **Ajustes** → **Estilo** → **Movimiento de labios** (`style.lipSync`): `auto` (solo si hay caras), `siempre` o `nunca` . Si algo falla al componerlo, se anima **sin** la nota en vez de tumbar la generación del clip.

El bloque **depende del modelo**, porque no todos entienden lo mismo:

Perfil	Proveedores	Qué recibe	Por qué
<code>dialogo</code>	<code>gemini</code> (Veo), <code>grok</code>	La frase literal entrecomillada , el idioma, y la petición de sincronía fonema a fonema	Generan audio: pronuncian de verdad, así que pueden cuadrar los labios con ese texto
<code>mudo</code>	<code>fal</code> , <code>replicate</code> , <code>freepik</code> (familia Kling)	Solo quién habla y quién calla , en 300 caracteres	No pronuncian nada: pedirles precisión de fonemas es gastar un prompt que además truncan

Un proveedor que no esté en la tabla cae en `dialogo` , que es el bloque completo: con uno nuevo, más vale pasarse que quedarse corto.

El bloque va ANTES del sufijo de estilo. El primer intento lo ponía al final, razonando que lo último que se lee pesa más; el problema es que los Kling **truncan el prompt por detrás**, así que lo primero que se perdía era justo la sincronía. El estilo es decorativo y la sincronía no: si algo se ha de caer, que se caiga el estilo.

Pruebas: `node test-labios.js` (62 comprobaciones, sin gastar API).

4.2 Detalles por proveedor

Gemini (`https://generativelanguage.googleapis.com/v1beta/models` , cabecera `x-goog-api-key`) - Imagen: `POST /{modelo}:predict` CON `{instances:[{prompt}], parameters:{sampleCount, aspectRatio}}` . Devuelve `predictions[].bytesBase64Encoded` . - **Imagen 4 ya no acepta `negativePrompt`** (responde `400 Setting negativePrompt is no longer supported`). El prompt negativo de Ajustes solo se envía a los proveedores que lo soportan (Flux vía `fal/Replicate`) . - `callPredictTolerant()` absorbe este tipo de cambios: ante un 400 que diga *no longer supported / Unknown name*, identifica el parámetro citado, lo elimina y reintenta. Si Google retira otro parámetro opcional mañana, el lote no se muere. - Si el modelo configurado empieza por `gemini-` , se usa la vía multimodal `:generateContent` con `responseModalities:['IMAGE']` . - Vídeo (Veo): `POST /{modelo}:predictLongRunning` → operación de larga duración. Se sondea `GET /v1beta/{op.name}` cada 5 s hasta 15 minutos. La URI del vídeo se descarga añadiendo `?key=` . - TTS: `:generateContent` con `responseModalities:['AUDIO']` . Devuelve **PCM crudo**; `pcmToWav()` le pone la cabecera RIFF con el sample rate que anuncia el `mimeType` .

Modelos permitidos (normativa del proyecto, validados en `pick()`): `gemini-3-flash-preview` , `gemini-3-pro-preview` , `gemini-flash-latest` (*por defecto*) , `gemini-flash-lite-latest` , `imagen-4.0-generate-001` , `imagen-4.0-ultra-generate-001` . Si se configura otro, se avisa por consola y se cae al de fábrica. Veo y Gemini TTS quedan fuera

de esa lista: son **campo libre** en Ajustes porque su nomenclatura cambia con frecuencia. Si Google renombra el modelo, se corrige ahí sin tocar código.

ElevenLabs — `POST /v1/text-to-speech/{voiceId}?output_format=mp3_44100_128`, cabecera `xi-api-key`, cuerpo `{text, model_id, voice_settings}`. `GET /v1/voices` puebla el desplegable. `POST /v1/music` y `POST /v1/sound-generation` para música y efectos.

OpenAI — `POST /v1/images/generations` (`gpt-image-1`, devuelve `b64_json`), `POST /v1/audio/speech`, `POST /v1/chat/completions`.

Claude — `POST /v1/messages` con `x-api-key` y `anthropic-version: 2023-06-01`.

Replicate — `POST /v1/models/{owner}/{name}/predictions` (o `/v1/predictions` con `version` si el modelo lleva `:`), sondeo de `GET /v1/predictions/{id}`. La salida puede ser string, array u objeto: `run()` extrae la primera URL en cualquiera de los tres casos.

fal.ai — `POST https://fal.run/{modelId}` con `Authorization: Key <k>`, síncrono. `collectUrls()` recorre la respuesta recursivamente porque el esquema varía entre modelos.

4.3 Robustez de red (`http.js`)

`AbortController` con timeout por llamada (3 min texto/imagen, 10–15 min vídeo). Reintentos con backoff exponencial (2s, 4s, 8s... tope 30 s) **solo** ante `429` y `5xx`: un `400` es culpa del prompt y reintentar es tirar dinero. Los errores incluyen proveedor, código y los primeros 600 caracteres del cuerpo.

5. El motor de montaje (`ffmpeg.js`)

5.1 Principio de sincronía

```
duración_plano = max(minShotDuration, voiceDuration + padStart + padEnd)
```

Si el plano no tiene narración, se usa la duración sugerida por el doc 04. **Vídeo y audio se generan por separado pero con la misma duración exacta por plano**, así la concatenación no puede desincronizarse: el error acumulado medido en un episodio real de 7:29 es de **46 milisegundos**.

5.2 Ken Burns

Se genera con `zoompan` alimentado por `-loop 1 -framerate FPS -t D`, con `d=1` para que cada fotograma de entrada produzca uno de salida y la variable `on` avance de verdad (con `d=N` el zoom se atasca y da el típico tirón).

La imagen se preescala a `zoom × 1.12` (no al doble): es lo justo para que el recorte nunca se quede sin píxeles, y baja el coste del filtro a la mitad.

El movimiento se deduce del texto de cámara del guion:

Guion dice	Movimiento
dolly-in / push-in / acercamiento	zoomin
pull-back / retroceso	zoomout
paneo de izquierda a derecha	panright
paneo de derecha a izquierda	panleft
tilt ascendente / hacia el cielo	tiltup
órbita / parallax / cámara fija	zoomin lento
(sin indicación)	alterna cíclicamente entre los seis

Las expresiones de `zoompan` se construyen **sin comas** (`'1+0.16*on/449'` en vez de `min(zoom+0.001,1.16)`) para no tener que escapar el filtergraph. Nada de `min()` / `max()` .

5.3 Cadena completa

Por plano:

```

vídeo → [clip: -stream_loop -1, scale/crop, fps]  ó [imagen: zoompan]
        → fade in/out → yuv420p → seg_NNN.mp4      (sin audio)
audio → [voz: adelay(padStart), apad, -t D]  ó [anullsrc -t D]
        → aud_NNN.wav                          (48 kHz estéreo)

```

Concatenación:

```

concat demuxer + -c copy          (vídeo: todos los segmentos son idénticos)
concat demuxer + pcm_s16le       (audio)

```

Mezcla:

```

[música] volume, atrim, afade in/out
[voz]    asplit
sidechaincompress(música ← voz)   ducking automático
amix + alimiter

```

Mux final:

```

con rótulos: -vf ass=titles.ass[,subtitles=subs.srt] -c:v libx264
sin rótulos: -c:v copy
audio: aac 192k · +faststart

```

Doble codificación evitada. Si se van a quemar rótulos, el mux recodifica todo el vídeo de todas formas; en ese caso los segmentos se generan con `-preset veryfast -crf (CRF-5)` porque son un paso intermedio. Si no se queman, el mux copia y los segmentos se generan ya con la calidad final. Esta optimización redujo el montaje de un episodio de 7:29 de **445 s a 334 s** sin tocar la calidad de salida.

Escapado de rutas en Windows. El filtro `ass= / subtitles=` es un infierno de escapado con rutas absolutas que llevan `Z:\` y espacios. Se resuelve ejecutando `ffmpeg` con `cwd` en el directorio de trabajo del episodio y pasando **solo nombres de fichero relativos**.

Degradación controlada. Si `libass` falla (fuente inexistente, por ejemplo), `muxFinal()` lo registra y reintenta **sin** los filtros de subtítulos en vez de perder todo el render. El `.ass` y el `.srt` se exportan siempre por separado.

5.4 Rótulos y subtítulos

Los rótulos se generan como **ASS** con estilo propio (serif dorada `&H005BB4D9` = RGB `D9B45B`, alineación 1 = abajo-izquierda, `\fad(400,400)`), no con `drawtext`: permite acentos, varias líneas y desvanecidos sin escapar nada.

Los subtítulos de narración se generan repartiendo la duración real del audio de cada plano entre sus frases, proporcionalmente a la longitud en caracteres, con corte por puntuación (`(?<=[.!?…:;])\s+`), máximo 84 caracteres y plegado a dos líneas equilibradas.

Los capítulos de YouTube salen de los títulos de `###` del guion, deduplicados y con el timecode real del montaje.

6. Cola de trabajos (`jobs.js`)

Cola en memoria con `MAX_PARALLEL` (2 por defecto, tope 6). Las tareas rápidas —imágenes, voz— entran con `priority: true` y adelantan a los montajes.

Cada tarea recibe un `ctx` con `log()`, `progress(0-100, msg)` e `isCancelled()`. Los cambios se publican en un `EventEmitter` que el endpoint `GET /api/events` retransmite como **SSE**; el navegador se suscribe con `EventSource` y refresca el episodio automáticamente cuando termina un trabajo suyo.

Los lotes (`/batch`) **no abortan al primer fallo**: capturan el error de cada plano, lo registran en el log del trabajo y siguen con el siguiente. Un prompt rechazado no tira por tierra los otros 7 planos.

7. API HTTP

Todas las respuestas: `{ok: true, data}` o `{ok: false, error}` con HTTP 400.

Método	Ruta	Qué hace
GET	/api/health	Node, ffmpeg, claves presentes
GET-POST	/api/settings	Leer / guardar ajustes
POST	/api/keys	Escribir una clave en .env
POST	/api/project/import	Parsear el kit (no destructivo)
GET	/api/project	Resumen con progreso por episodio
GET	/api/episode/:n	Episodio completo + línea de tiempo
POST	/api/episode/:n/shot/:id	Editar prompts, selección, locked
POST	/api/episode/:n/shot/:id/delete-asset	Borrar una variante
POST	/api/episode/:n/shot/:id/images	→ job
POST	/api/episode/:n/shot/:id/video	→ job
POST	/api/episode/:n/shot/:id/voice	→ job
POST	/api/episode/:n/batch	{what:'images'\ 'voice'\ 'video', onlyMissing} → job
POST	/api/episode/:n/music	→ job
POST	/api/episode/:n/render	{burnTitles, burnSubtitles} → job
POST	/api/episode/:n/podcast	→ job
POST	/api/ai/improve-image-prompt	Claude reescribe el prompt
POST	/api/ai/improve-video-prompt	Claude escribe el prompt de animación
POST	/api/ai/youtube-meta	Título, descripción y etiquetas (JSON estricto)
GET	/api/voices	Catálogo de voces de ElevenLabs
GET	/api/jobs · POST /api/jobs/:id/cancel	Cola
GET	/api/events	SSE de progreso
PUT	/api/upload?ep&kind&shotId&name	Subida de binario crudo
GET	/api/output	Listado de entregas

Sobre la subida: se usa `express.raw` y el fichero va como cuerpo binario. Evita la dependencia `multer` y funciona igual desde el `<input type=file>` del navegador.

8. Seguridad y coste

- El servidor escucha en `127.0.0.1`: no es accesible desde la red local.
- Las claves viven en `.env`, incluido en `.gitignore` junto con `data/` y `output/`. La API nunca devuelve el valor de una clave, solo si está presente.
- **No hay ninguna acción automática recurrente.** Nada se genera sin que el usuario pulse un botón; no hay cron, ni scans, ni reprocesado de históricos. Cada llamada a una API de pago es consecuencia directa de un clic.
- Los lotes están acotados a los planos del episodio abierto (máximo 9), nunca a los 72 de la temporada.

9. Pruebas

```
npm run doctor      # ffmpeg, claves, parseo del guion, recuento por episodio
node test-render.js 1
```

`test-render.js` fabrica imágenes y locuciones sintéticas con `ffmpeg` (proporcionales al número real de palabras de cada plano) y monta el episodio completo **sin llamar a ninguna API de pago**. Al terminar compara la duración prevista con la real y falla si el desfase supera 0,5 s. Es la prueba de regresión del motor de montaje.

Resultado de referencia (EP01, 8 planos, 1013 palabras): 449,04 s previstos · 449,00 s reales · desfase -0,046 s.

10. Extensiones previstas

Quiero...	Dónde tocar
Añadir un proveedor	<code>providers/nuevo.js</code> + registro en <code>providers/index.js</code>

El encaje "blur" y por qué el montaje iba lento

Una foto **horizontal** en un lienzo **vertical** se monta con fondo borroso (`decideFit` devuelve `blur`). La cadena original desenfoaba con `gblur=sigma=28` **a 1080x1920 y en cada fotograma**: un núcleo de 84 px de radio, 1.800 veces en un plano de 60 s... sobre una imagen fija, así que los 1.800 resultados eran idénticos.

Dos cambios, según de dónde salga el fondo:

Fuente	Qué se hace	Por qué
Imagen	La imagen entra una segunda vez como entrada suelta. Se desenfoca una sola vez y <code>loop=loop=-1:size=1</code> reparte ese fotograma	El fondo ya no acompaña al Ken Burns, pero está borroso hasta ser una mancha: el movimiento no se ve
Vídeo	Se desenfoca por fotograma, pero a 1/4 de lado y con sigma/4	Ahí el fondo sí cambia. Un gaussiano cuesta ~área × sigma: 16 veces menos área

Medido a 1080x1920 sobre 10 s de vídeo, en la misma máquina:

	Segundos
Cadena anterior	28,2
Desenfocado una vez, tamaño completo	18,8
Sin desenfocado ninguno (referencia)	19,5
Cadena actual	14,0

Está **por debajo de la referencia sin desenfocado**: el `gblur` ha dejado de costar.

`loop=-1` produce un flujo **infinito**. Lo cortan el `shortest=1` del `overlay` y el `-t` de la salida. Quitar cualquiera de los dos cuelga ffmpeg para siempre; hay una prueba que lo vigila.

La dirección de cámara se leía mal

`cameraMove` traduce la indicación del guion ("dolly-in", "paneo de izquierda a derecha") al movimiento de Ken Burns. Tenía dos fallos:

- **dolly-out no estaba reconocido.** `dolly-in` sí desde el principio, pero su contrario no aparecía en ninguna lista, así que caía al alternador por índice del final y salía un movimiento cualquiera.
- **El atajo `pan.*derecha` cazaba los paneos al revés.** "paneo de derecha a izquierda" lleva `pan` y lleva `derecha`, así que devolvía `panright`: **el plano se movía al contrario de lo que pedía el guion.**

Se arregló ordenando las comprobaciones —las frases que dicen las dos direcciones van antes que los atajos— y hay 20 formas de escribirlo cubiertas en `test-montaje.js`.

Capturas de la interfaz

`scripts/capturar.js` las hace con el **propio Electron**. Los navegadores en modo headless no valen: la interfaz mantiene abierta una conexión SSE para el progreso de los trabajos, así que nunca hay «red inactiva» y tanto Chrome como Edge se quedan esperando indefinidamente. `webContents.capturePage()` no depende de eso.

```
# consola 1
PORT=4399 NO_OPEN=1 node server/index.js
# consola 2
env -u ELECTRON_RUN_AS_NODE npx electron scripts/capturar.js docs/img/ui
```

Si lo lanzas desde una terminal de VS Code hay que quitar `ELECTRON_RUN_AS_NODE`, que VS Code exporta a 1: con esa variable puesta Electron arranca como un Node pelado, `require('electron')` devuelve una ruta en vez de la API y el script muere diciendo que `app` es `undefined`.

El **Editor** no se captura: sin un proyecto cargado la aplicación se queda en la vista Crear. Esa hay que hacerla a mano con una serie abierta.

Por qué un episodio tardaba ocho horas

Un capítulo de 200 planos llegó a tardar **ocho horas**. Se midió dónde se iba el tiempo y salieron tres cosas, ninguna de ellas obvia:

1 • El codificador estaba escrito a mano. Todo pasaba por `libx264 -preset medium`, que es excelente y también lo más lento de la lista. ffmpeg trae compilados NVENC, Quick Sync y AMF **siempre**, estén o no la tarjeta y el driver, así que no basta con mirar `-encoders`: hay que **probarlos de verdad**. Eso hace `server/codecs.js`, codificando medio segundo de nada con cada uno y mirando el código de salida.

Medido con `scripts/bench-codec.js` sobre 6 s de documental a 1080p, en un portátil Intel m3-8100Y (2 núcleos a 1,1 GHz):

Codificador	Tiempo	VMAF	Veredicto
<code>h264_qsv</code> (GPU Intel)	2,2 s	94,8	muy bueno
<code>h264_mf</code>	3,8 s	93,5	muy bueno
<code>libx264 -veryfast</code>	6,9 s	93,8	muy bueno
<code>libx264 -medium</code> ← <i>lo que se usaba</i>	20,6 s	96,3	indistinguible
<code>libx264 -slow</code>	24,6 s	96,3	indistinguible

VMAF es la métrica de Netflix y la única que se parece a lo que ve una persona: 95+ es indistinguible, 90+ muy bueno, por debajo de 80 se nota. **Quick Sync es 9,4 veces más rápido perdiendo 1,5 puntos que nadie ve.**

2 • Los planos se montaban de uno en uno. Con 200 planos y un `ffmpeg` por plano, la máquina se pasaba horas con un solo núcleo ocupado. Ahora van varios a la vez (`enParalelo` en `studio.js`).

El límite **no** es el número de núcleos: las tarjetas de consumo limitan cuántas codificaciones simultáneas aceptan — NVIDIA suele permitir entre 3 y 8— y pasarse de ahí hace que las de más **mueran a media faena**. Por eso cada codificador declara `maxParalelo` y `codecs.paralelismo()` toma el mínimo entre eso y los núcleos disponibles.

3 • El desenfoque del fondo. Ver el apartado del encaje `blur` más arriba: 28,2 s → 14,0 s.

Los tres se multiplican entre sí.

`server/codecs.js`

Función	Qué hace
<code>detectar(forzar)</code>	Prueba cada codificador de verdad. Cachea en <code>data/codecs.json</code> con una huella de la máquina: solo se repite si cambias de ordenador o instalas un driver
<code>mejorAutomatico()</code>	El primero de la lista de preferencia que funcione: GPU si la hay, <code>libx264</code> si no
<code>elegir(settings, intermedio)</code>	Devuelve <code>-c:v</code> y los argumentos de calidad ya traducidos. La escala no es la misma en cada familia : en <code>libx264</code> el CRF va de 0 a 51 y menos es mejor; en Media Foundation va de 0 a 100 y más es mejor. Ahí se convierte
<code>paralelismo(settings, codec)</code>	Cuántos planos a la vez
<code>resumen()</code>	Lo que ven Ajustes y el doctor

Si el usuario tiene elegido un codificador que ya no funciona —cambió de ordenador, desinstaló un driver— **se cae al automático con un aviso** en vez de reventar el montaje.

Varias imágenes en un mismo plano

`shot.selectedImages` es una lista. Con más de una, se encadenan con `xfade`.

La cuenta importa: con **N** imágenes, **N-1** fundidos de **F** segundos y un total de **D**, cada imagen dura

$$d = (D + (N-1) \cdot F) / N$$

porque cada fundido **solapa** dos imágenes y se come F segundos del total. El fundido *k* empieza en `k · (d-F)`. Si esta cuenta se hace mal el plano no dura lo que debe y **el episodio entero se desincroniza**, que es lo único que este programa no se puede permitir. Hay pruebas con 2, 3, 4 y 5 imágenes en las dos orientaciones.

El fundido se recorta solo si el plano es corto: nunca puede comerse más del 60 % de cada imagen. Con 5 imágenes en 4 segundos, F baja de 0,8 s a 0,48 s.

`selectedImage` se mantiene apuntando a la primera de la lista. No es redundante: es lo que miran la miniatura del episodio, el «usar como base» y todo lo escrito antes de que un plano pudiera llevar varias.

`server/revisión.js` — mirar antes de gastar horas

Montar son horas. Enterarse al final de que catorce planos salieron en negro porque nunca se eligió imagen es tirar toda esa tarde, y el vídeo **no falla**: sale entero y simplemente está mal, así que hay que verlo completo para darse cuenta.

`revisar(episode, lang)` devuelve problemas en tres niveles:

Nivel	Significa	Ejemplos
<code>grave</code>	El episodio saldría roto. Se pide confirmar	Planos sin imagen, imágenes generadas pero sin elegir, texto sin locutar
<code>aviso</code>	Se puede montar, pero probablemente no es lo que se quería	Locuciones que el proyecto cree tener y no encuentra (se regenerarían y eso cuesta dinero)
<code>nota</code>	Información, nada que arreglar	Planos sin narración: en cabeceras y transiciones es lo correcto

La interfaz lo consulta en `GET /api/episode/:n/revisión` antes de encolar. Con `options.forzar` se monta igualmente: hay casos legítimos de montar a medias para ver cómo va.

El previo — `public/previo.js`

Ver el episodio **sin codificar nada**: el navegador enseña la imagen a pantalla completa y reproduce el `.mp3` de la locución que ya está en disco. Cada plano dura lo que dura su audio, que es **el mismo criterio que usa el montaje real**, así que el ritmo que se ve ahí es el que va a salir.

No enseña Ken Burns, ni rótulos quemados, ni música, ni el encaje al lienzo final. Sirve para lo que sirve: ver si la imagen acompaña al texto y si el episodio se sigue. Con varias imágenes en un plano, se turnan con un fundido de CSS.

`server/cachemontaje.js` — no rehacer lo que no ha cambiado

Un segmento es **determinista**: sale de unas imágenes, una duración, un movimiento de cámara, un lienzo y unos ajustes de codificación. Con las mismas entradas sale el mismo fichero. Así que se guarda una **huella** de esas entradas junto al segmento y, si coincide y el fichero sigue ahí, se reutiliza.

Una caché mal invalidada es de lo peor que puede pasar aquí: entrega un episodio con un plano viejo y **nadie se entera**, porque el vídeo sale entero y sin errores. Por eso la huella incluye todo lo que puede cambiar el resultado, y ante la duda se rehace.

Entra en la huella: los ficheros de origen (ruta, **tamaño y fecha**), la duración, la cámara, el tipo de plano, el lienzo, `fps`, `fit`, Ken Burns y su zoom, la transición, el fundido entre imágenes, **el codificador ya resuelto con sus argumentos** —cambiar de libx264 a Quick Sync produce otro fichero— y si es paso intermedio o entrega final.

Y hay un `VERSION` en el módulo: **al tocar el motor de montaje hay que subirlo**, y toda la caché anterior queda invalidada de golpe. Es la red de seguridad para cuando se cambie algo y no se caiga en la cuenta de que afectaba a la huella.

Antes se vaciaba la carpeta de trabajo entera al empezar. Ahora `limpiar()` conserva lo vigente y tira el resto: los intermedios grandes (`video.mp4` , las pistas de audio, el MP4 final) se rehacen siempre y ocupan cientos de megas.

Reanudar sale de aquí sin código extra: el manifiesto se guarda **cada 10 planos**, no al final. Si se corta la luz en el plano 180, la próxima vez los anteriores están anotados y se reutilizan. Guardarlo solo al terminar no habría servido justo en el caso que duele.

`options.rehacer` descarta la caché y empieza limpio.

`server/estimar.js` — cuánto va a tardar

No se puede calcular con una fórmula fija: depende de la CPU, de si hay GPU, de la resolución, de si hay fondos desenfocados y de si se queman rótulos. Así que **se mide**: cada montaje anota cuánto ha tardado por segundo de vídeo, y eso alimenta la estimación del siguiente.

Se guardan las últimas 20 medidas de cada combinación `codificador|resolución|quemado` y se usa la **mediana**, no la media: un montaje que se quedó esperando porque el usuario abrió otra cosa no debe estropear la estimación. La primera vez sale de una tabla base y **se dice que es a ojo**.

Solo se apunta si **no se ha reaprovechado nada**. Un montaje que reutilizó 190 de 200 planos tardó poquísimo, y meter ese dato en el histórico haría que la siguiente estimación prometiera diez minutos para un episodio que se monta de cero.

Ritmo y silencios

`render.ritmo` multiplica los silencios que rodean la voz en todos los planos: `agil` 0,55 · `normal` 1 · `pausado` 1,7 · `solemne` 2,4. Es lo que más cambia la sensación de un documental, y antes había que tocar `padStart` y `padEnd` a mano sabiendo lo que se hacía.

Un plano concreto puede llevar los suyos (`shot.pausaAntes` , `shot.pausaDespues`) y **mandan sobre el ritmo**: un silencio antes de la frase que remata un bloque vale más que cualquier efecto, y eso no se decide en Ajustes para todo el episodio. Topados entre 0 y 10 s para que un valor absurdo no convierta un plano de 8 segundos en uno de tres minutos.

Planos que se hacen eternos

La revisión avisa de los planos de **más de 90 segundos sobre una sola imagen fija**. No es un capricho: con el zoom por defecto el Ken Burns deja de percibirse pasados los ~107 s (ver `MIN_ZOOM_POR_SEGUNDO`), así que lo que queda en pantalla es una foto quieta minuto y medio.

Se mira cuántas imágenes tiene **asignadas**, exista o no el fichero ahora mismo —que falte uno es otro problema y se reporta aparte—. Un clip de vídeo no se marca: ya se mueve solo.

Sonoridad: por qué el episodio sale a -14 LUFS

YouTube, Spotify y compañía **normalizan todo lo que subes** a un objetivo fijo. Si tu episodio sale más bajo, la plataforma lo sube — y con él sube el ruido de fondo y la respiración. Si sale más alto, lo baja y pierde pegada. En los dos casos el resultado es peor que si hubiera salido ya en su sitio.

`cadenaSonoridad()` añade `loudnorm=I=-14:TP=-1.5:LRA=11` al final de la mezcla, **después** de mezclar y limitar: lo que hay que dejar en su sitio es el resultado, no cada pieza. El `TP=-1.5` deja aire para el recodificado a AAC, que puede pasarse de 0 y distorsionar aunque el WAV no lo hiciera.

Se apaga con `render.loudnorm: false` y el objetivo se cambia con `render.loudnormI`.

Ken Burns con entrada y salida suaves

El movimiento era lineal: $1 + k \cdot (on/N)$. Arrancaba de golpe, iba a velocidad constante y paraba en seco. Una cámara de verdad acelera al empezar y frena al acabar.

Ahora el avance pasa por la curva del coseno: $(1 - \cos(\pi \cdot on/N))/2$. Vale 0 en el primer fotograma y 1 en el último, **igual que la lineal**, así que el encuadre inicial y el final no cambian: solo cambia cómo se llega de uno a otro.

Ninguna expresión puede llevar **comas**: el filtergraph las usa de separador y habría que escaparlas por todas partes. Hay una prueba que lo vigila.

El silencio inicial de las locuciones

Los proveedores de voz a veces devuelven medio segundo de nada antes de la primera palabra. Como aquí **la duración del plano sale de la duración del audio**, ese silencio se suma al que ya pone el programa: el plano arranca mudo más rato del previsto y, plano tras plano, el episodio se va haciendo más lento sin que se sepa por qué.

`recortarSilencioInicial()` lo quita **al generar la voz, antes de medir su duración**. Es el único sitio que cuadra: recortarlo en el montaje dejaría la duración ya guardada sin valor y el silencio simplemente se movería al final.

Solo se toca un silencio que **empiece en el cero**: uno a mitad de frase es una pausa del locutor y hay que respetarla. Nunca más de dos segundos, y si el resultado no tiene sentido se deja el original — no recortar es siempre seguro, recortar de más se come el principio de la frase.

El modo borrador, y por qué no bastaba con bajar la resolución de salida

`render.borrador` monta a **un tercio de lado** conservando la proporción — si se cambiara, el borrador enseñaría un encuadre distinto del que va a salir y no serviría de nada.

El primer intento bajó solo la resolución de salida y **apenas ganó tiempo**: 14,4 s contra 13,6 s. El motivo es que el Ken Burns trabajaba a 1920x1080 **sea cual sea el lienzo**, así que el `zoompan` seguía moviendo los mismos píxeles. Bajando también esa resolución de trabajo: **8,3 s contra 3,9 s**.

Por qué el Ken Burns NO se baja de resolución en el montaje bueno

Se midió con `scripts/bench-kb.js`, salida 1080x1920:

Resolución de trabajo	Tiempo	VMAF
1920x1080 (lo que hace)	13,1 s	referencia
1440x810	13,8 s	86,2
1080x608 (al ancho de salida)	9,1 s	87,4
720x405	9,5 s	74,7

Bajarla al ancho de salida da un **31 % de velocidad y cuesta unos 8 puntos de VMAF**. Hacerlo grande y reducir después actúa de **supermuestreo** y sale más nítido, así que en el montaje bueno no compensa. En el borrador sí: ahí lo que se mira es el ritmo, no la nitidez.

server/rotulos.js — diseños y subtítulos con resalte

Los rotulos salian siempre igual —dorado, con remate, abajo— escrito a fuego. Ahora hay seis diseños (`documental` , `noticias` , `minimal` , `impacto` , `manuscrito` , `tecnico`) que definen fuente, cuerpo, colores, borde, alineacion y efecto de entrada.

Los colores en ASS son `&HAABGGRR` : van **al revés** que en HTML y con la transparencia delante, donde `00` es opaco. Es el error mas comun escribiendo `.ass` a mano, asi que hay una funcion `ass()` que convierte desde RGB normal.

Subtítulos con resalte. Un `.srt` pinta la linea entera igual; en vertical lo que funciona es ver marcada la palabra que se esta diciendo. `buildSubtitulosAss()` genera un `.ass` con una linea de dialogo **por palabra activa**: se repite el grupo entero y solo cambia de color la que toca.

Se probó con `\k` (el karaoke de libass) y el relleno progresivo queda mas vistoso pero **se lee peor**: el ojo persigue el borde en movimiento en vez de la palabra.

La sincronia es aproximada, y hay que decirlo. Aqui no hay alineamiento forzado: nadie ha escuchado el audio para saber en que milisegundo empieza cada palabra. El tiempo se reparte por longitud, con un extra en las palabras que llevan puntuacion pegada porque ahi se respira. Cuadra en frases normales y se desvia con cifras y siglas, que se leen mas despacio de lo que su longitud sugiere. Para seguir la frase con la vista sobra; para doblaje no valdria.

El `.srt` plano **sigue saliendo aparte**: es el que se sube a YouTube como pista.

Zona segura

Las apps tapan parte del video con su interfaz: en vertical, TikTok se come alrededor del 20 % de abajo y YouTube el 10 % de arriba. Los margenes de rotulos y subtítulos salen de `ZONA_SEGURA` y **se aplican segun donde este anclado el texto**: si el rotulo va arriba manda el margen de arriba.

Caretas

`render.careta` y `render.cierre` , **opcionales**. Se pegan **al final del todo**, cuando el episodio ya esta montado con sus rotulos y subtítulos quemados.

Meterlas antes obligaria a desplazar TODOS los tiempos del `.ass`, del `.srt` y de los capitulos, que es exactamente donde se rompe la sincronia. Pegadas al final, el episodio se monta igual que sin ellas.

`normalizarCaret()` las adapta al lienzo: **encaja sin recortar** —es material acabado del usuario y comerse los bordes le estropearia el diseno—, iguala fps, y **si no traen audio les pone silencio**, porque un tramo sin pista de audio en medio de un concat desincroniza todo lo que viene detras.

Los capitulos se desplazan lo que dure la cabecera, o apuntarian a un sitio equivocado.

Transiciones

Entre las **imagenes de un mismo plano** se puede elegir cualquiera de las de `xfade`: fundido, disolucion, barridos, circulos, pixelado.

Entre **planos** solo hay fundido (a negro o a blanco) y corte seco. **No hay encadenado real entre planos, y no es un olvido**: obligaria a solapar los segmentos, y aqui la duracion de cada plano la manda la locucion. Solaparlos desincronizaria el episodio entero.

`server/buscar.js` — buscar y reemplazar

Busca en narracion, rotulos, prompts, camara y notas. Sin `regex`, lo tecleado se toma **literal**: quien busca "1.2" espera "1.2" y no "132", que es lo que pasaria dejando el punto suelto.

El reemplazo **siempre se simula primero** y avisa aparte de los planos cuya voz ya esta generada: cambiar el texto **no cambia el audio**, y ese plano seguiria sonando como antes. Es el fallo mas facil de cometer aqui, asi que va en la respuesta y no en una nota al pie.

`server/costes.js` — cuanto va a costar

Ya habia control de gasto y tope diario, pero uno se entera **despues**. Esto lo dice antes.

Los precios **cambian** y dependen del plan contratado. Los de aqui son ordenes de magnitud publicos, redondeados **hacia arriba** a proposito: mas vale que asuste de mas y luego salga barato. No son una factura: sirven para decidir si merece la pena. Lo que de verdad gastas lo dice la consola de cada proveedor.

Plantillas

Guardan `providers`, `models`, `style`, `voice`, `render`, `freepik` y `localMedia`. **No guardan claves de API ni `sourceDir`**: eso es de la maquina, no de la serie, y meterlo haria que una plantilla compartida apuntara a carpetas que no existen en el otro ordenador. Se aplican **sobre** lo que hay, no sustituyendo, para que una plantilla antigua no borre ajustes nuevos.

`scripts/comprobar-ui.js`

`node --check` solo dice que un fichero **parsea**. Un `getElementById` sobre un id que no existe parsea perfectamente y luego tumba el resto del arranque de la interfaz: la pantalla se queda a medias y el error se queda en una consola que nadie abre.

Este script carga la pagina de verdad en Electron, recorre **las ocho pestanas**, escucha la consola y los errores no capturados, y comprueba que existan los elementos y funciones esperados. Se planta con codigo 1 si algo falla.

```
PORT=4399 NO_OPEN=1 node server/index.js      # consola 1
env -u ELECTRON_RUN_AS_NODE npm run comprobar-ui  # consola 2
```

Pruebas del motor de montaje

`node test-montaje.js` (27 comprobaciones, no gasta API: el material lo fabrica ffmpeg). Vigila, por orden de importancia, que **la duración del segmento sale exacta** —todo el programa se apoya en que el plano dura lo que dura la locución—, que ninguna combinación se cuelga, y que los tres encajes y los dos tipos de fuente producen un MP4 válido. Con `--bench` mide además el encaje `blur`. | Otro guion / otra serie | Ajustes → *Carpeta del guion*. El parser es genérico. | Cambiar el look de los rótulos | `buildTitlesAss()` en `ffmpeg.js` | Nuevos movimientos de cámara | `cameraMove()` y `kenBurnsFilter()` | Transiciones con `xfade` | Sustituir el concat demuxer por un `filter_complex` encadenado | Vertical / Shorts | Ajustes → Montaje: 1080×1920, y `aspect 9:16` en Estilo |

11. El generador de guiones (`generator.js`)

El principio de diseño: el generador escribe exactamente el formato de marcadores que el parser sabe leer. Por eso todo lo de aguas abajo —planos, imágenes, voz, montaje— funciona sin una línea de cambio, tanto si el guion lo escribió la IA como si lo escribió una persona.

11.1 Las tres fases

1. CONDENSAR solo si la fuente supera ~22.000 palabras
map: resumen denso por trozos de 55k caracteres
reduce: se concatenan los resúmenes
2. ESCALETA una llamada -> JSON con título, logline, biblia de estilo,
títulos SEO y N episodios con sus bloques
3. GUIONES una llamada por episodio -> markdown con marcadores

Un libro no se manda entero por dos razones: no cabe en la ventana y, aunque cupiera, salen guiones peores (el modelo se pierde). Trocear y resumir da mejor resultado y cuesta menos.

11.2 Normalización defensiva

Los modelos se desvían del formato: escriben `**[Voz]:**`, `[IMAGEN]`, `[Rotulo]`, o envuelven todo en un bloque de código. `normalizeScript()` repara los desvíos conocidos con expresiones regulares antes de escribir el fichero, y garantiza que la cabecera `# EPISODIO n –` existe y lleva guion largo. Sin esto, un episodio de cada tres se perdería.

11.3 Derivados

El guion de podcast **no se genera aparte**: se deriva del de vídeo (misma narración, sin `[IMAGEN IA]`, y los `[RÓTULO]` pasan a `[DECIR FECHA]` porque en audio las fechas se dicen). Los documentos 03 y 04 se extraen del propio guion. Una sola llamada por episodio produce los cinco ficheros.

12. Extracción de texto (`extract.js`)

Sin dependencias: solo `zlib`.

Formato	Cómo
TXT · MD · RTF	Lectura directa (el RTF se limpia con regex)
HTML · XHTML	<code>stripMarkup()</code>
DOCX	Es un ZIP → <code>word/document.xml</code> → <code>stripMarkup()</code>
EPUB	Es un ZIP → <code>META-INF/container.xml</code> → OPF → spine (orden real de capítulos)
PDF	No se parsea. Se manda al modelo como documento adjunto

El lector ZIP (`readZip`) recorre el directorio central, soporta los métodos 0 (almacenado) y 8 (deflate) —los únicos que usan DOCX y EPUB— y **normaliza** `\ a /` en los nombres de entrada, porque algunas herramientas de Windows (`Compress-Archive` , entre otras) escriben separadores no estándar.

Sobre el PDF: un extractor casero de PDF es frágil (streams comprimidos, codificaciones de fuente, texto en columnas). Gemini y Claude lo leen nativamente y con mucha mejor calidad, así que el PDF viaja como `inline_data` / `document` en la propia llamada. OpenAI no lo admite por esta vía y devuelve un mensaje que lo explica.

13. Clonado de voz (`voices.js`)

1. El navegador graba con `MediaRecorder` (webm/opus), con `echoCancellation` , `noiseSuppression` y `autoGainControl` **desactivados**: esos filtros mejoran una videollamada pero estropean una muestra para clonar.
2. El servidor normaliza cada muestra con ffmpeg: `highpass=f=70` (quita retumbe), `afftdn` (reducción de ruido) y `dynaudnorm` (nivel constante), a WAV mono 44,1 kHz.
3. `POST /v1/voices/add` de ElevenLabs con **multipart construido a mano** (`buildMultipart`), para no arrastrar una dependencia por una sola llamada.
4. La voz creada se guarda en `data/voz/muestras.json` y se selecciona automáticamente como narrador.

Los cinco párrafos de lectura cubren registros distintos a propósito —enunciado largo, interrogación, enumeración de cifras, tensión dramática y cierre cálido—: la variedad prosódica de la muestra pesa más en la calidad del clon que la duración total.

14. Multi-proyecto

```

data/
  settings.json
  current.txt          slug del proyecto activo
  voz/                 muestras de voz (compartidas entre proyectos)
  fuentes/             ficheros de origen subidos
  projects/<slug>/
    guion/              los .md generados
    project.json        estado
    assets/epNN/        images · videos · voice · music · build
  output/<slug>/        entregas

```

El proyecto activo se resuelve en `projectRoot()` . El `data/project.json` de las primeras versiones se sigue leyendo como «proyecto inicial» para no romper instalaciones existentes.

15. Empaquetado a .exe

`npm run build ( build.js )` produce `dist/GeneradorContenido/` con el ejecutable, `bin/` (ffmpeg + ffprobe), `docs/` , `.env.example` y un `LEEME.txt` .

Se usa **@yao-pkg/pkg** (el fork mantenido de `pkg`), destino `node22-win-x64` .

Tres detalles que hay que respetar:

- 1. **Rutas.** Con `process.pkg` activo, `__dirname` apunta al sistema de ficheros virtual del ejecutable. `config.js` distingue `ROOT` (carpeta del `.exe` , donde viven `.env` , `data/` y `output/`) de `BUNDLE_ROOT` (dentro del ejecutable, donde viven `public/` y `docs/`).
- 2. **Estáticos.** `express.static` no sirve de forma fiable desde el FS virtual de `pkg`. `serveBundled()` lee con `fs.readFileSync` , que sí está parcheado, y bloquea el salto de directorio. `/assets` y `/output` siguen con `express.static` porque son ficheros reales y el vídeo necesita peticiones por rango.
- 3. **ffmpeg.** No hay `node_modules` en el ejecutable. `locateBinary()` busca por orden: variable de entorno → `bin/` junto al `.exe` → `node_modules` → `PATH` .

`pkg` se invoca con `process.execPath` sobre `lib-es5/bin.js` , **no** con el shim `.cmd` : con `shell:true` una ruta con espacios se parte y el comando falla.

16. Cambios de API absorbidos

Los proveedores mueven el suelo con frecuencia. El programa está preparado:

Qué pasó	Cómo se absorbe
Imagen 4 retiró <code>negativePrompt</code>	<code>callPredictTolerant()</code> detecta el 400, quita el parámetro citado y reintenta
<code>imagen-4.0-*</code> dejó de servirse a cuentas nuevas	Migración automática de ajustes (<code>DEAD_MODELS</code>) a <code>gemini-3-pro-image</code>
429 por falta de facturación	<code>isHardQuota()</code> no reintenta y <code>explain()</code> da la instrucción concreta
Freepik reorganiza endpoints	Las rutas son campos de texto en Ajustes
Google renombra Veo / TTS	Los modelos son campos de texto en Ajustes

17. Lectura de páginas web (`web-read.js`)

Convierte una URL en Markdown limpio **y en un catálogo de fotos reales**. Sin dependencias: extractor propio, al estilo del modo lectura de un navegador.

17.1 Texto

`bloquePrincipal()` prueba primero las etiquetas semánticas (`<article>`, `<main>`, `role=main`). Si no las hay, puntúa cada `<div>` / `<section>` por densidad de texto en párrafos y penaliza los que se llaman `menu`, `sidebar`, `footer`, `cookie` ... `esProsa()` descarta después las líneas que no son prosa (celdas de fichas técnicas, listas de navegación).

`datosEstructurados()` lee el JSON-LD: si la tienda publica el producto ahí, es la mejor fuente —precio, marca, disponibilidad, valoraciones— sin adivinar nada.

17.2 Fotos

`extraerImagenes(html, baseUrl, contenidoHtml)` devuelve candidatas ordenadas por `peso`:

Origen	Peso
<code>og:image</code>	100
JSON-LD (<code>image</code> del producto/artículo)	95
<code>twitter:image</code>	90
<code></code> dentro del contenido	50
<code>background-image</code> en el estilo	30

Filtros: `IMG_BASURA` (sprite, icon, logo, avatar, badge, pixel, tracking, visa, gravatar, favicon...) sobre la URL y sobre el `alt`; descarte por `width` / `height` declarados (`<300×200`); resolución de carga diferida vía `data-src`, `data-srcset`, `srcset`, `data-lazy-src`, `data-original`.

`descargarImagenes()`:

- **Pide la versión grande.** `versionGrande()` reescribe `/250px-` → `/1024px-` (patrón Wikimedia) y `nombre-800x600.jpg` → `nombre.jpg` (patrón WordPress). Una miniatura de 250 px no aguanta el zoom del Ken Burns en 1080p.
- **No hace ráfagas.** 350 ms entre peticiones y reintento con `Retry-After` en 429/503. Sin esto, Wikimedia devolvía 429 a partir de la quinta imagen y se perdían 19 de 22.
- Descarta lo que no es `image/*`, los SVG (casi siempre iconos), lo que pesa menos de 15 KB y lo que pasa de 25 MB.

17.3 Visión: que la IA mire las fotos

`describirImagenes()` pasa cada foto descargada por la capacidad `vision` y pide un JSON `{vista, tipo, sirve}`. Tres en paralelo.

Por qué existe: el `alt` de una foto suele estar vacío. En la página de la penicilina de Wikipedia lo estaba en las **22 candidatas**. Sin visión, el guionista recibiría un catálogo de líneas en blanco y elegiría al azar.

Con visión, el catálogo que se le enseña al modelo dice *«placa de Petri con colonia de moho verde y halo transparente»*, y las reglas del guion le piden explícitamente que **la narración de cada plano hable de lo que hay en esa foto**.

`sirve: false` descarta lo que la vista reconoce como logo, banner o anuncio: basura que el filtro por nombre de fichero no pillaba.

Tres decisiones defensivas, todas por fallos reales:

1. **maxTokens: 1500 , no 400.** Los modelos que razonan gastan parte del presupuesto pensando; con 400 la respuesta llegaba cortada a mitad de frase y `JSON.parse` reventaba en las tres imágenes.
2. **leerRespuesta() rescata lo que haya:** quita el vallado ````json`, recorta entre la primera `{` y la última `}`, y si el JSON viene truncado saca el campo `"vista"` con una regex. Si el modelo contesta en prosa, esa prosa ya describe la foto y se usa tal cual.
3. **Un fallo de visión nunca tira la imagen.** Se queda con el `alt` y sigue. Sin proveedor de visión configurado (`providers.disponible('vision')`), la función devuelve las fotos intactas y no se llama a nadie.

17.4 Del catálogo al plano

1. `routes.js` (`/projects/create`, rama `url`) junta las candidatas de todas las páginas, deduplica por URL y ordena por `peso`.
2. Dentro del **trabajo** (no en el handler HTTP: tarda) descarga hasta 14 y las describe.
3. `brief.imagenesWeb = [{alt, vista, tipo, url}]. scriptRules()` cambia el marcador a `[IMAGEN WEB]` e imprime el catálogo numerado con las descripciones.
4. Tras `importFromSource()`, `colocarFotosWeb()` traduce cada `webImageIndex` a un fichero real, lo guarda con `project.saveAsset(ep, 'images', ...)` y rellena `selectedImage`, `imagePrompt` (lo que se ve) y `webImageOrigen`.
5. Un plano cuyo número esté fuera de rango coge la siguiente foto sin usar, antes que quedarse en blanco.

Prueba real (Wikipedia · penicilina · formato «vídeo corto vertical», 21 s): 22 candidatas → 14 descargadas → 8 planos, de los que **7 usan foto real**. El octavo es `[IMAGEN IA]` porque el guion menciona el melón podrido del que salió la cepa buena y en la página no hay ninguna foto de eso: exactamente el respaldo previsto.

18. Duración real vs duración pedida

Un vídeo de 21 s salía de **30,8 s**. El cálculo de palabras era una regla de tres con las palabras por minuto y **no descontaba el silencio de cada corte**: ocho planos con los márgenes de documental (0,35 + 0,65 s) son 8 segundos de silencio, casi la mitad del vídeo.

Dos cambios:

- `plataformas.MARGENES` ata el aire al ritmo del formato. En `corto`: `padStart 0,10`, `padEnd 0,22`, `minShotDuration 1,2`, `transitionDuration 0,18`. Los cortes van secos, que además es el lenguaje del formato. Se aplican solos al crear el proyecto, junto con la orientación.
- `palabrasPara()` descuenta `planosPara(segundos, ritmo) × (padStart + padEnd)` antes de convertir a palabras, con un suelo del 50 % del tiempo para que nunca se quede sin narración.

Pedido	Ritmo	Planos	Palabras	Duración estimada
15 s	corto	5	39	15,0 s
21 s	corto	7	55	21,1 s
30 s	corto	10	78	29,9 s
600 s	normal	24	1455	600,0 s
900 s	documental	26	2039	899,9 s

Medido en el ejecutable: el mismo TikTok de 21 s pasó de **30,8 s a 24,3 s** (1080×1920). Sigue 3 s largo porque el modelo escribió 62 palabras en vez de 55 y la voz de Gemini va algo por debajo de 175 ppm, pero entra en el rango bueno de las cuatro redes cortas.

19. Que el video dure lo que se ha pedido

Dos fallos distintos, uno encima del otro. El sintoma que se veia era el mismo: pedir un TikTok y recibir un video de cinco minutos.

19.1 La interfaz mandaba siempre 600 segundos

`aplicarFormato()` rellenaba `cEpisodes` y un campo oculto `cMinutes` con `salida.minutes`. **Ningun formato define minutes** —todos definen segundos—, así que `s.minutes || 10` valia siempre 10. Y `pintarDuraciones()` estaba definida pero **no se llamaba desde ningun sitio**: el `<select id="cDuracion">` se quedaba vacío, `Number('')` daba 0, y el envío caía al respaldo `cMinutes * 60 = 600 segundos, elijas lo que elijas`.

Se corrigio asi:

- `aplicarFormato()` llama a `pintarDuraciones(formato)`, que repinta el desplegable con la franja del formato y deja marcada su duracion propia.
- El campo oculto `cMinutes` **se ha eliminado**. La duracion sale solo del desplegable.
- `FRANJAS` limita lo que se ofrece: `corto` 15-300 s, `normal` 90-1800 s, `documental` 300-3600 s. No es cosmetica: si en "video corto vertical" se puede elegir 20 minutos, alguien lo elige.
- Por encima de `TOPE_REDES` (60 s) en un formato corto, la nota se pone en color de aviso y explica el rango que rinde.

Leccion: esto se me escapo porque probe la API (`POST /api/projects/create` con `segundos: 21`) y no el formulario. La API funcionaba perfectamente. El fallo estaba entre el formulario y la API. Las pruebas de la interfaz (`test-ui-duracion.js` , Playwright sobre Chrome del sistema, interceptando `**/api/projects/create` para leer el cuerpo sin gastar API) recorren los 9 formatos y comprueban duracion por defecto, opciones ofrecidas, el aviso y el cuerpo que se envia.

19.2 El modelo se pasa del presupuesto de palabras

Arreglado lo anterior, un TikTok de 21 s salia de **32,7 s**: el modelo habia escrito **112 palabras** cuando el presupuesto era 55. Decirle "ni una mas" en el prompt no sirve — los modelos cuentan mal.

`ajustarDuracion()` lo comprueba de forma determinista despues de `normalizeScript()`:

1. `palabrasHabladas()` cuenta **solo las lineas** `[VOZ]`, que es lo unico que se locuta. Los prompts de imagen en ingles y los rotulos no suenan y no deben contar.
2. Si se pasa del margen (8 % en formatos cortos, 15 % en el resto), se hace **una** llamada de recorte diciendo el numero exacto de palabras que sobran, con la orden de mantener todos los marcadores y quitar palabras, no ideas.
3. Se reintenta como mucho una vez mas.
4. **Salvaguarda:** si el recorte se deja planos por el camino o vuelve vacio, se descarta y se queda el guion original. Antes un video largo que uno roto.

Medido de punta a punta por la interfaz del ejecutable, formato `redes` a 21 s:

	Palabras habladas	Video final
Antes (fallo de la interfaz)	—	~5 min
Solo con la interfaz corregida	112	32,7 s
Con el recorte	54	18,0 s (1080×1920)

20. Por que los textos salian en telegrama

Sintoma real, subtítulos de un video de producto de 21 s:

```

Mi hijo se cayo de la litera.      Las camas altas dan miedo.
A ras de suelo, sin ayuda.        Anade patas y crece con el.
No parece mueble de nino.         Quedan pocas unidades disponibles.
  
```

Siete frases sueltas, sin ninguna relacion entre ellas. **No era culpa del modelo: era del reparto.**

20.1 La causa: 7,9 palabras por plano

`planosPara(21, 'corto')` daba un plano cada 3 segundos = **7 planos**, y el presupuesto era de 55 palabras. Eso son **7,9 palabras por plano**. Como cada plano lleva su propio `[VOZ]`, el modelo no tiene sitio para una frase completa: solo le cabe un fragmento. Escribia exactamente lo unico que se podia escribir.

Cambiado a un plano cada 4,5 s (`Math.max(3, Math.min(9, segundos / 4.5))`):

Duracion	Planos antes	Planos ahora	Palabras/plano antes	Ahora
15 s	5	3	7,8	13,7
21 s	7	5	7,9	11,4
30 s	10	7	7,8	11,6
45 s	12	9	10,0	13,7

El corte sigue siendo rapido —cada 4,5 s en pantalla es ritmo de red social—, pero ahora cabe una frase de verdad.

20.2 Las reglas empujaban al mismo sitio

Decian literalmente "*Frases de una linea. Sujeto, verbo y al grano*", que es una receta para el telegrama. Sustituido por reglas que exigen lo contrario:

- El guion es **un texto seguido**, no una lista: cada frase encadena con la anterior y la frase de un plano puede terminar en el siguiente.
- Se incluye el **ejemplo real del fallo** como contraejemplo ("ASI NO") junto a la version buena ("ASI SI"). Los modelos corrigen mucho mejor con un contraejemplo concreto que con una instruccion abstracta.
- Prohibido el relleno ("en un mundo donde...", "todos sabemos que...").
- Concreto antes que generico: "*aguanta 200 kilos*" dice algo, "*gran calidad*" no.

20.3 Y algo mas grave: se inventaba datos

En ese guion aparecia "**Quedan pocas unidades disponibles**" y "**Reservala antes de que se agote**". Esos datos **no estaban en el material de origen**: se los invento el modelo. No es un problema de estilo, es una afirmacion comercial falsa, y la urgencia inventada es publicidad enganosa.

Regla nueva, en todos los formatos:

NO TE INVENTES NADA. Ni precios, ni descuentos, ni unidades que quedan, ni plazos, ni garantias, ni valoraciones, ni premios, ni cifras. Si el dato no esta en el material de origen, NO se dice.

20.4 El recorte tambien empujaba al telegrama

`ajustarDuracion()` (§19.2) pedia "*quita palabras, frases mas secas*", que sobre un texto ya corto lo acaba de romper. Ahora pide **quitar ideas enteras, no palabras sueltas** —mejor tres cosas bien contadas que siete a medias— y comprueba explicitamente que el resultado siga sonando a alguien contando algo.

21. Formato del episodio en el momento de montar

La orientacion estaba solo en *Ajustes > Montaje*, como valor global. Para sacar un episodio en vertical habia que ir a Ajustes, cambiarlo, volver, montar y deshacerlo.

Ahora hay un desplegable **Formato del episodio** junto al boton *MONTAR EPISODIO*: 16:9 horizontal, 9:16 vertical, 1:1 cuadrado, los dos y los tres. En blanco usa el de Ajustes, que es el comportamiento de siempre.

- `studio.renderEpisode()` ya aceptaba `options.orientation`; solo faltaba la interfaz.
- La ruta **valida** el valor contra una lista blanca. `ff.formatosDe()` cae a horizontal ante cualquier cosa desconocida, y montar veinte minutos en la orientacion equivocada sin avisar es peor que fallar.
- La orientacion va en la etiqueta del trabajo (`Montaje EP1 (vertical)`): con varios montajes del mismo episodio en la cola, si no se distinguen no sirve de nada.

22. La orientacion era GLOBAL: un proyecto reconfiguraba a los demas

El sintoma: en una serie documental, las imagenes eran 16:9, los clips 16:9, y el MP4 salia vertical con la imagen encajada en el centro entre barras borrosas.

22.1 La prueba

Dimensiones reales de los MP4 entregados, ordenados por hora:

Cuando	Que se monto	Salio
30/07 08:29–10:04	penicilina, cielo azul, flamencos, camas	1080×1920 vertical
30/07 11:37–12:04	documental EP02, EP03, EP04	1080×1080 cuadrado
30/07 12:43	carburo	1920×1080 horizontal
31/07 11:41	documental EP01	1920×1080 horizontal

La orientacion seguia **la hora, no el proyecto**. Tres episodios del mismo documental salieron cuadrados porque justo antes se habia creado un proyecto de podcast (cuya `salida.orientation` es `cuadrado`).

22.2 La causa

En `POST /projects/create` habia esto:

```
config.saveSettings({ render: { orientation: salidaFormato.orientation, ... }, style: { aspect: ... }
});
```

`settings.json` es **uno solo para todo el programa**. Cada proyecto que se creaba reescribia la orientacion, el aspecto de las imagenes y los margenes de ritmo para TODOS los demas. Crear un TikTok dejaba en vertical el documental que tenias a medias, y quien mandaba siempre era el ultimo proyecto creado.

Peor todavia: como las imagenes ya se habian generado antes en 16:9, al montar sobre un lienzo 9:16 entraba `decideFit()` y las metia con fondo borroso. Todo "funcionaba"; el resultado era inservible.

22.3 La correccion

La configuracion de salida es **del proyecto**, no del programa:

- `project.json` guarda `salida: {orientation, aspect, ritmo, burnSubtitles, formato}`, escrita al crear el proyecto y conservada al reimportar.
- `studio.getSettings()` (interno) devuelve los ajustes globales **con la salida del proyecto superpuesta**, solo en los campos que son suyos. Todo `studio.js` ya lo usaba por ese nombre, asi que el cambio es de una sola funcion.
- `POST /projects/create` **ya no escribe ajustes globales**.
- Prioridad al montar: `options.orientation` (selector del episodio) → `project.salida.orientation` → `settings.render.orientation`.
- Los proyectos anteriores no tienen `salida`: caen a la global y se comportan como siempre.
- `project.summary()` expone `salida`, y el selector del episodio muestra el formato real de la serie — (el de esta serie: horizontal 16:9) —. Que no se viera en ninguna parte es lo que permitio que tres episodios salieran cuadrados sin que nadie lo notara hasta abrir el MP4.

Comprobado con `test-salida-proyecto.js`, que trabaja sobre una carpeta de datos aparte y **no gasta ninguna llamada de IA**: crea un documental, crea un TikTok despues, vuelve al documental y verifica que conserva orientacion, aspecto y ritmo; que los ajustes globales quedan intactos; y que un proyecto sin `salida` sigue usando la global.

23. Encaje: cuando recortar y cuando rellenar

Despues de arreglar §22 seguia saliendo mal, y por otro motivo: **los clips de video estaban generados en vertical** aunque el lienzo ya era 16:9. En el montaje entraban con fondo borroso a los lados, ocupando un tercio de la pantalla.

23.1 Por que los clips salieron verticales

`generateShotVideo()` pide el clip con `settings.style.aspect`. Y **la mayoría de los formatos no declara `aspect`**: solo lo hacen los de redes (`redes`, `producto`). Un documental solo declara `orientation: 'horizontal'`. Así que la proporcion caía a la global, que estaba contaminada en 9:16 por §22.

Corregido en `studio.getSettings()`: si la salida del proyecto no trae `aspect`, **se deriva de la orientacion**.

salida.orientation	Se piden imagenes y clips en
horizontal	16:9
vertical	9:16
cuadrado	1:1
both / todos	16:9 (el principal; los otros se sacan encajando)

Comprobado con la proporcion global puesta a 9:16 **a proposito**: los cuatro tipos de proyecto piden la suya y ninguno hace caso de la global.

23.2 `auto` era simetrico y no debia serlo

`decideFit()` devolvía `blur` para cualquier diferencia mayor del 12%. Eso trata igual dos casos que no se parecen en nada:

- **Vertical dentro de horizontal** → rellenando, el contenido queda en una franja central que ocupa **un tercio** de la pantalla y los dos lados son fondo borroso. Se ve pobre y se desperdicia el video. **Ahora recorta.**
- **Horizontal dentro de vertical** → recortando, de un 16:9 solo sobrevive el **32% del ancho**, lo que parte por la mitad casi cualquier encuadre. Además en redes el fondo desenfocado es el lenguaje normal del formato. **Se sigue rellenando.**

La perdida es la misma en ambos sentidos (32% de lo que habia); lo que cambia es que en un caso el recorte da un resultado utilizable y en el otro no. Por eso la regla es asimetrica a proposito.

También se recorta cuando fuente y destino tienen **la misma orientacion** y la diferencia es menor del 35% (4:3 en 16:9, por ejemplo): recortar quita poco y llena la pantalla, mientras que el fondo borroso ahí canta mucho.

Fuente	Lienzo	Antes	Ahora
9:16	16:9	blur	cover
16:9	9:16	blur	blur
4:3	16:9	blur	cover
16:9	16:9	cover	cover
9:16	1:1	blur	blur

23.3 Bonus: la pantalla de Ajustes prometia algo que el codigo no cumplia

El texto decia "*Las capturas de pantalla nunca se recortan*", pero la comprobacion de `kind === 'screenshot'` estaba **despues** de la preferencia manual:

```
if (preferido && preferido !== 'auto') { return preferido; } // <- salia por aqui
if (kind === 'screenshot') { ... } // <- no llegaba nunca
```

Poner "Llenar recortando" en Ajustes recortaba tambien las capturas, que es exactamente lo que no puede pasar en un video de ayuda: se pierde parte de la interfaz que se esta explicando. La proteccion va ahora la primera.

24. Sistema multiidioma (`idiomas.js`)

24.1 Por que el idioma de Ajustes no hacia nada

Habia **dos cosas distintas que no se hablaban**:

- En *Crear* un campo de TEXTO LIBRE (`español de Espana`) que iba a `brief.language` y era lo unico que miraba el generador.
- En *Ajustes* unos desplegables con CODIGOS (`es` , `en`) en `settings.languages` , que solo usaban el doblaje y los subtítulos.

Cambiabas el idioma en Ajustes, el generador seguia leyendo el campo de texto y todo salia en castellano. No era un fallo de guardado: es que apuntaban a sitios distintos.

Ahora manda **un codigo**: `brief.idiomaPrincipal` . De ahi se saca el texto que ve el modelo (`translate.idioma(code).prompt`). El campo de texto libre ha desaparecido.

24.2 Los idiomas son del PROYECTO

Misma leccion que con la orientacion (\$22): en `settings.json` habrian sido globales y el ultimo proyecto creado le habria cambiado el idioma a los demas. Viven en `project.json` :

```
"idiomas": {
  "principal": "es",           // en el que ESCRIBE la IA
  "adicionales": ["en", "fr"], // a los que se traduce
  "conVoz": ["es"],           // en cuales se locuta (cuesta)
  "conVideo": ["es"],         // en cuales se genera video (lo mas caro)
  "conSubtitulos": ["es","en","fr"],
  "viendo": "es"              // lo que se esta editando ahora
}
```

Prioridad, que es lo que pidio el usuario y coincide con el coste real:

	Se hace en	Motivo
TEXTO	todos los idiomas, al crear	traducir es barato
IMAGEN	se comparte	la imagen no habla
VOZ	solo los marcados	cuesta por caracter
VIDEO	solo los marcados	lo mas caro con diferencia

24.3 Donde vive cada traduccion

El idioma **principal NO se duplica**: sigue en los campos de siempre (`shot.narration` , `shot.titles` , `shot.voiceFile`). Los demas van en `shot.i18n[codigo]` . Asi un proyecto de un solo idioma queda byte a byte como antes y no hay migracion que hacer.

`versionPlano(shot, lang, principal)` resuelve cual toca. Es la unica funcion que sabe de esa asimetria; el resto del programa pide "la version en X" y no se entera.

24.4 La duracion cambia con el idioma

`buildTimeline(episode, lang)` calcula la linea de tiempo **por idioma**, porque la duracion de cada plano la marca su locucion y una frase no dura lo mismo en espanol que en aleman. El mismo episodio puede durar 10 s en un idioma y 15 en otro; comprobado en la prueba en frio.

El MP4 lleva el codigo en el nombre salvo el principal: `EP01_TITULO.mp4` , `EP01_TITULO.en.mp4` .

25. Lo demas que se ha anadido

25.1 Imagen de partida ("usar como base")

`generateShotImages({ base, instruccion })` . `base` puede ser `'seleccionada'` (la imagen que ya tiene el plano) o la ruta de cualquier material del proyecto, incluida una que subas en ese momento.

Con imagen de partida el prompt **cambia de naturaleza**: ya no describe una imagen nueva, es la instruccion de que hacerle a la que hay. Sin instruccion se asume "el mismo plano con el estilo de la serie".

En el proveedor de Gemini la imagen va **primero** en las partes del mensaje y el texto detras: asi el modelo la trata como material y el texto como orden. Los modelos de la familia `imagen-*` son de texto a imagen y no la mirarian, asi que con `base` se fuerza un modelo `gemini-*` .

25.2 "Mejorar con IA" en todo

`studio.mejorarConIA({ campo, usarActual })` cubre narracion, rotulos, prompt de imagen, prompt de video, escena y titulo. Un solo camino con una receta por campo (`MEJORAS`), en vez de una funcion por campo que se irian separando con el tiempo.

`usarActual: false` es "propon otra cosa": se le da el contexto del plano pero **no** el texto actual, para que no se limite a parafrasear. La propuesta se ensena antes de aplicarla — la IA acierta a menudo, pero perder un texto bueno por un boton es muy molesto.

25.3 Ampliar y rehacer episodios

- `ampliarEpisodio()` escribe planos NUEVOS al final. Se le pasa lo que ya hay para que no repita y enlace. Se reutiliza el parser del kit, asi que no hay dos gramaticas.
- `regenerarEpisodio()` rehace imagenes, voz y montaje de una pasada. **Respetar los planos fijados** (`locked`) por defecto: es donde el usuario ya decidio algo a mano.

25.4 Marca de agua / logotipo

Se compone en el mux final con `filter_complex` (no `-vf`: hay que mezclar dos flujos de video). El logo se escala a un **porcentaje del ancho**, asi que el mismo PNG vale para 16:9 y para 9:16.

La marca del EPISODIO manda sobre la del PROYECTO, para poder poner un patrocinador distinto en un episodio suelto. Si el fichero ya no existe se avisa y se monta sin el; si el filtro falla, el rescate parte de cero (el `-map [v]` del filtro roto no sirve).

25.5 Reiniciar el programa

En la version de escritorio **el servidor corre dentro del proceso de Electron**, asi que `process.exit()` cerraria el programa y no lo volveria a abrir. El boton usa `app.relaunch()` por IPC. En modo navegador si sale del proceso y sondea `/api/health` hasta que vuelve a responder, en vez de esperar un tiempo fijo.

Hay tambien un **Recargar proyectos** que solo vacia el cache en memoria: para cuando se copian proyectos a mano en la carpeta de datos.

25.6 Podcast

- Con "**generar siempre el podcast**" marcado, la version sonora se **reescribe con IA** en vez de derivarse quitando las lineas de imagen. Un guion de video sin sus imagenes deja frases que no se entienden ("aquí vemos...") y pierde los datos que iban en rotulos.
- `POST /episode/:n/podcast/desde-texto` convierte un documento subido —o texto pegado— en guion de podcast: frases mas cortas, siglas deletreadas, cifras escritas como se dicen, respetando el contenido.

25.7 Formatos

De 9 a 29. Los nuevos: entrevista, noticia, resumen de libro, caso de estudio, receta, viaje, cuento infantil, corporativo, anuncio, testimonio, preguntas frecuentes, comparativa, historia personal, lista/top N, explicativo, meditacion, audiolibro, mini documental, trailer y resumen periodico. Cada uno con su `salida` para que la interfaz se autoconfigure.

26. Acotaciones de dialogo (`acotaciones.js`)

El sintoma. En los guiones de dialogo aparece `(Rie)`, `[suspira]`, `*baja la voz*`. Eso llegaba tal cual a ElevenLabs y el locutor lo LEIA: se oia "parentesis rie".

Por que. El modelo por defecto (`eleven_multilingual_v2`) no entiende etiquetas de audio; para el son texto normal. Las etiquetas (`[laughs]`, `[sighs]`) solo las actuan los modelos `*_v3`.

La solucion. Un modulo que separa lo que se DICE de lo que se HACE, y dos caminos segun el modelo:

Modelo	Camino
<code>*_v3</code>	La acotacion se traduce a su etiqueta de audio y se antepone a la linea. El modelo la actua.
resto	Se QUITA del texto y se convierte en un delta sobre <code>voice_settings</code> de esa linea.

Detalles que importan:

- **Tres formas** aceptadas: `(...)`, `[...]`, `*...*`. Cada IA que escribe guiones usa la suya.
- **Limite de 60 caracteres.** Un parentesis largo casi nunca es una acotacion: es texto del guion que hay que locutar (`Roma cayo en 476 (segun los manuales)`). Cortar por longitud + catalogo cerrado de senales evita comerse frases enteras.
- Los deltas de tono se **acumulan y se recortan** al rango valido. `speed` se limita a 0.7–1.2 porque fuera de ahi ElevenLabs devuelve error de validacion.
- Una linea que es SOLO una acotacion se queda sin texto. Mandar cadena vacia a la API es un 400: se devuelve un MP3 de silencio del largo del gesto, generado con ffmpeg en memoria.
- `acotaciones.palabras()` sustituye al conteo crudo en `palabrasHabladas()`: una acotacion no se locuta, luego no ocupa tiempo, luego no puede contar para el presupuesto de palabras.

27. Cache de voz global (`cachevoz.js`)

El sintoma. Locutar el episodio para el VIDEO y despues exportar el PODCAST cobraba dos veces exactamente el mismo texto con las mismas voces.

Por que. Cada camino guardaba su audio en su carpeta y con su nombre. Ninguno veia el del otro. Habia ocho sitios distintos que locutaban y cada uno cacheaba a su manera (o no cacheaba).

La solucion. Una cache en el PROYECTO, enganchada en `providers.call('voice', ...)`, que es el unico punto por el que pasan todas las locuciones. Poner el control en cada ruta era justo lo que fallaba: siempre quedaba una sin enganchar.

La firma es `sha1(texto + voz + proveedor + modelo + idioma + ajustes redondeados)`.

- Los **ajustes se redondean a 2 decimales** antes de firmar. Sin eso, `0.5` y `0.5000000001` (que salen de sumar los deltas de las acotaciones) darian firmas distintas y no acertaria nunca.
- `previo` / `siguiente` **NO entran en la firma** a proposito: cambian la entonacion un pelo, pero incluirlos significaria que corregir una coma en el bloque 3 invalida tambien el 2 y el 4.
- Limpieza por **fecha de ultimo uso**, no de creacion: la sintonia que suena en veinte episodios tiene que sobrevivir aunque se grabara el primer dia.
- Si algo falla (sin proyecto abierto, sin permisos), se locuta y ya esta. La cache es un extra, nunca un requisito.

Se ve y se vacia desde la barra de Episodios (`GET /api/voz/cache` , `POST /api/voz/cache/vaciar`).

28. Traducir: el JSON que llegaba cortado

El sintoma. La IA no devolvio un JSON valido al traducir. Empieza por: { .

Por que. `translateMany` mandaba **12 bloques FIJOS** por llamada, contando bloques y no contenido, con `maxTokens: 8000` fijo. En un documental un bloque son 30 palabras y no pasa nada; en un podcast conversacional un bloque son 500, asi que doce bloques eran ~6000 palabras de entrada y otras tantas de salida. El modelo cortaba la respuesta a mitad, el JSON llegaba **empezado pero sin cerrar** —de ahí el "Empieza por: {"— y `JSON.parse` lo rechazaba entero, perdiendo tambien las traducciones que SI estaban completas.

Tres arreglos, en este orden:

1. `repartirEnTandas()` mide en **palabras** (tope 900) ademas de en bloques (tope 12). Manda el limite que se alcance antes. Un bloque que por si solo pase del tope va en su propia tanda: partirlo descuadraria la correspondencia uno a uno.
2. `maxTokens` se calcula sobre la entrada (`palabras * 4.5 + 800` , techo 32000) en vez de ser fijo. Traducir ocupa mas que el original en casi todos los idiomas.
3. `leerJson()` tolerante: quita vallas ``json,` extrae el primer `,` y si sigue sin parsear ****rescata por regex los pares** "clave": "valor" que si estan cerrados**. Quien llama ve que faltan claves.

Y una red debajo: si faltan bloques, `traducirTanda()` **parte la tanda en dos y reintenta cada mitad** (hasta 4 niveles). Repitiendo, cualquier bloque acaba yendo solo en su llamada, que es el caso en el que es imposible que no quepa. Un bloque que aun asi falle se queda en el idioma original y se avisa en el log, en vez de tumbar el episodio entero.

29. Varios kits en un proyecto: anadir episodios en vez de reemplazar

El sintoma. "Importar kit de otra IA" **reemplazaba** el proyecto en vez de anadir el episodio 2.

Dos causas distintas:

1. La ruta `/projects/importar-kit` **siempre creaba un proyecto nuevo** (`slug + Date.now()`).
2. Aunque no lo hiciera, `importProject` solo sabia leer UN kit: `localizarKit()` busca el mejor `.md` de todo el arbol y se queda con uno. Copiar un segundo kit en `guion/` habria hecho que uno de los dos guiones desapareciera.

La solucion.

- Cada kit se copia en **su propia subcarpeta** de `guion/` con una marca `_kit.json` que guarda su numero de episodio, el nombre y la carpeta de origen.
- `importProject()` mira si hay marcas. **Si no las hay, se comporta exactamente como antes** (`leerUnKit`): un kit normal viene repartido en subcarpetas (`01_YOUTUBE/` , `02_PODCAST/`) y trocearlo por carpetas romperia todos los que ya funcionan. Solo se parte con marcas explicitas.
- `unirKits()` junta los episodios resolviendo choques de numeracion: si el numero esta cogido, se pasa al primero libre. Perder un episodio por un choque es mucho peor que verlo reenumerado.
- `renumerar()` cambia tambien el prefijo de los ids de plano (`ep02_s07`), que es lo que ata cada imagen y cada locucion a su plano.

- Un proyecto antiguo con los `.md` sueltos en `guion/` se **recoge** a su propia carpeta la primera vez que se le anade un segundo episodio (`recogerKitSuelto`), respetando su numeracion original.

`numeroDesdeNombre()` deduce el numero del nombre de carpeta: `EP02`, `T2E05`, `capitulo-3`, `PARTE 7`, `episodio_12`. El separador se escribe a mano (`(?:^[^A-Za-z0-9])`) y **no con** `\b`, porque el guion bajo cuenta como letra: en `LA_OTRA_LINEA_EP02` no hay ningun limite de palabra delante de `EP` y `\bEP` no encuentra nada — que es justo el nombre de carpeta de los kits reales. `EP2026` se descarta por el tope de 300: es un ano, no un episodio.

Se deduce del **nombre de la carpeta y no de la cabecera del guion** a proposito: practicamente todos los kits generados por IA empiezan por `# EPISODIO 1` aunque sean el septimo.

30. Borrar un episodio (`DELETE /api/episode/:n`)

Se lleva por delante tres cosas, y las tres hacen falta:

1. El episodio de `project.json`.
2. **La carpeta del kit que lo genero.** Sin esto la siguiente reimportacion lo resucita y parece que el borrado no ha funcionado.
3. Sus assets — **solo si se pide** (`?assets=1`). Por defecto se conservan: casi siempre se borra un episodio para volver a importarlo corregido, y rehacer treinta imagenes cuesta dinero.

Si el episodio venia de un kit suelto (proyecto de un solo episodio, sin marca), no se toca el guion —borrarlo dejaria el proyecto sin nada— y se devuelve un `aviso` para decirlo.

31. "Se queda parado" en el montaje: no lo estaba, es que no informaba

El sintoma. El montaje se planta en `Plano 7/9 – 1.7 (image)` y no se mueve.

La causa. `run()` buscaba el avance con un regex sobre `time=` del stderr. Esa linea SOLO la escribe ffmpeg con el nivel de log por defecto, y aqui se usa `-loglevel warning`: **el progreso no llegaba nunca**. Dentro de un plano no habia ni una senal. En un documental daba igual —cada plano son 8 segundos— pero un podcast conversacional montado en video tiene planos de tres y cuatro minutos, y ahi la pantalla se queda quieta un cuarto de hora.

Los arreglos:

1. `-progress pipe:1 -nostats` en todas las llamadas. Escribe `clave=valor` en la salida estandar independientemente del nivel de log; se lee `out_time_us` y se traduce a porcentaje y a tiempo restante estimado. El buffer puede partir una linea en dos, de ahi el acumulador `resto`.
2. La etiqueta del plano lleva ahora su DURACION (`Plano 7/9 – 1.7 (image, 3:24)`), para que se vea de antemano que ese plano va a tardar.
3. **Vigilante de bloqueo.** El corte NO va por tiempo total —un plano puede tardar minutos de verdad— sino por tiempo SIN AVANZAR: si ffmpeg no emite nada en 4 minutos, se le mata y se dice que casi siempre es una imagen corrupta en ese plano. Antes no habia ningun timeout: se podia quedar colgado para siempre.

El coste del Ken Burns: cuidado con medir a la resolucion equivocada.

Primera medicion, a **1280x720**: 60 s tardaban 19,8 s con Ken Burns y 19,6 s sin el. Conclusion —equivocada— "no cuesta nada, manda el codificador". A esa resolucion el filtro cabe de sobra en el tiempo que tarda x264 y la diferencia se diluye.

Repetida a **1920x1080**, que es como se monta de verdad:

Plano (1080p, medium, crf 19)	Con Ken Burns	Sin Ken Burns
60 s	30,9 s	20,6 s

Un **50 % mas**. La leccion: medir a la resolucion real, no a una comoda.

El criterio para quitarlo no es la duracion, es la VELOCIDAD del movimiento. Lo que se percibe no es el zoom total sino cuanto cambia el encuadre por segundo. Un 16 % en 8 s son 2 %/s y se ve; el mismo 16 % en 3 min y medio son 0,076 %/s y no lo ve nadie. Por debajo de `MIN_ZOOM_POR_SEGUNDO = 0.15` el zoompan no aporta nada y cuesta ese 50 %, asi que se quita y se dice en el log con el numero exacto.

Con el zoom por defecto el umbral cae en planos de mas de ~107 s. **En un documental no salta nunca.** Salta en un podcast conversacional montado en video, donde cada bloque son tres o cuatro minutos sobre una sola imagen — que es justo el caso donde el montaje se hacia eterno. Medido: un plano de 210 s pasa de ~108 s a 72,5 s.

Otra idea probada y descartada. Codificar los segmentos intermedios (los que se recodifican despues al quemar subtítulos) con `ultrafast` en vez de `veryfast`. Medido a 1080p: 60 s pasan de 31,5 s a 29,7 s (un 6 %), pero el temporal pasa de 3,9 MB a 21,6 MB. En un episodio de media hora son 650 MB de temporales para ahorrar un minuto. No compensa.

32. Traducir: las tandas van en paralelo

Al pasar el reparto de lotes de "12 bloques" a "900 palabras" (ver el apartado 28) salen mas tandas que antes, y se lanzaban una detras de otra. Cada tanda es una llamada independiente —no comparten contexto— asi que encadenarlas solo servia para esperar.

Ahora van **de tres en tres**. Tres y no mas: por encima de eso los proveedores empiezan a devolver 429 y se tarda mas por los reintentos. Una tanda que falle del todo no tumba a las demas: esos bloques se quedan en el idioma original y se avisa en el log.

33. Video: el catalogo de modelos (`videomodelos.js`)

Hasta la version 0.5.0 el precio del video era **uno por proveedor** (`costes.js`) y el modelo un campo de texto libre sin ninguna pista. Eso escondia el mayor gasto evitable de todo el programa: por animar **la misma imagen**, con el mismo prompt, Veo 3.1 estandar cuesta 0,40 €/s y Veo 3.1 Lite 0,05 €/s. Ocho veces. Un episodio de 8 clips de 8 segundos pasa de 25,60 € a 3,20 € cambiando una cadena de texto que nada en la pantalla invitaba a cambiar.

`server/videomodelos.js` es la tabla de verdad: **por modelo y por resolucion**. Cada entrada lleva el proveedor que lo sirve, la **familia** (como se le habla), el precio por segundo en cada resolucion, las duraciones que acepta y si genera audio.

De ahi beben cuatro sitios:

Quien	Para que
<code>costes.js</code>	La estimacion previa usa el precio del MODELO, no el del proveedor, y si hay uno mas barato del mismo proveedor lo dice
<code>studio.js</code>	Escribe en el registro lo que va a costar el clip antes de pedirlo, y lo guarda en el plano (<code>costeEstimado</code>)
<code>providers/*.js</code>	Redondea la duracion pedida a una que ese modelo acepte, y elige el cuerpo de la peticion segun la familia
<code>public/app.js</code>	Ajustes ensena el precio por segundo junto a cada modelo y lo que sale un clip de 8 s con lo que hay puesto

Un modelo que no este en el catalogo no rompe nada: se estima con el precio alto del proveedor y se avisa de que es una estimacion. El campo sigue siendo libre.

33.1 `auto` : que la API elija el barato

El valor de fabrica de *Gemini · video* es `auto`. Con el, `resolverVeo()` pregunta a `GET /v1beta/models` que modelos `veo-*` puede usar **esa clave** y se queda con el primero de `VEO_PREFERENCIA` (Lite → Fast → estandar) que exista. La lista se cachea una hora.

Asi el programa se pone al dia **solo**: cuando Google saque un Veo nuevo y lo anada a la lista de la clave, aparece sin tocar codigo ni publicar version. Es la misma idea que ya habia con los nombres de modelo como campo de texto, pero sin tener que enterarse el usuario.

33.2 Los modelos caros de fabrica se cambian al arrancar

`config.migrate()` tenia ya una tabla de modelos RETIRADOS (`DEAD_MODELS`). Se le ha anadido otra de modelos **caros de fabrica** (`VIDEO_CARO_DE_FABRICA`): si el ajuste guardado es **exactamente** el que venia de serie en una version anterior —es decir, si el usuario nunca lo toco— se cambia por el barato equivalente y se avisa por consola. La comparacion es con `===` a proposito: lo que alguien haya elegido a mano no se toca jamas.

```
veo-3.1-generate-preview      -> auto
fal-ai/kling-video/v2/master/image-to-video  -> minimax/h3-max-turbo/image-to-video
```

33.3 Cada familia habla distinto

No hay un esquema de entrada comun, y el fallo tipico no da error: da un 422 que solo se ve pagando. Lo que espera cada una:

Familia	Duracion	Resolucion	Formato	Audio
veo (Gemini)	<code>durationSeconds</code> numero, solo 4 / 6 / 8	<code>resolution</code> 720p / 1080p / 4k	<code>aspectRatio</code>	siempre, no se puede apagar en esta API
h3 (MiniMax)	<code>duration</code> numero , 5 a 15	<code>resolution</code> 480P / 768P	solo en texto a video	si
ltx	<code>duration</code> texto de una lista cerrada	<code>resolution</code> 1080p / 1440p / 2160p	—	<code>generate_audio</code>
klings	<code>duration</code> texto , "5" o "10"	—	<code>aspect_ratio</code>	mudo (los v3 cobran +50 % con audio)
veofal	<code>duration</code> "8s"	<code>resolution</code>	<code>aspect_ratio</code>	<code>generate_audio</code> , y sin audio cuesta la mitad

`fal.entradaVideo()` monta el cuerpo de cada una y esta **exportada a proposito:** `test-video.js` la comprueba entera sin red y sin gastar un centimo.

33.4 El video de fal va por COLA

`fal.run` es sincrono: la conexion se queda abierta hasta que el clip esta hecho. Con Kling (dos o tres minutos) se cortaba por el camino y, como fal cobra por peticion, el reintento **pagaba dos veces**. Ahora el video va por `queue.fal.run` : devuelve un `request_id` al instante y se sondea el estado. Se informa del puesto en la cola. La imagen y la musica siguen siendo sincronas porque tardan segundos.

33.5 Parametros que Veo retira, y el filtro de contenido

Dos cosas que aparecieron al probarlo de verdad contra la API el 7/9/2026, y que en las dos el sintoma era una pantalla de error inutil:

1. **`negativePrompt` ya no existe en Veo 3.1.** La API responde `400 · "negativePrompt isn't supported by this model"` . Antes el negativo de las imagenes se heredaba al video y **dejaba muerto el boton de animar**. Ahora `postTolerante()` lee del mensaje que parametro sobra, lo quita y repite; un 400 se contesta antes de generar nada, asi que reintentar no cuesta dinero. Es el mismo patron que ya usaba `grok.js` . El `prompt` y la imagen no se sacrifican nunca.
2. **El filtro de contenido bloquea por el AUDIO.** Veo devuelve `done: true` , sin video, con un `raiMediaFilteredCount` y el motivo `«we encountered an issue with the audio for your prompt»`. Lo que se veia era `Veio termino sin video utilizable` con 400 caracteres de JSON detras. Ahora `motivoFiltrado()` lo traduce a una frase con la solucion dentro. **Y esa llamada ya se ha cobrado**, asi que la pista tiene que llegar a la primera.

Por eso los clips se piden **mudos** por defecto (`geminiVideoAudio: 'no'`): el montaje descarta el audio del clip de todas formas —el plano dura lo que dura la locucion y el clip se repite en bucle— y ademas es lo que mas veces dispara el filtro.

Ojo con lo que se dice de vuelta. La API de Gemini **no admite** `generateAudio` (solo Vertex AI), asi que `postTolerante()` lo quita y el clip llega **con** banda sonora aunque se pidiera mudo. Por eso devuelve tambien la lista de lo que quito: si no, el programa anotaria «sin audio» en un fichero que lo lleva. Comprobado con `ffprobe` : pista `aac` presente.

34. Aviso de version nueva (`actualizaciones.js`)

No hay actualizador automatico y no lo va a haber: `electron-updater` necesita instaladores **firmados** y un servidor de actualizaciones; sin certificado, en Windows salta el SmartScreen igual y en macOS ni arranca.

Lo que si se puede hacer sin nada de eso es preguntar a la API publica de GitHub cual es la ultima release, compararla con la version que corre y enseñar un aviso con el enlace en Ajustes. Una peticion cada seis horas, sin clave (GitHub admite 60 por hora sin autenticar), con ocho segundos de espera maxima. **Si no hay red o GitHub no contesta, no se pinta nada**: un aviso de actualizacion no puede ser motivo de que una pantalla no cargue.

La comparacion es por numero, no por texto: `0.10.0` es posterior a `0.9.0`.

📺 **Generador de Contenido** **From a book (or just an idea) to a finished YouTube episode, without opening a video editor.**
 [[License: AFL-3.0]](https://img.shields.io/badge/License-AFL--3.0-blue.svg)](LICENSE) [[Node.js]](https://img.shields.io/badge/Node.js-%E2%89%A518.17-339933?logo=node.js&logoColor=white)](https://nodejs.org) [[Windows · macOS · Linux]](https://img.shields.io/badge/Windows%20%C2%B7%20macOS%20%C2%B7%20Linux-0078D4?logo=windowsterminal&logoColor=white)](#installation) [[Dependencies]](https://img.shields.io/badge/dependencies-3-brightgreen.svg)](#how-its-built) [[Local]](https://img.shields.io/badge/your%20API%20keys-never%20leave%20your%20machine-success.svg)](#privacy) AI writes the scripts, generates the images, clones your voice, and assembles the episode as an MP4 with titles, subtitles and chapters. All on your own machine. [Installation](#installation) · [How it works](#how-it-works) · [AI providers](#ai-providers) · [Docs](#documentation) **English · [Español](README.md)
 ![Four shots from an episode assembled by the program](docs/img/demo.gif) **This isn't a mock-up: it's a real, complete episode made with the program.** Script written by AI, images generated, voice cloned, assembly automatic. ### ▶ [Full episodes on YouTube](https://www.youtube.com/@podcastElOcasoRomano)

Made with this

[imagen no encontrada: docs/img/muestras.jpg] />

Four different episodes of the same series. The **entire channel** — scripts, voice, images and assembly — comes out of this program: [youtube.com/@podcastElOcasoRomano](https://www.youtube.com/@podcastElOcasoRomano)

The episodes are in Spanish, but the program writes in whatever language you ask it to.

What it is

A local application (Node server + web UI) that covers the **whole production chain** of a YouTube channel or a podcast:

```

a book · a PDF · an EPUB · an article · or just a topic
    ↓
👉 the AI writes the scripts
(video + podcast + image/video prompts + series bible)
    ↓
🖼️ images    🗣️ your cloned voice    📺 clips    🎵 music
    ↓
🎬 assembled MP4 + SRT + YouTube chapters
  
```

It is **not** a one-click video generator. It's a **studio**: at every step you decide — you pick which of the three proposed images you want, you edit the prompt, you regenerate what doesn't convince you — but you never touch a timeline or nudge a single frame.

The idea that makes it work

The voiceover sets the length of every shot. The narrator takes 43 seconds to read their paragraph → that shot lasts 43 seconds plus the lead-in and lead-out silences. There is nothing to line up by hand, and nothing drifts out of sync: measured drift across a 7½-minute episode is **46 milliseconds**.

And if you don't generate video — which is by far the expensive part — the shots **don't sit still**: a Ken Burns move is applied that reads the camera direction out of the script itself. "dolly-in" pushes in, "pan left to right" sweeps, "tilt up" rises. Free, and it looks like a proper documentary.

Installation

Option A — Download the app

Everything is bundled: **Node.js and ffmpeg included**, nothing else to install. Grab the one you need from [Releases](#):

OS	File	Notes
Windows	GeneradorContenido-*-x64.zip	Start here. Unzip and run <code>Generador de Contenido.exe</code> . Least likely to upset antivirus software
	GeneradorContenido-*-x64.exe	Installer, with shortcuts
	GeneradorContenido-portable-*.exe	Single file, installs nothing
macOS	GeneradorContenido-*-mac-arm64.dmg	Apple Silicon (M1 and later)
	GeneradorContenido-*-mac-x64.dmg	Intel
Linux	GeneradorContenido-*-linux-x86_64.AppImage	Works on any distro. <code>chmod +x</code> and run
	GeneradorContenido-*-linux-amd64.deb	Debian, Ubuntu and derivatives
	GeneradorContenido-*-linux-x64.tar.gz	Extract and run, installs nothing

Every release ships a `SHA256SUMS.txt` if you want to verify what you downloaded.

macOS: "the application is damaged"

It isn't. Signing a Mac app costs **\$99/year**, and without a signature macOS quarantines it. One command clears the flag:

```
xattr -cr "/Applications/Generador de Contenido.app"
```

Linux: the AppImage

```
chmod +x GeneradorContenido-*.AppImage
./GeneradorContenido-*.AppImage
```

Install the DejaVu fonts (`fonts-dejavu` on Debian/Ubuntu) so burned-in titles and subtitles get a decent typeface. The app only reaches for them on Linux, where Georgia and Arial don't ship by default.

Windows: SmartScreen and antivirus

SmartScreen will warn you the binary isn't code-signed (a certificate runs €200–400/year). **More info** → **Run anyway**.

Some antivirus products flag the installer. ESET, for one, deletes it claiming `Backdoor.PHP/Webshe11.NBV`. It's a false positive, and here's the check rather than the promise — run with ESET's own command-line scanner (`ec1s.exe`):

What was scanned	Result
Source code (<code>server/</code> , <code>public/</code> , <code>electron/</code>)	0 detections
Unpacked build (<code>win-unpacked</code>), 610 objects	0 detections
Freshly built installer, portable and ZIP	0 detections
Searching the <code>.exe</code> and the <code>app.asar</code> for <code><?php</code> , <code>shell_exec</code> , <code>passthru(</code> , <code>base64_decode</code> , <code>eval(\$_</code>	0 matches
<code>.php</code> files inside the package	none

ESET's verdict came prefixed with `@` , which marks a **cloud reputation (LiveGrid)** call, not a signature match. A freshly compiled, unsigned self-extracting executable has no reputation, so it gets lumped in with malware samples. And "webshell" describes exactly what the program genuinely does — it starts a local HTTP server and shells out to `ffmpeg` .

Your options, cheapest first: **use the ZIP** (not self-extracting, doesn't trip the heuristic) · **build it yourself** (the source is all here) · add an exclusion · or report the false positive to your vendor, which helps everyone.

Option B — From source

```
git clone https://github.com/ecomyseo/generador-de-contenido.git
cd generador-de-contenido
npm install
npm start          # opens the desktop app
```

Needs Node.js ≥ 18.17. `ffmpeg` comes in as an npm dependency.

```
npm run server      # engine only, drive it from a browser
npm run doctor      # checks ffmpeg, API keys and script parsing
npm run test        # the test suites – no API calls, no cost

npm run build       # build for Windows
npm run build-mac   # for macOS
npm run build-linux # for Linux
```

Each OS has to be built on that OS. The macOS installer needs `hdiutil`, which only exists on macOS, and `ffmpeg-static` only downloads the binary for the machine it's installed on. So you don't need three computers, `.github/workflows/build.yml` does it for you: push a `v*` tag and GitHub builds all three and drafts the release.

Setting up API keys

```
cp .env.example .env
```

Fill in **only the ones you'll use** — none are required, and the program starts with none. You can also paste them in **Settings** inside the app without touching files.

`.env` is gitignored. **This repository ships no keys**, and the app's own API never returns a key's value — only whether one exists.

How it works

1 · Create the series

[imagen no encontrada: docs/img/ui/01-crear.jpg] />

The **Create** tab. Pick where the content comes from:

Source	What it does
A topic	The AI writes the series from its own knowledge
A file	PDF, DOCX, EPUB, TXT, MD, HTML, RTF
Pasted text	An article, some notes, a chapter
A URL	Blog post, product page, article. Navigation and banners are stripped... and the page's own photos are used
A code folder	Reads a program's docs and writes its walkthrough video

Then what it should be: **documentary, YouTube video, video essay, narrative podcast, course, vertical short** (TikTok · Reels · Shorts), **product video, software manual** or **audio fiction**. Plus episode count, duration, language, audience and tone.

Each format configures itself: orientation, resolution, typical length, subtitles and cutting rhythm. Duration is asked for in **seconds**, and the program works out how many words actually fit — discounting the silence around each cut — so a 21-second TikTok runs 21 seconds and not 31.

The photos come from the page, not from the AI

When the source is a URL, the shots use the **real photos from that page**. For a product page that's the difference between a video that works and one that doesn't.

And before writing a word, **the AI looks at every photo**. Alt text is usually empty — in a Wikipedia test run it was empty on all 22 images — so without seeing them it would be writing blind. With vision it knows photo 4 is "*a scientist in a white coat examining a Petri dish*", picks which one belongs in each shot, and **narrates what's actually visible**. Anything that reads as a logo or a banner gets dropped. Only what doesn't exist on the page is generated with AI.

The AI produces the full kit:

00_README_KIT.md	index
01_GUION_YOUTUBE.md	video script with markers
02_GUION_PODCAST.md	audio version
03_PROMPTS_IMAGENES.md	per-shot image prompts
04_PROMPTS_VIDEOS.md	video prompts + camera moves
05_BIBLIA_PRODUCCION.md	tone, style, voice, music, pipeline

A whole book doesn't get sent in one go: it's summarised in parts, then the outline is designed, then episodes are written one at a time. PDFs go straight to the model — Gemini and Claude read them better than any extractor.

Already have your scripts? Great. Put them in the marker format ([VOZ] , [IMAGEN IA] , [RÓTULO] , [MÚSICA] , [ESCENA]) and hit *Import*. The generator is optional.

2 · Your voice

[imagen no encontrada: docs/img/ui/02-mi-voz.jpg] />

The **My voice** tab. You record five paragraphs in the browser — deliberately written to cover different registers: neutral narration, questions, numbers, tension and a warm close — and the program creates a **clone of your voice** on ElevenLabs that narrates the whole series.

60 seconds works; 2–3 minutes is noticeably better.

3 · Produce

The **Editor** tab. The buttons are numbered in the right order:

1 · Images → 1b · Upscale (Magnific) → 2 · Voice → 3 · Animate → 4 · Music → 5 · ASSEMBLE

- **Images** — 3 variants per shot; pick one with a click.
- **Upscale** — Magnific Upscaler. Matters more than it sounds: Ken Burns zooms into a still, and without spare resolution that zoom looks mushy. There's also **relight** and **style match** (imposes the look of an approved shot on the rest, so the series hangs together).
- **Voice** — narrates each shot. This is where the episode's real rhythm gets set.
- **Animate** — *optional*. Turns a still into a 5–8 s clip.
- **Music** — an instrumental bed that ducks itself under the narrator (sidechain).
- **Assemble** — MP4 + `.srt` + chapters + an `.ass` for titles.

👁 Preview: see it before rendering it

Assembling a 200-shot episode takes hours. Just to check whether an image fits what's being said over it, that's a terrible trade.

Preview shows each shot full-screen with its voiceover playing, and **encodes nothing** — it's instant. Since each shot lasts exactly as long as its audio (the same rule the real assembly uses), **the pacing you see is the pacing you'll get**.

Space pauses, ← → move, Esc closes.

Several images in one shot

A long narration block over a single photo drags. **Ctrl+click** the thumbnails to add images to a shot's sequence and they cross-fade between each other. The shot lasts the same: it's split between however many it has.

It warns you before assembling

Hit assemble and it checks the episode first, stopping if there are shots with no image, images generated but never picked, or text that was never voiced. You can confirm and assemble anyway — normal when you want to see a half-finished episode — but you no longer find out after eight hours that fourteen shots came out black.

Lip sync, which is where the trick shows

Video models, the moment they see a face, make it talk. Always. In a documentary the voice belongs to a **narrator who isn't on screen**, so what you got was a Roman moving his mouth while saying nothing, out of sync with a voice that isn't his. It fools you for half a second and then it screams.

The script already knows who speaks in every block, so the model just gets told:

In the script	What the video model is asked for
[VOZ] with no name, or [VOZ:NARRADOR]	Voice-over: every mouth closed and still . The face lives through eyes, blinking and breathing
[VOZ:BRUNO]	Lips, jaw and tongue in sync with Bruno's literal line , in the audio language

When two people talk, only whoever has the turn moves their mouth; the other listens with theirs closed.

And it adapts to the model. Veo and Grok Imagine generate audio, so they actually pronounce the line and can lip-sync to it — they get the literal quoted text. The Kling family (fal, Replicate, Freepik) is mute and truncates long prompts, so asking it for phoneme accuracy is wasted budget: it only gets *who is talking and who is silent*.

Turn it off in **Settings → Style → Lip movement** if you prefer the old behaviour.

Subtitles with the spoken word highlighted

What works in vertical: the word being said is highlighted, which holds attention far better than a static block of text. The plain `.srt` is still produced separately for YouTube.

Timing is **approximate**: it is worked out from how long each word takes to say, not by listening to the audio. Plenty for following along; not enough for dubbing.

And **titles now have six designs** — documentary, news, minimal, impact, manuscript, technical — each with its own font, colour, size and position.

Intro, outro and safe area

Drop in an MP4 for the front and another for the end; they adapt themselves to your episode resolution and frame rate. **Optional.**

And titles keep clear of where the apps put their own interface: in vertical, TikTok covers roughly the bottom 20% and YouTube the top 10%.

Find and replace

Ctrl+F, across narration, titles, prompts and notes. Replacement is previewed before it applies, and **it flags the shots whose voice is already recorded**: changing the text does not change the audio, so those have to be re-narrated.

Audio comes out at the level YouTube expects

Without touching anything. Platforms **normalise everything** you upload to about -14 LUFS: if your episode is quieter they turn it up, and the background hiss comes up with it; if it is louder they turn it down and it loses punch. It now arrives already in the right place.

And the Ken Burns move **eases in and out** instead of starting abruptly and stopping dead, which is how a real camera moves.

Quick draft

Assembles the episode **for real** but at low resolution and full speed: more than twice as fast in testing. It lasts exactly as long as the good render, so the pacing you see is the pacing you get.

Re-assembling costs seconds, not hours

Change one image in a shot and assemble again: **only that shot is redone**. A segment is deterministic — it comes from some images, a duration, a camera move and some settings — so a fingerprint of those inputs is stored, and if nothing changed it gets reused.

And if assembly is interrupted, it **picks up where it left off**: progress is written every ten shots rather than at the end, which is precisely the case that hurts.

It also **tells you how long it will take** before starting. The first time it says so is a guess; after that the number comes from your own previous runs.

And it flags shots that will drag

More than 90 seconds on a single still image is a photo sitting there for a minute and a half: past ~107 s the Ken Burns move is so slow it stops being perceptible. The pre-flight check points them out, and now there is a fix — split the shot across several images.

Pacing

Settings → **Assembly** → **Pacing** scales the silences around the voice: *brisk* for social, *unhurried* for narrated documentary, *solemn* for plenty of air. It is what changes the feel of an episode most. Individual shots can carry their own and they win.

Making it not take eight hours

One 200-shot chapter took **eight hours**. Measured where the time went, three things came out:

The encoder was hardcoded — everything went through `libx264 -preset medium`, which is excellent and also the slowest. But ffmpeg ships NVENC, Quick Sync and AMF compiled in **always**, whether or not the card and driver exist, so listing them proves nothing: you have to actually try them. It now does that at startup and picks the best one that really works.

Measured with `scripts/bench-codec.js` over 6 s of documentary footage at 1080p, on a dual-core Intel m3 laptop:

Encoder	Time	VMAF
<code>h264_qsv</code> (Intel GPU)	2.2 s	94.8
<code>h264_mf</code>	3.8 s	93.5
<code>libx264 -veryfast</code>	6.9 s	93.8
<code>libx264 -medium</code> ← <i>the old default</i>	20.6 s	96.3

VMAF is Netflix's metric: 95+ is indistinguishable, 90+ very good. **The GPU is 9.4× faster for a point and a half nobody sees.**

Shots were rendered one at a time — with 200 shots, the machine spent hours with a single core busy. They now run several at once, bounded by core count and, when encoding on GPU, by how many concurrent sessions the card allows: exceed that and the extra ones die mid-job.

The blurred background was recomputed every frame — 1,800 times per shot, from a still image, with identical results every time. Now once: 28.2 s → 14.0 s.

The three multiply. **Settings** → **Assembly** tells you what it found on your machine.

AI providers

Task	Providers	Recommended
Scripts and prompts	Claude · Gemini · OpenAI · Grok	Claude
Images	Gemini · Grok Imagine · Freepik (Mystic) · fal.ai (Flux) · Replicate · OpenAI	Gemini
Upscale / relight / style	Magnific (Freepik)	Magnific
Video	Gemini (Veo 3.1 Lite/Fast/standard) · fal.ai (MiniMax H3 , LTX, Kling, Veo) · Grok Imagine · Replicate · Freepik	Veo 3.1 Lite or MiniMax H3 Max Turbo
Voice and cloning	ElevenLabs · OpenAI TTS · Gemini TTS	ElevenLabs
Music	ElevenLabs Music · Replicate · fal.ai	ElevenLabs

[imagen no encontrada: docs/img/ui/03-ajustes.jpg] />

You choose a provider **per task** in Settings. **No key is mandatory**: it runs with whatever you have. Keys live in a local `.env` and never leave your machine.

It also speaks to models running **on your own computer**, which need no key and cost nothing: Ollama, LM Studio, Jan, llama.cpp/vLLM, LocalAI, Stable Diffusion WebUI, ComfyUI and Piper.

Model names and API paths are **editable text fields**. When a provider renames a model or moves an endpoint — and they do, often — you fix it in Settings without touching code.

What it costs

For a 10-episode series:

Item	Ballpark
Scripts (text AI)	cents
Images (3 variants × ~8 shots × 10 eps)	a few euros
Voiceover + cloning	ElevenLabs monthly plan (~€5–22)
Music	cents
Generative video	tens of euros per episode

Video costs more than everything else combined. **Assemble with Ken Burns first, watch it, and only animate the 2–3 shots that genuinely ask for it.** The trailer is where it pays off best.

Set spending limits in each provider's console. That's the only real safety net.

Documentation

Currently in Spanish.

Document	Who for
User manual	From zero to a published episode
Technical docs	Architecture, API, assembly engine
AI providers guide	What to sign up for and how

How it's built

server/	
index.js	Express bootstrap
config.js	.env + settings + paths (dev mode and packaged mode)
generator.js	source → outline → scripts (the whole kit)
extract.js	PDF/DOCX/EPUB/TXT → text (own ZIP reader, no dependencies)
web-read.js	a URL → clean Markdown, with its real images
labios.js	lip sync: who talks in each shot and who stays quiet
codecs.js	what this machine can encode (probed for real, not listed)
revision.js	whether the episode is ready, before spending hours
cachemontaje.js	skip shots that have not changed
estimar.js	how long it will take, learned from your own runs
rotulos.js	title designs and highlighted subtitles
buscar.js	find and replace across the whole episode
costes.js	what it will cost, before you press the button
parser.js	marked-up markdown → episodes and shots
project.js	on-disk state, multi-project, asset management
studio.js	production operations and assembly orchestration
ffmpeg.js	Ken Burns, concat, ducked mix, ASS titles, SRT
voices.js	recording, audio cleanup and voice cloning
jobs.js	background queue with SSE progress
routes.js	HTTP API
providers/	gemini · claud · openai · grok · elevenlabs · freepik · fal · replicate
public/	UI (HTML + JS, no framework, no build step)

Three dependencies: `express`, `ffmpeg-static`, `ffprobe-static`. That's it.

Adding a provider = one file in `providers/` exporting `{id, name, caps, ...methods}` and one line in the registry.

Tests

```
npm run doctor          # environment, keys and script parsing
npm test                # both suites below
node test-labios.js     # 62 checks: lip sync per provider, provider registration
node test-montaje.js    # 117 checks
node test-buscar.js     # 19 for find/replace and cost estimation
npm run comprobar-ui    # loads the UI in Electron and looks for errors: exact duration, fit modes,
                        # camera, multi-image shots,
                        # encoders and the pre-flight check
node scripts/bench-codec.js your-video.mp4 # which encoder suits YOUR machine
node test-render.js 1   # assembles a full episode from synthetic material, no API calls
```

`test-montaje.js` guards the thing everything else rests on: that a rendered shot comes out at **exactly** the requested duration. `test-render.js` builds a whole episode and **fails if sync drift exceeds 0.5 s** — it's the regression test for the assembly engine.

Privacy

- The server listens on `127.0.0.1` only. Not reachable from your network.
- Keys live in `.env` (gitignored). The API never returns a key's value, only whether it exists.
- **Nothing runs on its own.** No scheduled jobs, no automatic reprocessing: every paid call comes from a click of yours.
- Your voice recordings stay in `data/voz/`. They're only sent to ElevenLabs when you press *Create my cloned voice*.

Contributing

Bugs and ideas go to [Issues](#); there's a template for each.

If you send a pull request:

- **No new dependencies.** There is deliberate, and that's the point. If the Node standard library can do it, it does it.
- **A new provider** = one file in `server/providers/` exporting `{ id, name, caps, ...methods }`, plus one line in the registry in `providers/index.js`. Nothing else.
- **Before opening the PR**, run the tests: `bash npm test node test-render.js 1`. The second one fails on its own if sync drift goes over 0.5 s.
- **Never commit a** `.env`, a key, or a path off your own disk.

Spanish and English are both fine in issues and PRs.

Who makes it

Written by **Ecom Experts**. If you happen to work with e-commerce, my other corner of the internet is gmartos.es: modules, tutorials and help for **PrestaShop, in Spanish**. Unrelated to this, but there it is.

The channel where you can see the output: youtube.com/@podcastElOcasoRomano.

License

[AFL-3.0](#) · *Ecom Experts*

The AI services it connects to have their own terms and pricing. This program includes no keys and no credits.